
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 20:19:24
Файлов исходного кода: 82
Суммарный объём кода: 1.14 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs-bfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler-cycle]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-borushka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/82: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
```

Универсальное пособие • полный исходный код

```
-----
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+....
```

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

- ```
        if [ -f "$policy" ]; then
            sudo sed -i 's/rights="none" pattern="\{P
        fi
    done
    convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \public/export/full_code_all_pages"
 echo "| \public/export/full_code_all_pages"
 echo "| PNG-страниц | \$(ls tmp/export-pages
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7

```
    steps:
      - name: Checkout
      - uses: actions/checkout@v4
      + uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
      - uses: actions/setup-node@v4
      + uses: actions/setup-node@v7
        with:
          node-version: "22"
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload artifacts (PDF + TXT)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-export
          path: |
@@ -107,7 +107,7 @@ jobs:
          retention-days: 30

      - name: Upload artifacts (PNG-страницы)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-pages-png
          path: tmp/export-pages/*.png
    ....

## `index.html`
```

Универсальное пособие • полный исходный код

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

Patch относительно `main`

````diff

diff --git a/public/favicon.svg b/public/favicon.svg

new file mode 100644

index 00000000..3bbbba0

--- /dev/null

+++ b/public/favicon.svg

@@ -0,0 +1,5 @@

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

````

`src/App.tsx`

Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { chapters } from '../data/content';
import { Navbar } from '../components/Navbar';
import { Sidebar } from '../components/Sidebar';
import { MobileToc } from '../components/MobileToc';
import { ChapterView } from '../components/ChapterView';
import { ChapterNav } from '../components/ChapterNav';
```



```

<div className="min-h-screen bg-slate-950 text-slate-50" >
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab} />

 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 m-4" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0" >
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" >
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
 style={{ width: `${progress}%` }}
 />

```

### Patch относительно `main`

```diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/Sim

import { SimulatorHub } from "../components/SimulatorHub

+import { PythonCompiler } from "../components/PythonComp

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" |

+ const [activeTab, setActiveTab] = useState<"guide" |

const [activeChapterId, setActiveChapterId] = useSta

const [modalVizId, setModalVizId] = useState<string

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

-) : (

+) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

}}

/>

</main>

+) : (

+ <PythonCompiler />

}}

<SimulatorModal vizId={modalVizId} onClose={() =>

```
    <div className="min-w-0 flex-1">
      <h1 className="text-xl font-extrabold text-slate-800">
        Универсальное пособие
      </h1>
      <p className="text-xs text-indigo-400 font-extrabold">
        Подготовка к экзаменам: Алгоритмы, Структуры
      </p>
    </div>
  </div>
```

```
<div className="flex items-center w-full lg:w-auto" >
  >
  <button
    id="tab-guide-btn"
    onClick={() => setActiveTab('guide')}
    className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold cursor-pointer duration-200 whitespace-nowrap ${
      activeTab === 'guide'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </button>
  <button
    id="tab-simulator-btn"
    onClick={() => setActiveTab('simulator')}
    className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold cursor-pointer duration-200 whitespace-nowrap ${
      activeTab === 'simulator'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <Cpu className="w-4 h-4" /> Тренажёры
  </button>
  <button
    id="tab-compiler-btn"
    onClick={() => setActiveTab('compiler')}
    className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold cursor-pointer duration-200 whitespace-nowrap ${
      activeTab === 'compiler'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <Code className="w-4 h-4" /> Компилятор
  </button>
  <button
    id="tab-about-btn"
    onClick={() => setActiveTab('about')}
    className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold cursor-pointer duration-200 whitespace-nowrap ${
      activeTab === 'about'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <Info className="w-4 h-4" /> О проекте
  </button>
</div>
```

```
-----
        className="flex items-center justify-center
        er:bg-amber-600/20 border border-amber-500/
        title="Скачать всё пособие одним автономным
    >
        {busy ? <Loader2 className="w-4 h-4 animate
    </button>
    </div>
  </div>
</header>
);
};
....
```

Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

 export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
 >
 <Cpu className="w-4 h-4" /> Тренажёры
```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index-gz.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL, {
 integrity: PYODIDE_MODULE_HASH,
 })
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}
```

```

 }
 }, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>

 <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
 <section className="rounded-2xl border border-slate-200 p-4">
 <div className="flex flex-wrap items-center">
 <div className="flex items-center gap-2">


```

```

 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 p-4" >
 <label className="block px-4 pt-3 text-slate-500" >
 Ввод для input() • по одной строке на клавишу
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.target.value)}
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-transparent border border-slate-800"
 style={{border: '1px solid #333'}}
 />
 </div>
</section>

<section className="rounded-2xl border border-slate-800 p-4" >
 <div className="flex items-center justify-between" >
 <div className="flex items-center gap-2" >
 <Terminal className="w-4 h-4 text-emerald-500" />
 </div>

 {runtimeReady && <CheckCircle2 className="text-emerald-500" />}
 {runtimeReady ? "готов" : "не загружен"}

 </div>
 <pre className="min-h-[360px] max-h-[560px] border border-slate-800 p-4" >
 {output}
 </pre>
 <div className="border-t border-slate-800 p-4" >
 </div>
</section>
</div>

```

```

+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) =>
+ setStderr: (options: { batched: (message: string) =>
+ setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE
+ (module as PyodideModule).loadPyodide({ indexURL
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
+ const [code, setCode] = useState(STARTER_CODE);
+ const [stdin, setStdin] = useState("");
+ const [output, setOutput] = useState("Нажмите «Запус
+ const [running, setRunning] = useState(false);
+ const [runtimeReady, setRuntimeReady] = useState(fal

```



```

+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:items-center">
+ <div>
+ <div className="inline-flex items-center gap-2">
+ <Terminal className="w-4 h-4" /> Боковая панель
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-extrabold">
+ <p className="text-slate-400 mt-2 max-w-2xl">
+ Пишите небольшой код и запускайте его прямо в браузере,
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5"
+ >
+ Как это работает <ExternalLink className="text-slate-400" />
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+ <section className="rounded-2xl border border-slate-200 p-4">
+ <div className="flex flex-wrap items-center gap-2">
+ <div className="flex items-center gap-2">
+
+
+
+ </div>
+ <div className="flex items-center gap-2">
+ <button
+ type="button"
+ onClick={reset}

```

```

+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.target.value)}
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-transparent border-slate-200 p-2"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border border-slate-200 p-4">
+ <div className="flex items-center justify-between">
+ <div className="flex items-center gap-2">
+ <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+ </div>
+
+ <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+ {runtimeReady ? "готов" : "не загружен"}
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800 p-4">
+ </div>
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3">
+ <div className="flex gap-2 rounded-xl border border-slate-200 p-4">
+ <Info className="w-4 h-4 shrink-0 text-indigo-500" />
+ Ctrl или Cmd + Enter запускает код. L
+ </div>
+ <div className="flex gap-2 rounded-xl border border-slate-200 p-4">
+ <Info className="w-4 h-4 shrink-0 text-amber-500" />

```

```
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
 },
});
\\
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
  },
}
```

Универсальное пособие • полный исходный код

- Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы и классы элементов. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/82: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <title> Дискретка для тупых – Интерактивный гид по дискретной математике
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/82: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 байт

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "description": "Удобная web-читалка для подготовки к экзаменам",
  "requestFramePermissions": [],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```

Универсальное пособие • полный исходный код

```
-----  
    "@vitejs/plugin-react": "5.1.1",  
    "esbuild": "^0.28.2",  
    "tailwindcss": "4.1.17",  
    "typescript": "5.9.3",  
    "vite": "7.3.2",  
    "vite-plugin-singlefile": "2.3.0"  
  },  
  "description": "Интерактивное пособие по алгоритмам и  
}
```

ФАЙЛ 7/82: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

algv0 – Универсальное пособие по алгоритмам и структуре

Интерактивная web-читалка (React + Vite + Tailwind) с
и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash  
npm install
npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---------------------|
| `npm run export:txt` | код → txt | `public/export/ful` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/ful` |

Универсальное пособие • полный исходный код

известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор).
Словарь — `src/data/glossary.ts`, мини-анимации — `src/anim`.
* **Реестр интерактивов** — `src/components/vizRegistry` и для панели под главой, и для модальки из подсказки.

Структура

...

| | |
|--------------------|---|
| src/ | |
| App.tsx | — оболочка, привязка глав к визуализации |
| components/ | — интерактивные симуляторы (Действия) |
| data/ | — контент пособия (главы/билеты/карты) |
| scripts/export/ | — пайплайн код → txt → png → pdf |
| public/export/ | — сгенерированные TXT и PDF (обновляемые) |
| .github/workflows/ | — автоматизация |

...

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `

` и классы.
- * Подзаголовки: Используют тег `

`.

Текст и параграфы

- * Обычный текст содержится в тегах `

`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика — параграфы `

`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| | | |
|--------------------|-----------|----------------------|
| Глава | id | Что делать |
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| | | |
|---------------|--------------------------------|-----------------------------|
| Билет | Тема | Статус |
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. К |
| | ит вхолостую. | |
| **7** | Планарность и раскраска графов | Номерной глав |
| | огии**. | |
| **11** | Мосты в графах | Номерной главы нет; есть бе |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- * `stack-dfs` → `StackViz.tsx` – ****Стек и DFS****
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – ****Очередь и BFS****
- * `heap-beam-search` → `HeapViz.tsx` – ****Куча и лучевой поиск****
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – ****Дерево отрезков****
- * `dynamic-programming` → `DPVisualizer.tsx` – ****Динамическое программирование****

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

| | | |
|--|--------------|-------------|
| Глава | id | Комментарий |
| --- | --- | --- |
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e | |

Универсальное пособие • полный исходный код

| | | | | |
|----|--------------------------------------|---|---|---|
| 17 | 17. Графы. Алгоритм Джонсона | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - | - |
| 21 | 21. Префикс-функция (π), КМП | 4 | - | - |
| 22 | 22. Z-функция | 4 | - | - |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| - | Обзор: Кратчайшие пути и Графы | - | - | - |
| - | Алгоритмы на карте | - | - | - |
| - | Эйлер: Путь ≠ Цикл | - | - | - |
| - | Мосты в графах: код + визуализация | 1 | - | - |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)
это 5 готовых компонентов, которые сейчас просто не...
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш...
3. ****Добавить 2D**** к 4 главам «Уровня 4» – минимум inli...
4. ****Решить судьбу `alg-map`**** – раскрыть или удалить.

ФАЙЛ 9/82: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6...

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],
```



```
-----
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

=====

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

ФАЙЛ 11/82: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 KB

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
```

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
  <div className="min-h-screen bg-slate-950 text-slate
```

```
    <Navbar activeTab={activeTab} setActiveTab={setAc
```

```
    {activeTab === "guide" ? (
```

```
      <>
```

```
        <MobileToc
```

```
          selectedChapterId={activeChapterId}
```

```
          setSelectedChapterId={goTo}
```

```
          currentTitle={activeChapter.title}
```

```
          index={index}
```

```
          total={chapters.length}
```

```
        />
```

```
      <div className="flex flex-col lg:flex-row max
```

```
        {/* Сайдбар только на десктопе – на мобильном
```

```
        <div className="hidden lg:block lg:w-80 shr
```

```
          <Sidebar selectedChapterId={activeChapter
```

```
        </div>
```

```
      <main id="main-content" className="flex-1 m
```

```
        {/* Шапка главы с быстрыми переходами */}
```

```
        <div className="flex items-center justify
```

```
          <div className="min-w-0">
```

```
            <div className="text-[10px] uppercase
```

```
              {activeChapter.category || "Универс
```

```
            </div>
```

```
            <h2 className="text-base sm:text-xl f
```

```
              {activeChapter.title}
```

```
            </h2>
```

```
          </div>
```

```
          <ChapterNav prev={prev} next={next} ind
```

```
        </div>
```

```
      <div className="h-1 w-full bg-slate-800 r
```

```
        <div
```

ФАЙЛ 12/82: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
  const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l))
  // границы «не буква/цифра» – \b не работает с кириллицей
  return new RegExp(`(?<![\\p{L}\\p{N}]_)(?=${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * <span class="term-hint" data-term-id="...">, чтобы на
 * повесить всплывающую карточку (как превью ссылок в Википедии)
```

```

    const usedTerm = perTerm.get(id) ?? 0;
    const usedSurface = perSurface.get(surface) ?? 0;
    if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
      perTerm.set(id, usedTerm + 1);
      perSurface.set(surface, usedSurface + 1);
    }

    if (m.index > last) frag.appendChild(document.createElement("div"));
    const span = document.createElement("span");
    span.className = "term-hint";
    span.dataset.termId = id;
    span.tabIndex = 0;
    span.setAttribute("role", "button");
    span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
    span.textContent = m[1];
    frag.appendChild(span);
    last = m.index + m[1].length;
  }

  if (last === 0) continue;
  if (last < text.length) frag.appendChild(document.createElement("div"));
  if (textNode.parentNode?.replaceChild(frag, textNode)) {
    //
  }
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
  const run = () => {
    const el = ref.current;
    if (!el) return;
    try {
      annotateTerms(el);
    } catch {
      //
    }
  };
  useLayoutEffect(run, []);
}

```

```
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
  from {
    transform: translateX(-100%);
  }
  to {
    transform: translateX(0);
  }
}

@keyframes hintIn {
  from {
    opacity: 0;
  }
}
```

```
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
    white-space: pre-wrap;
    word-break: break-word;
}

.chapter-body pre {
    overflow-x: auto;
    max-width: 100%;
    font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
    display: block;
    width: 100%;
    overflow-x: auto;
    border-collapse: collapse;
    font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
    cursor: pointer;
    padding: 0.35rem 0;
}

@media (max-width: 767px) {
    .chapter-body :where(.grid) {
        grid-template-columns: minmax(0, 1fr) !important;
    }
    .chapter-body :where(h2) {
        font-size: 1.25rem !important;
    }
}
```

```
    from {  
      width: 0%;  
    }  
    to {  
      width: 100%;  
    }  
  }  
}
```

ФАЙЛ 14/82: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";  
import { createRoot } from "react-dom/client";  
import "./index.css";  
import App from "./App";  
  
createRoot(document.getElementById("root")!).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

ФАЙЛ 15/82: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {  
  id: string;  
  title: string;  
  type: "html" | "markdown";  
  content: string;  
  description?: string;  
  category?: string;
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
          ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные заготовки
            ь отрезок длины 7 (перекрывая друг друга по 1 единицу)
            ничего складывать, просто пересекаем куски.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
              Аналогия 2: Ступени
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
`
  }
]
```



```

        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
метода: Split и Merge.</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Удар катаной (Sp
        </div>
        <p class="text-slate-300 text-sm mb
> &gt; х». Спуск от корня: узел, который вме
ез вскрытия</b> – дальше досматривается тол
    </div>
    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
            Аналогия 2: Слияние капель (M
        </div>
        <p class="text-slate-300 text-sm mb
дый шаг – одно сравнение: чей у у корня бол
что сохраняет порядок х). Итого O(h) сравн
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {

```

«Поиск ≠ сортировка» – разбор двух граб.

```
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #333; border-radius: 10px; background-color: #1a202c; color: white; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
    Ты пытаешь
  </div>
  <p class="text-slate-300 text-sm mb-2">не быстрее  $n \cdot \log n$ .</p>
  <p class="text-slate-300 text-sm mb-2">
    /p>
    <p class="text-slate-300 text-sm">.
    сшиваются merge за  $O(h)$ .</p>
  </div>
</div>
```

оение — `: сортировка ($n \cdot \log n$ сравнений или 0(
 очку:
 ерацию,
 построение.`

```
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-
  -b border-slate-700/50">
```

Код: вставка и удаление (Скрыто)

</summary>

```
<div class="p-5 text-sm text-slate-300
  <pre class="bg-slate-950 p-4 rounde
```

```
def insert(root, key, pri):
```

```
if root is None:
```

```
return Node(key, pri)
```

```
if key <= root.key: # равные x —
```

```
root.left = insert(root.left, key, pri)
```

```
else:
```

```

<!-- 0. Определение -->
<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-500">
  <h3 class="text-xl font-bold text-blue-400 mb-4">0. Определение
  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-blue-300 italic">Введение
    <p class="text-lg sm:text-xl font-mono">
поиска <span class="text-slate-500">без</span>
: серия поворотов, которая поднимает v в ко
    <p class="text-sm text-slate-400 text-c">
т – поворот.</p>
  </div>

  <div class="grid grid-cols-1 lg:grid-cols-3">
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-xs uppercase tracking-wid">
      <p class="text-white font-bold">мож
      <p class="text-slate-400 text-xs mt">
    </div>
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-xs uppercase tracking-wid">
      <p class="text-emerald-300 font-bol">
      <p class="text-slate-400 text-xs mt">
    </div>
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-xs uppercase tracking-wid">
      <p class="text-white font-bold">0(n
      <p class="text-slate-400 text-xs mt">
    </div>
  </div>
</div>

<!-- 1. Зачем -->
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-500">
  <h3 class="text-lg font-bold text-white mb-4">1. Зачем
  <p class="text-slate-300 text-sm mb-4">В об
ючи по возрастанию – и дерево вырождается в
  <pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">вставили 10, 20, 30, 40, 50

```

`<p class="text-slate-300 text-sm">Ты ид
ода дорожка сама укорачивалась вдвое: чем ч
оплачивает все следующие. Пойдѐшь один раз
</div>`

<!-- 3. Поворот -->

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-700">
  <h3 class="text-lg font-bold text-white mb-4">Почему это важно?
  <p class="text-slate-300 text-sm mb-4">Поворот, движение, три перевешенные ссылки</span> и
  <pre class="bg-slate-950 p-4 rounded-lg overflow-hidden">
    adding-relaxed">                                40
```

40

```

      /  \
    20    C
   /  \
  A    B

```

ПОВОРОТ 20

-----&at;

<-----

ПОВОРОТ 40

$\begin{array}{cc} / & \backslash \\ A & 40 \\ & / \quad \backslash \\ & B \quad C \end{array}$

обход слева направо ДО: А 20 В 40 С

обход слева направо ПОСЛЕ: A 20 B 40 C ← тот же са

```
<div class="grid grid-cols-1 md:grid-cols-2
  <div class="bg-slate-900 rounded-lg p-4
    <p class="text-emerald-400 font-bol
    <p class="text-slate-300 text-xs">Π
```

сла поворотов — сломать его поворотами нево.

<div class="bg-slate-900 rounded-lg p-4

<p class="text-amber-400 font-bold

<p class="text-slate-300 text-xs">

одно и то же место, поэтому В просто перев

<!-- 4. Три случая -->

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
```

class="text-lq font-bold text-white mb-

0603

нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделат руками.</p>

</div>

<!-- 5. Главная ловушка -->

<div class="bg-rose-950/30 p-6 rounded-xl borde

<h3 class="text-lg font-bold text-rose-300

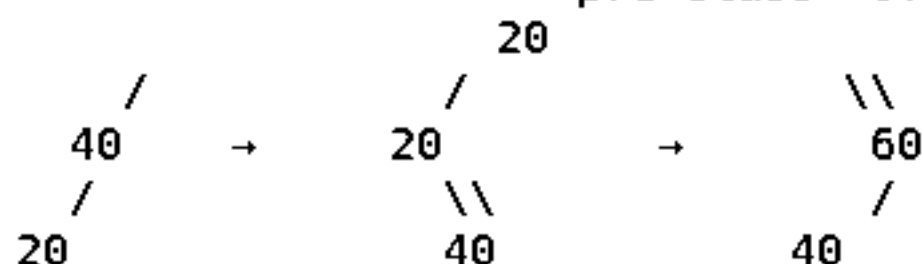
<p class="text-slate-300 text-sm mb-4">Это
ми.</p>

<div class="grid grid-cols-1 lg:grid-cols-2

<div class="bg-slate-950 rounded-lg p-4

<p class="text-rose-400 font-bold t

<pre class="overflow-x-auto text-[1



было: бамбук влево

стало: бамбук вправо

глубина никого не сократилась!</pre>

<p class="text-slate-400 text-xs mt

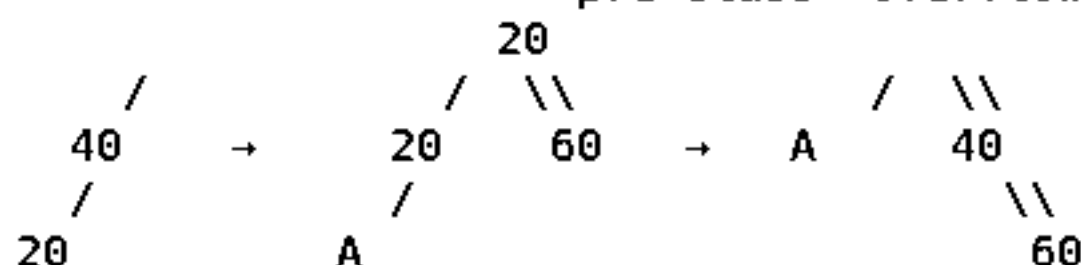
мортизации не получится.</p>

</div>

<div class="bg-slate-950 rounded-lg p-4

<p class="text-emerald-400 font-bol

<pre class="overflow-x-auto text-[1



было: линия из 3 узлов

стало: кустик высоты 2

глубина ветки упала вдвое!</pre>

```

<div class="bg-slate-900 rounded-lg p-4" style="background-color: #1e293b; border-radius: 10px; padding: 10px;">
  <p class="text-white font-bold mb-1">
  <p class="text-slate-400 text-xs">
</div>
<div class="bg-slate-900 rounded-lg p-4" style="background-color: #1e293b; border-radius: 10px; padding: 10px;">
  <p class="text-white font-bold mb-1">
  <p class="text-slate-400 text-xs">
</div>
<div class="bg-slate-900 rounded-lg p-4" style="background-color: #1e293b; border-radius: 10px; padding: 10px;">
  <p class="text-white font-bold mb-1">
  <p class="text-slate-400 text-xs">
</div>
</div>
<p class="text-slate-400 text-xs mt-4">Отсю
сти запросов splay-дерево не хуже (с точнос
дерево. Оно как бы само подстраивается под
</div>

<!-- 8. Операции -->
<div class="bg-slate-800/60 p-6 rounded-xl border" style="background-color: #2d3748; border-radius: 15px; padding: 10px; border: 1px solid #4a5568;">
  <h3 class="text-lg font-bold text-white mb-1">
  <p class="text-slate-300 text-sm mb-4">Крас
лается в корне.</p>
  <div class="space-y-2 text-sm">
    <div class="bg-slate-900 rounded-lg p-3" style="background-color: #1e293b; border-radius: 10px; padding: 5px;">
    </span> <span class="text-slate-400">— спус
маем последний узел на пути (соседний по зн
    <div class="bg-slate-900 rounded-lg p-3" style="background-color: #1e293b; border-radius: 10px; padding: 5px;">
    )</span> <span class="text-slate-400">— Spl
росто отрезаем ссылки.</span></div>
    <div class="bg-slate-900 rounded-lg p-3" style="background-color: #1e293b; border-radius: 10px; padding: 5px;">
    k)</span> <span class="text-slate-400">— sp
  </div>
    <div class="bg-slate-900 rounded-lg p-3" style="background-color: #1e293b; border-radius: 10px; padding: 5px;">
    )</span> <span class="text-slate-400">— Spl
него не будет правого сына — туда и вешаем l
  </div>
</div>
</div>

```

```

        rotate(p);                                // ZIG-ZIG: сн
        rotate(x);
    } else {
        rotate(x);                                // ZIG-ZAG: сн
        rotate(x);
    }
}
return x;                                         // теперь x –
}

```

// поиск: спустились и подняли найденное наверх

```

function find(root, key) {
    let cur = root, last = null;
    while (cur) {
        last = cur;
        if (key === cur.key) break;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return last ? splay(last) : null;           // splay делает
}

```

<p class="text-xs text-slate-400">Вся р
ate(x); против теперь x –
 т свою оценку.</p>

</div>

</details>

<!-- 10. Плюсы/минусы -->

<div class="grid grid-cols-1 md:grid-cols-2 gap-

<div class="bg-emerald-950/30 rounded-xl p-

<p class="text-emerald-300 font-bold mb

<ul class="list-disc list-inside text-s

Простой код: нет высот, цветов,

Меньше памяти, чем у AVL и красн

Сам подстраивается под «горячие

split / merge получаются почти

</div>

<div class="bg-rose-950/30 rounded-xl p-5 b

```

        p>
      </div>
    </div>
  </details>

</div>
</section>`,
  },
  {
    id: "prefix-sums-2d",
    title: "5. Двумерная матрица префиксных сумм",
    type: "html",
    description: "Сжатие суммы подматрицы за  $O(1)$ ",
    category: "Алгоритмы матрицы",
    content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
              × Аналогия 1: Вырезание прямоу
            </div>
          <p class="text-slate-300 text-sm mb">
            з него маленький целевой прямоугольник в цел
            ый верхний угол отрезается дважды! Поэтому
          </div>

```



```
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Учеба в вузе
          </div>
          <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Ограничения: Циклы
          </div>
          <p class="text-slate-300 text-sm mb-4">
```

```

        </div>
      </div>
</section>`,
    },
    {
      id: "graph-bridges",
      title: "11. Графы. Мосты",
      type: "html",
      description: "Рёбра, удаление которых расхреначивает",
      category: "Графы. Продвинутое",
      content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-rose-400">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
и  $f(u,v) > t(u,v)$ .</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Этот мост, экономикой обоих кусков придет
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
  ],

```

```

        description: "Пройти по всем ребрам ровно 1 раз.",
        category: "Графы",
        content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрывая
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Если сходится нечетное число линий, значит, из этой
            точки всегда зашел-вышел. Если сходится четное число
            линий, значит, зашел-вышел.
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
      },
      {
        id: "pathfinding-accel",
        title: "17. Графы. Ускорения поиска",
        type: "html",
        description: "",
        category: "Графы. Пути",
        content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-10">

```

```

</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
          <p class="text-xl font-mono text-white">
            (проверяем через DSU) - берем в ответ.</p>
          </div>
          <div class="grid grid-cols-1 gap-6">
            <div class="bg-slate-800 p-6 rounded-lg">
              <div class="absolute -top-3 left-4 l
                rder border-emerald-500 shadow-md">
                  Аналогия: Прокладка интернет
                </div>
                <p class="text-slate-300 text-sm mb"
                  с самых дешевых дорог в стране". Он строит
                  единую сеть.</p>
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "mst-prima",
    title: "19. Графы. Остовное дерево. Прима",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r"
    <h2 class="text-2xl sm:text-3xl font-bold text-v"
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

        <p class="text-xl font-mono text-white">
ается с соседом.</p>
</div>
<div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия: Племена
        </div>
        <p class="text-slate-300 text-sm mb
изкому племени для объединения. К вечеру пл
ства шлют гонцов к ближайшему чужому царств
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
        id: "string-kmp",
        title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
                <p class="text-xl font-mono text-white">
i], который одновременно является её суффикс
            </div>

```

```

        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
щегося с i.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Копипаста
          </div>
          <p class="text-slate-300 text-sm mb">
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной коп
          </div>
        </div>
        <details class="bg-slate-900/40 rounded-lg">
          <summary class="p-4 font-bold text-slate-300">
            -b border-slate-700/50">
              Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
              <pre class="bg-slate-950 p-4 rounded">
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
  if (i <= r) z[i] = min(r - i + 1, z[i - l]);
  while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
  if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}

```

```

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #4b4b9b; border-radius: 10px; box-shadow: 0 10px 30px #4b4b9b; position: relative; width: 100%; height: 100%;">
            <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #4b4b9b; border-radius: 10px; padding: 10px; position: relative;">
                Аналогия 2: Машина-переводчик
            </div>
            <p class="text-slate-300 text-sm mb-0">
                Это отличный переводчик на английский (Сведения о нем в разделе В, то В – это <b>NP-Полная</b> (сосредоточена на переводе))
            </p>
        </div>
    </div>
</section>`,
    },
];

```

ФАЙЛ 18/82: src/data/content_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМЕР: 1.5 КБ

```

export interface Chapter {
  id: string;
  title: string;
  type: "html";
  content: string;
}

export const chapters: Chapter[] = [
  {
    id: "euler-path-vs-cycle",
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
    type: "html",
    content: `<section id="euler-path-vs-cycle" class="mb-6">
      <div class="flex items-center mb-6">
        <span class="bg-indigo-600 text-white px-4 py-1 rounded-md">

```

талый пришёл в бар (вторая нечётная
Домой вернуться не смог, потому что
</p>
</div>

```
<div class="bg-slate-900 p-6 rounded-lg border-2 border-rose-500 shadow-md bg-slate-950">
  <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
    Цикл: Гамильтонов синдром (болезнь)
  </div>
  <p class="text-slate-300 text-sm mb-4">
    <strong>Гамильтон – это болезнь.</strong>
    ng> ровно раз и вернуться домой.
    Ты обходишь все бары (вершины) ровно
    Главное – зайти и выйти, не заходя
    Никто не знает, как решать быстро.
    <br><span class="text-xs text-rose-500">
      лноту).</span>
    </p>
  </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-2 border-slate-700">  
  <summary class="p-4 font-bold text-slate-400">  
    Душный режим: Формальное доказательство  
  </summary>  
  <div class="p-5 text-sm text-slate-300 space-y-2">  
    <p class="text-blue-400">Теорема Эйлера

Связный граф содержит эйлеров цикл ⇔  
Эйлеров путь (не цикл) ⇔ ровно две вершины с нечётной степенью



Почему Гамильтон? Потому что это NP-полная задача.



Задача коммивояжёра (TSP) — это взрыв сложности. Доказано, что любой полиномиальный алгоритм для TSP — это чудо. В общем, не парься. Просто запомни:


```



```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
    Мост: Единственная веревка
  </div>
  <p class="text-slate-300 text-sm mb-4">
    Представь, что ты альпинист. Ты спустился в пещеру.
    Из пещеры и нет других выходов — только вверх.
    Если верёвка (ребро v-u) оборвётся, ты погиб.
    Если бы из пещеры и был чёрный ход, ты бы вышел.
  </p>
</div>

<div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
  Аналогия: Кровеносный сосуд
</div>
<p class="text-slate-300 text-sm mb-4">
  Граф — это кровеносная система. Ребра — это артерии.
  (Вспомни: в графе нет циклов, как в кровеносной системе. Ребра
  терали) — это не мост, живём.
  Если же перерезать единственную артерию, ты погиб.
  Алгоритм DFS ищет такие «критические» ребра.
</p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">
    Полный код на C++ (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <p>Классическая реализация — <code class="text-slate-400">DFS</code>
    оходов.</p>
  </div>
</details>
```

```

        visited.assign(n, false);
        tin.assign(n, -1);
        low.assign(n, -1);

        for (int i = 0; i < n; ++i) {
            if (!visited[i]) dfs(i);
        }
    }</pre>
</div>
<p class="text-xs text-slate-400">Занес</p>
</div>
</details>
</div>
</section>`,
    },
    {
        id: "planarity-euler-formula",
        title: "Планарность: K5, K3,3 и формула Эйлера",
        type: "html",
        content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
        <h3 class="text-xl font-bold text-blue-400 r

        <div class="bg-slate-900 p-4 rounded-lg mb-4
            <p class="text-lg text-blue-300 italic r
            <p class="text-3xl font-mono text-white
            <div class="text-sm text-slate-400 spac
                <p><span class="text-blue-400 font-l
                <p><span class="text-emerald-400 fo
                <p><span class="text-rose-400 font-l
            </div>
        </div>
    </div>
</div>

```

```

</div>

<details class="bg-slate-900/40 rounded-lg border-1
  <summary class="p-4 font-bold text-slate-400
    order-slate-700/50">
    Доказательство непланарности K5 (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <p class="text-blue-400">Доказательство непланарности K5
    <p>
      Пусть K5 планарен.  $V = 5$ ,  $E = 10$ .
      <br> $F = E - V + 2 = 10 - 5 + 2 = 7$ 
      <br>Каждая грань ограничена минимумом 3 ребрами.
      <br>Сумма длин граней  $= 2E = 20$ .
      <br>Отсюда:  $2E \geq 3F \Rightarrow 20 \geq 21$  — противоречие.
      <br>Значит, K5 не может быть планарным.
    </p>
  </div>
</details>
</div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    content: `<section id="graph-articulation" class="mb-6">
      <div class="flex items-center mb-6">
        <span class="bg-rose-600 text-white px-4 py-1 rounded">12.
        <h2 class="text-3xl font-bold text-white">Точки сочленения
      </div>

      <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-1">
          <h3 class="text-xl font-bold text-rose-400">Определение
        </div>

        <div class="bg-slate-900 p-4 rounded-lg mb-6">
          <p class="text-lg text-rose-300 italic">Граф G называется

```

абелем (мостом). Сами свитчи — это точки со

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border-
<summary class="p-4 font-bold text-slate-400
order-slate-700/50">

Душный мод: Код и корень DFS (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-

<p class="text-blue-400 font-bold">Особ

<p>

Для стартовой вершины обхода правил
то выше неё прыгать некуда). Поэтому для не
у него больше 1 независимого ребенка

</p>

<div class="bg-slate-950 p-4 rounded-lg

<pre class="text-xs font-mono text-

```
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
```

```

-----
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as newChapters } from "../content_temp

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph
const toRemove = ["graph-planar-colors", "graph-bridges

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
  if (!toRemove.includes(c.id)) {
    // We prefer the newer versions (like tickets14_20
    dedup.set(c.id, c);
  }
});

// Add the new ones
if (newChapters) {
  newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
  const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
  return numA - numB;
});

export const chapters: Chapter[] = finalChapters;

```

```
export const GLOSSARY: GlossaryEntry[] = [
  {
    id: "bfs",
    labels: ["BFS", "обход в ширину", "поиск в ширину",
    title: "BFS – обход в ширину",
    short:
      "Волна от источника: сначала все соседи, потом со
    formal:
      "Очередь (FIFO). Кладём старт, достаём вершину –
        у рёбер.",
    complexity: "O(V + E)",
    demo: "bfs",
    simulator: "queue-bfs",
  },
  {
    id: "dfs",
    labels: ["DFS", "обход в глубину", "поиск в глубину",
    title: "DFS – обход в глубину",
    short:
      "Прёшь вперёд до упора, упёрся – откатываешься на
    formal:
      "Стек (или рекурсия). Из DFS растут: времена вход
    complexity: "O(V + E)",
    demo: "dfs",
    simulator: "stack-dfs",
  },
  {
    id: "binsect",
    labels: ["binsect", "бинсект", "бинарный поиск", "д
    title: "Бинарный поиск (и его самозванец binsect)",
    short:
      "Бинарный поиск НАХОДИТ готовый элемент в уже отс
        о не сортирует. «binsect» из конспекта – шутлив
        quicksort.",
    formal:
      "Поиск одного элемента в отсортированном массиве –
        O(n log n). Бинарный поиск сортировку не выполн
```

```

    formal:
      "find со сжатием путей + union по рангу/размеру. I
complexity: "почти  $O(1)$  – обратная функция Аккермана",
demo: "dsu",
simulator: "mst-kruskal",
},
{
  id: "trie",
  labels: ["бор", "бора", "боре", "бору", "бором", "t
  title: "Бор (trie, префиксное дерево)",
  short:
    "Дерево, где путь от корня по буквам и есть слово
  formal: "Из корня по символам; в вершине хранится ф
  complexity: "поиск слова –  $O(|\text{слова}|)$ ",
  demo: "trie",
  simulator: "aho-corasick",
},
{
  id: "bfs-on-trie",
  labels: [
    "обход дерева",
    "обход бора",
    "обход в ширину по бору",
    "какой-то обход",
    "обходом дерева",
    "конечный автомат",
    "конечного автомата",
    "автомат",
    "автомата",
  ],
  title: "Обход бора в ширину (тот самый «какой-то об
  short:
    "В Ахо–Корасике бор обходят именно BFS: суффиксн
    о есть строго по уровням.",
  formal:
    "Кладём в очередь детей корня (их ссылка = корень
    кладём с в очередь.",
  complexity: " $O(\text{суммарной длины словаря} \times \text{алфавит})$ ",

```

```

{
  id: "prefix-sum",
  labels: ["префиксные суммы", "префиксная сумма", "префикс"],
  title: "Префиксные суммы",
  short:
    "Заранее посчитанные «сколько всего набежало от начала до i»",
  formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1].",
  complexity: "препроцессинг O(n), запрос O(1)",
  demo: "prefix-sum",
  simulator: "prefix-sums-2d",
},
{
  id: "sparse-table",
  labels: [
    "разреженная таблица", "разреженная таблица", "sparse-table",
    "разреженная", "двоичные подьёмы", "двоичный подьём",
  ],
  title: "Разреженная таблица (метод двоичных подьёмов)",
  short:
    "Заранее считаем ответ для всех отрезков длины 1, 2, 4, 8, ..., m.",
  formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
  complexity: "препроцессинг O(n log n), запрос O(1)",
  demo: "sparse-table",
  simulator: "sparse-table",
},
{
  id: "segment-tree",
  labels: ["дерево отрезков", "дерево отрезков", "дерево отрезков"],
  title: "Дерево отрезков",
  short:
    "Матрёшка из отрезков: корень отвечает за весь массив, а потом делится на две части.",
  formal: "Строится за O(n), запрос и обновление — O(log n)",
  demo: "segment-tree",
  simulator: "segment-trees",
},
{

```



```

        "амортизированная", "амортизированный", "амортизир
        "амортизированное", "амортизированного", "амортизир
    ],
    title: "Амортизированная сложность",
    short:
        "Средняя цена операции «по абонементу»: одна опер
    formal: "Метод потенциалов: дорогая операция оплачи
        ассив.",
    demo: "amortized",
},
{
    id: "np",
    labels: [
        "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
        "класс P", "полином", "полиномиальное",
    ],
    title: "Класс NP и NP-полнота",
    short:
        "NP — «решение проверить легко, найти трудно» (су
        ые в NP.",
    formal: "Задача NP-полна, если она в NP и к ней полн
    demo: "np",
},
{
    id: "potential",
    labels: ["потенциал", "потенциалы", "потенциалов",
    title: "Потенциалы (перевзвешивание Джонсона)",
    short:
        "Трюк «поднять все дороги на одинаковую высоту»:
    formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  — ра
        к путей сохраняется.",
    demo: "relax",
    simulator: "johnson-algo",
},
{
    id: "scc",
    labels: ["компонента сильной связности", "компонент
    title: "Компоненты сильной связности",

```

```
{
  id: "dp",
  labels: [
    "динамическое программирование", "динамики", "дин
    "мемоизация", "мемоизацию", "подзадач", "подзадач
  ],
  title: "Динамическое программирование",
  short:
    "Решаем задачу через ответы на её меньшие копии и
  formal:
    "Нужны: оптимальная подструктура + перекрывающиеся
      ция).",
  demo: "dp",
  simulator: "dynamic-programming",
},
{
  id: "shortest-path",
  labels: ["кратчайший путь", "кратчайшие пути", "кра
  title: "Кратчайший путь",
  short:
    "Самый дешёвый маршрут из A в B. «Дешёвый» — по с
  formal:
    "Дейкстра — неотрицательные веса,  $O(E \log V)$ . Фор
      ы,  $O(V^3)$ .",
  demo: "relax",
  simulator: "dijkstra",
},
{
  id: "negative-cycle",
  labels: ["отрицательный цикл", "отрицательного цикл
  title: "Отрицательный цикл",
  short:
    "Круг, пройдя по которому вы становитесь «богаче»
  formal:
    "Форд–Беллман: если на  $V$ -й итерации ещё удаётся с
  demo: "relax",
  simulator: "bellman-ford",
},
```

```

    labels: [
        "splay", "splay-дерево", "AVL", "AVL-дерево", "вс
    ],
    title: "Splay и повороты",
    short:
        "Каждый раз, когда к узлу обратились, его «вытягивае
    formal:
        "Три случая: zig (родитель – корень), zig-zig (уз
            рацию.",
    demo: "amortized",
    simulator: "splay-tree",
},
{
    id: "adjacency",
    labels: ["матрица смежности", "список смежности", "
    title: "Способы хранить граф",
    short:
        "Матрица смежности – таблица «есть ли ребро», быс
            й, экономно для разреженных графов.",
    formal: "Матрица: проверка  $O(1)$ , память  $O(V^2)$ . Спис
},
{
    id: "dijkstra",
    labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
    title: "Алгоритм Дейкстры",
    short:
        "Жадный «навигатор»: всегда раскрываем ближайший
            льные.",
    formal:
        "Куча хранит пары (расстояние, вершина). Достали
    complexity: " $O(E \log V)$  с бинарной кучей",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "bellman-ford",
    labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман
    title: "Алгоритм Форда-Беллмана",

```

```

    simulator: "mst-prima",
  },
  {
    id: "boruvka",
    labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
    title: "Алгоритм Борувки",
    short:
      "Все компоненты одновременно шлют «гонца» к ближайшим",
    formal:
      "Фаза: для каждой компоненты ищем минимальное исходящее ребро",
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-boruvka",
  },
  {
    id: "kruskal",
    labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
    title: "Алгоритм Краскала",
    short:
      "Сортируем все рёбра по весу и берём подряд те, которые не образуют циклов",
    formal:
      "Сортировка O(E log E) + E операций union/find. Сложность O(E log E)",
    complexity: "O(E log E)",
    demo: "mst",
    simulator: "mst-kruskal",
  },
  {
    id: "components",
    labels: [
      "компонента связности", "компоненты связности", "компонент",
      "компоненте связности", "связности", "связный", "связность",
    ],
    title: "Компоненты связности",
    short:
      "Граф-архипелаг: острова, между которыми нет мостов",
    formal:
      "Считаются одним проходом: пока есть непосещённая вершина",
  }

```

```

    formal:
      "Базис — split(t, x) (разрезать по ключу) и merge
        я через них. Ожидаемая высота  $O(\log n)$ .",
    complexity: "ожидаемо  $O(\log n)$ ",
    demo: "heap",
  },
  {
    id: "bst",
    labels: ["бинарное дерево поиска", "дерево поиска",
    title: "Бинарное дерево поиска",
    short:
      "Слева всё меньше корня, справа — всё больше. Пои
    formal:
      "Без балансировки дерево может вырождаться в списо
    complexity: " $O(\text{высоты})$ : от  $O(\log n)$  до  $O(n)$ ",
    demo: "segment-tree",
  },
  {
    id: "inclusion-exclusion",
    labels: [
      "включений-исключений", "включения-исключения", "
      "вырезание прямоугольников",
    ],
    title: "Включения-исключения на префиксных суммах",
    short:
      "Чтобы получить сумму прямоугольника, берём больш
      рый отрезали дважды.",
    formal:
      " $\text{Sum}(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] -$ 
    complexity: "запрос  $O(1)$  после  $O(nm)$  предподсчёта",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
  },
  {
    id: "reduction",
    labels: ["сведение", "сведения", "сведении", "сведён
    title: "Сведение задач",
    short:

```

```

    "Правый поворот вокруг p:  $x = p.left$ ;  $p.left = x$ .  

    g и Zig-Zag.",  

    complexity: "O(1)",  

    demo: "zig",  

    simulator: "splay-rotations",  

},  

{  

    id: "zig",  

    labels: ["Zig", "Zag", "зиг"],  

    title: "Zig – одиночный поворот",  

    short:  

        "Самый простой случай: родитель x уже сидит в кор-  

        канчивается, если глубина была нечётной.",  

    formal: "Применяется ровно один раз за всю операцию",  

    demo: "zig",  

    simulator: "splay-rotations",  

},  

{  

    id: "zig-zig",  

    labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],  

    title: "Zig-Zig – x и родитель с одной стороны",  

    short:  

        "Дерево вытянулось в «бамбук»:  $g \rightarrow p \rightarrow x$  идут в о-  

        м (p, x). Именно это вдвое сокращает глубину вс-  

    formal:  

        "Если крутить наивно (два раза поднимать x), бамб-  

    demo: "zig-zig",  

    simulator: "splay-rotations",  

},  

{  

    id: "zig-zag",  

    labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],  

    title: "Zig-Zag – x и родитель с разных сторон",  

    short:  

        "Узлы идут «змейкой»: p – левый сын g, а x – прав-  

        p и g становятся двумя детьми x.",  

    formal: "Эквивалентно двойному повороту AVL-дерева",  

    demo: "zig-zag",

```

```

        </div>
      </div>
    </div>
  </section>`
},
{
  id: "dijkstra",
  title: "Билет 14. Дейкстра",
  type: "html",
  category: "Графы",
  content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Дейкстра
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">
      Дейкстра

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
        Дейкстра — алгоритм нахождения кратчайшего пути
        <p class="text-xl font-mono text-white">
          Дейкстра
      </div>

    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
          Аналогия 1: Позитивное планирование
        </div>
        <p class="text-slate-300 text-sm mb-4">
          (нельзя поехать и заработать). Он всегда выбирает
          <div id="slot-chapter-image-dijkstra">
            </div>

      <!-- Аналогия 2 -->

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500" >
  <h3 class="text-xl font-bold text-rose-400" >Аналогия 1: Параноик

  <div class="bg-slate-900 p-4 rounded-lg mb-4" >
    <p class="text-lg text-rose-300 italic" >Вот что происходит:
    <p class="text-xl font-mono text-white" >Вот что происходит:
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-2" >
    <!-- Аналогия 1 -->
    <div class="bg-slate-800 p-6 rounded-lg" >
      <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md" >
        ♂ Аналогия 1: Параноик
      </div>
      <p class="text-slate-300 text-sm mb-4" >
        еряет ВСЕ возможные дороги V-1 раз подряд.
      <div id="slot-chapter-image-bellman" >
      </div>

    <!-- Аналогия 2 -->
    <div class="bg-slate-900 p-6 rounded-lg" >
      <div class="absolute -top-3 left-4 border border-rose-500 shadow-md" >
        Отрицательный цикл
      </div>
      <p class="text-slate-300 text-sm mb-4" >
        г станет ЕЩЕ короче – значит мы попали во в
      </div>
    </div>

    <div id="slot-bellman-viz" class="mt-8"></div>
  </div>
</div>
</section>`
},
{
  id: "floyd",

```



```

        </div>
      </div>

      <div id="slot-floyd-viz" class="mt-8"></div>
    </div>
  </div>
</section>`
  },
  {
    id: "aho-corasick",
    title: "Билет 23. Алгоритм Ахо-Корасик",
    type: "html",
    category: "Строки",
    content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1 round
  <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l
    <h3 class="text-xl font-bold text-blue-400 mb-4">

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text
      <p class="text-sm text-blue-300 italic mb-2">0с
      <p class="text-lg font-mono text-white">Бор (Tr
    </div>
    <p class="text-slate-300 text-sm">Позволяет найти
      -mono text-amber-300 bg-black/30 px-1 rounded">
      у.</p>
    </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)

```

```

void buildLinks() {
    queue<int> q;
    // Корни и их дети
    for (auto const& [ch, child] : nodes[0].trie) {
        nodes[child].link = 0;
        q.push(child);
    }

    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto const& [ch, child] : nodes[v].trie) {
            // Суффиксная ссылка ребёнка – это переход к
            nodes[child].link = nodes[nodes[v].link].go

            // Если суффиксная ссылка сама является сло
            // Иначе наследуем terminal_link
            if (nodes[nodes[child].link].is_terminal) {
                nodes[child].term_link = nodes[child].l
            } else {
                nodes[child].term_link = nodes[nodes[ch
            }

            q.push(child);
        }
    }
}

</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-1

```

```

    ],
    correctIndex: 1,
    explanation: "Если нечётных вершин ровно 2 – это нечётной, заканчиваем в другой."
  },
  {
    question: "Почему гамильтонов цикл – это NP-полная задача?",
    options: [
      "Потому что Гамильтон – болезнь, и лечить её тяжело",
      "Потому что для эйлерова цикла есть простой критерий",
      "Потому что Эйлер был умнее Гамильтона"
    ],
    correctIndex: 1,
    explanation: "Для эйлерова цикла работает критерий нечётности вершин. Это NP-полная задача, которую не решают за полиномиальное время."
  },
  {
    question: "Куда ведёт эйлеров путь, если нечётных вершин больше 2?",
    options: [
      "Всё ещё можно пройти по всем рёбрам ровно раз",
      "Такого графа не существует",
      "Эйлерова пути нет – нужно удалять лишние рёбра"
    ],
    correctIndex: 2,
    explanation: "Если нечётных вершин больше 2, эйлерова пути нет. Если ровно 2 нечётные или 0."
  }
],
"bridges-code": [
  {
    question: "После DFS у нас low[u] = 4, а tin[v] = 2. Это мост?",
    options: [
      "Да, так как low[u] (4) > tin[v] (2)",
      "Нет, low[u] должно быть меньше или равно",
      "Только если граф связный и неориентированный"
    ],
    correctIndex: 0,
    explanation: "Условие моста: low[u] > tin[v]. В неориентированном графе low[u] – это минимальный индекс вершины, достижимой из u, не считая v. Если low[u] > tin[v], то v – единственный путь из u в остальную часть графа, и удаление рёбра (u, v) разобьёт граф."
  }
]

```

```
        "3 грани",
        "5 граней",
        "10 граней"
    ],
    correctIndex: 1,
    explanation: "F = E - V + 2 = 10 - 7 + 2 = 5. Не .
},
{
    question: "Почему K5 непланарен?",
    options: [
        "Потому что  $10 > 3*5 - 6 = 9$  — нарушение",
        "Потому что Эйлер был против",
        "Потому что 5 вершин — несчастливое число"
    ],
    correctIndex: 0,
    explanation: "У K5: V=5, E=10,  $3V - 6 = 9$ .  $10 > 9$ 
}
]
};
```

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 23/82: src/data/tickets/ticket1_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ

```
import { Chapter } from '../../types';
```

```
export const tickets1to5: Chapter[] = [
    {
        id: "segment-trees",
        title: "1. Дерево отрезков с операциями на отрезках",
        type: "html",
        description: "Дерево отрезков — структура данных дл
```

```

        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg p-4"
      <summary class="p-4 font-bold text-slate-300"
        -b border-slate-700/50">
        Строгое доказательство / Код (Скрыть)
      </summary>
      <div class="p-5 text-sm text-slate-300"
        <pre class="bg-slate-950 p-4 rounded"
function push(node) {
  if (lazy[node] !== 0) {
    lazy[2 * node] += lazy[node];
    tree[2 * node] += lazy[node];
    lazy[2 * node + 1] += lazy[node];
    tree[2 * node + 1] += lazy[node];
    lazy[node] = 0;
  }
}</pre>
      </div>
    </details>
  </div>
</div>
</section>
`
  },
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

```

```

        </details>
      </div>
    </div>
  </section>`
},
{
  id: "treap",
  title: "3. Декартово дерево (Treap)",
  type: "html",
  description: "Дерево поиска по ключу и Куча по приоритету",
  category: "Продвинутые структуры",
  content: `
<section id="treap" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">3
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">Декартово дерево (Treap)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Описание
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Декартово дерево — это бинарное дерево поиска, в котором каждый узел имеет два свойства: ключ и приоритет. Узлы упорядочены по ключу в левом поддереве и по приоритету в правом поддереве. Это позволяет эффективно выполнять операции поиска, вставки и удаления.
        <p class="text-xl font-mono text-white">Аналогия: Удар катаной (Split) и Слияние капель (Merge).</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Удар катаной (Split)
          </div>
          <p class="text-slate-300 text-sm mb-2">Если узел с ключом X и приоритетом Y находится в дереве, то все узлы с ключом меньше X и приоритетом меньше Y будут перемещены в левое поддерево.
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Аналогия 2: Слияние капель (Merge)
          </div>

```

```
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        (Zig, Zig-Zig, Zig-Zag), гарантируя амортизи
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
              Аналогия 1: VIP-лифт
            </div>
            <p class="text-slate-300 text-sm mb">
            вится корнем дерева (VIP-статус). Если ты ч
            ка почти не требуется времени!</p>
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 l
              border-rose-500 shadow-md">
                Аналогия 2: Балансировка чере
              </div>
              <p class="text-slate-300 text-sm mb">
              и становиться кривым как бамбук. Но как тол
            </p>
          </div>
        </div>
      </div>
      <details class="bg-slate-900/40 rounded-lg">
        <summary class="p-4 font-bold text-slate">
        -b border-slate-700/50">
          Код (Скрыто)
        </summary>
```

Хотя один поиск может стоить
емах обладают прекрасным свойством: они не
по пути. "Палочное" дерево прессуется в сб
осов.


```

        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
                Аналогия 1: Позитивное планирование
            </div>
            <p class="text-slate-300 text-sm mb-0">
                (нельзя поехать и заработать). Он всегда в пути.
            </p>
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
                Ограничения
            </div>
            <p class="text-slate-300 text-sm mb-0">
                Он не может пересматривать уже "зафиксированный" и не умеет пересматривать уже "зафиксированный"
            </p>
        </div>
        <div id="slot-dijkstra-viz" class="mt-8"></div>
    </div>
</div>
</section>`
    },
    {
        id: "bellman-ford",
        title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
        type: "html",
        category: "Графы. Пути",
        content: `
<section id="bellman-ford-content" class="mb-12 scroll-py-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы. Пути</span>
        <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">15. Графы. Поиск кратчайшего пути. Форд-Беллман</h2>
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
            <h3 class="text-xl font-bold text-rose-400">15.1. Алгоритм Форда-Беллмана

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
    <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
          <p class="text-xl font-mono text-white">
        </p>
      </div>

    <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500 shadow-md">
      <div class="absolute -top-3 left-4 bg-slate-700/50 border-emerald-500 shadow-md">
        Суть фокуса (По шагам)
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Главный герой – <b>внешний цикл</b>
        шаг <code class="bg-slate-900 text-pink-400">
        шу себе проходить через вершину k?</i>
      </p>
      <ul class="text-sm text-slate-300 space-y-2">
        <li><b>Шаг k=0:</b> Ты знаешь только
        <li><b>Шаг k=1:</b> Разрешаем пересадки
        e> через путь <code class="text-indigo-300">
        <li><b>Шаг k=2:</b> Разрешаем пересадки
        внутри этих путей уже могут быть пересадки ч
        <li>...</li>
        <li><b>Шаг k=N:</b> Ты проверил все
      </li>
    </ul>
  </div>
</div>
</section>`
},
{
  id: "johnson-algo",
  title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
  type: "html",
  category: "Графы. Пути",

```

```

    },
    {
      id: "mst-kruskal",
      title: "18. Графы. Остовное дерево. Краскал",
      type: "html",
      category: "Графы. Остовные",
      content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">18</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Остовное дерево. Краскал</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Остовное дерево</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Остовное дерево — это подграф, который является деревом и содержит все вершины графа.</p>
        <p class="text-xl font-mono text-white">Пример: Рассмотрим граф с 5 вершинами и 7 ребрами. Найдем остовное дерево.</p>
        <p>(проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Прокладка интернет-кабеля.
          </div>
          <p class="text-slate-300 text-sm mb-4">Представим, что мы хотим проложить кабель между городами (вершинами) с минимальными затратами (минимальное остовное дерево).</p>
          <p>с самых дешевых дорог в стране". Он строит единую сеть.</p>
        </div>
      </div>
    </div>
  </div>
</section>`
    },
    {
      id: "mst-prim",
      title: "19. Графы. Остовное дерево. Прима",

```

```

<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
      <p class="text-xl font-mono text-white">
        ается с соседом.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия: Племена
        </div>
        <p class="text-slate-300 text-sm mb-4">
          изкому племени для объединения. К вечеру пле-
          ства шлют гонцов к ближайшему чужому царству.
        </div>
      </div>
    </div>
  </div>
</div>
</section>`
  }
};

```

ФАЙЛ 25/82: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМЕР: 1.2 КБ

```
import { Chapter } from '../types';
```

```
export const tickets21to24: Chapter[] = [
```



```
<!-- Аналогия 1 -->
<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    Аналогия 1: Линейка-шаблон
  </div>
  <p class="text-slate-300 text-sm mb-4">
    Тут мы берём всю строку и «прикидываю
    я начну читать строку отсюда, насколько длин
    Это самый быстрый способ найти индекс
    >, считаешь Z-функцию, и там, где <code>Z[i]
  </p>
</div>

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
    Аналогия 2: LLM Самокопираст
  </div>
  <p class="text-slate-300 text-sm mb-4">
    В LLM Z-функция может применяться
    [i]</code> (где <i>i</i> указывает назад на
    сама себя.
  </p>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
    -b border-slate-700/50">
    Пример вычисления (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <p>Тот же пример: <b>abcabacd</b></p>
    <ul class="list-disc pl-5 space-y-2">
      <li><code>Z[0]</code> – не считаем
      <li><code>Z[1]</code> (<b>bcabacd</b>)
```

```
</div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border-l
```

```
    <h3 class="text-xl font-bold text-blue-400 mb-4">П
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-6 text
```

```
      <p class="text-sm text-blue-300 italic mb-2">Ос
```

```
      <p class="text-lg font-mono text-white">Бор (Tr
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm">Позволяет найти
```

```
      го один проход.</p>
```

```
  </div>
```

```
<!-- Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
```

```
  <div class="bg-slate-800 p-6 rounded-lg border bo
```

```
    <div class="absolute -top-3 left-4 bg-slate-700
```

```
      emerald-500 shadow-md">
```

```
      Аналогия 1: Паутина (Бор)
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm mb-4">Представ
```

```
      ет — мы падаем по <b>суффиксной ссылке</b> ("
```

```
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2 l
```

```
  <div class="absolute -top-3 left-4 bg-rose-900
```

```
    -500 shadow-md">
```

```
    Аналогия 2: Матёшка (Терминальные)
```

```
  </div>
```

```
  <p class="text-slate-300 text-sm mb-4">Представ
```

```
    нашли и "he", потому что оно спрятано внутри
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border b
```

```
  <summary class="p-4 font-bold text-slate-400 ou
```

```
    r-slate-700/50">
```

```

<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-purple-300 italic">
    <p class="text-xl font-mono text-white">
      Сведение (Reduction) A->B: Если я умею реша
    </div>
  <div class="grid grid-cols-1 xl:grid-cols-2">
    <!-- Аналогия 1 -->
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
        Аналогия 1: Судoku (P vs NP)
      </div>
      <p class="text-slate-300 text-sm mb-4">
        пример сортировка чисел). Класс <b>NP</b> –
        енную сетку (ответ), он БЫСТРО (за класс P)
      </div>
    <!-- Аналогия 2 -->
    <div class="bg-slate-900 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
        Аналогия 2: Машина-переводчик
      </div>
      <p class="text-slate-300 text-sm mb-4">
        ь отличный переводчик на английский (Сведенн
        аче B, то B – это <b>NP-Полная</b> (сосредо
      </div>
    </div>
  </div>
</div>
</section>`
  }
];

```



```

        </div>
        <p class="text-slate-300 text-sm mb-4">
            т. Оптимально для почти всех задач.</p>
        </div>
    </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl border border-indigo-500 shadow-md">
    <h3 class="text-xl font-bold text-indigo-400">Аналогия DFS</h3>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия DFS -->
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-indigo-500 shadow-md">
                ♂ DFS (Поиск в глубину) = Жадный алгоритм
            </div>
            <p class="text-slate-300 text-sm mb-4">
                <b>Что делает:</b> Жадный алгоритм находит первую доступную
                вершину (например, минимальную по номеру) и пробует другой путь.
            </p>
            <ul class="text-xs text-indigo-300">
                <li><b>Где используется:</b></li>
                <li>Топологическая сортировка (и DFS)</li>
                <li>Поиск мостов и точек сочленения</li>
                <li>Сильно связанные компоненты Куты</li>
                <li>Поиск циклов и просто проверка</li>
            </ul>
        </div>

        <!-- Аналогия BFS -->
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
                ♂ BFS (Поиск в ширину) = Волна
            </div>

```



```

        карту – остались ли серые (неисследованные)
        ь через океан – столько и компонент.</p>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">Топологическая сортировка</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Топологическая сортировка – это расписание, которое определяет порядок выполнения задач, таких как задачи на графах, которые имеют зависимости между собой. Например, если у вас есть задача, которая зависит от двух других задач, то вы должны сначала выполнить эти две задачи, прежде чем сможете выполнить задачу, которая зависит от них.</p>
        <p class="text-xl font-mono text-white">Аналогия 1: Учеба в вузе</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Учеба в вузе
          </div>
          <p class="text-slate-300 text-sm mb-4">Топологическая сортировка – это расписание, которое</p>
        </div>
      </div>
    </div>
  </div>
`
  }
]

```

```

} </pre>
        </div>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
        category: "Графы. Продвинутые",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
            <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
        </div>

        <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-emerald-300 italic">Макро-вершина — это компонент сильной связности (SCC) ориентированного графа, в котором все вершины взаимно достижимы. Другими словами, для любых двух вершин u и v в SCC существует путь из u в v и путь из v в u.
        </div>

        <p class="text-slate-300 text-sm mb-4">
            <strong>«Это просто цикл?» — Нет!</strong>
        </p>
        <ul class="list-disc list-inside text-sm text-slate-300">
            <li>Если у тебя есть цикл A → B → C → A, то A, B и C принадлежат к одной и той же макро-вершине.
        </li>
        </ul>
    </div>
    </div>
    </section>`
    }
]
    </pre>

```

```

<p class="text-emerald-400 font-bol
<ol class="list-decimal list-inside
  <li>Запускаем DFS на исходном г
li>
  <li>Разворачиваем этот список (
  <li>Переворачиваем все ребра в
  <li>Идем по <code>order</code>:
!</li>
  </ol>
</div>
</details>
</div>
</div>
</section>`,
},
{
  id: "graph-bridges",
  title: "11. Графы. Мосты",
  type: "html",
  description: "Рёбра, удаление которых расхреначивае
  category: "Графы. Продвинутое",
  content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-rose-400 m
      <div class="bg-slate-900 p-4 rounded-lg mb-1
        <p class="text-lg text-rose-300 italic m
        <p class="text-xl font-mono text-white"
и fup[v] > tin[u].</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg
          <div class="absolute -top-3 left-4

```

```
    </div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
    <strong>Важно:</strong> Это работает то
    епятся к точкам сочленения. То есть, <strong>
    ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
    <div class="bg-slate-800 p-6 rounded-lg border-
```

```
        <div class="absolute -top-3 left-4 bg-slate-800
        rder-rose-500 shadow-md">
```

```
            Аналогия: Главный перекресток
```

```
        </div>
```

```
        <p class="text-slate-300 text-sm mb-4">
```

```
            Представь город, где два района сое
            её перекроют (удалят вершину) – районы буд
            g>хрупкость</strong> системы.
```

```
        </p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
    <div class="absolute -top-3 left-4 bg-emerald-500
    er border-emerald-500 shadow-md">
```

```
        Телеком: Центральный свитч
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm mb-4">
```

```
        У тебя несколько компьютеров в одной
        абелем (мостом). Сами свитчи – это точки со
```

```
    </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
    <summary class="p-4 font-bold text-slate-400">
    order-slate-700/50">
```

```

        Это глубокий дуальный мост между ориентир
    </p>
    <ul class="list-disc list-inside text-sm text-rose-400">
        <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
        <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
        <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
    </ul>
    </div>
</div>
</section>`,
    },
    {
        id: "graph-euler",
        title: "13. Графы. Эйлеров цикл",
        type: "html",
        description: "Пройти по всем ребрам ровно 1 раз.",
        category: "Графы",
        content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-indigo-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-indigo-300 italic">
                <p class="text-xl font-mono text-white">
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg

```

```

}

const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
  { id: 1, x: 100, y: 150, label: "1" },
  { id: 2, x: 250, y: 250, label: "2" },
  { id: 3, x: 400, y: 150, label: "3" },
  { id: 4, x: 550, y: 150, label: "4" },
  { id: 5, x: 700, y: 50, label: "5" },
  { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 },
  { u: 3, v: 4 }, // 3 and 4 are articulation points (3
  { u: 4, v: 5 },
  { u: 5, v: 6 },
  { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
  { length: Object.keys(nodes).length },
  () => [],
);

edges.forEach((e) => {
  adj[e.u].push(e.v);
  adj[e.v].push(e.u);
});

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
}

```



```

        if (to === p) continue;

        if (visited.has(to)) {
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);
            recordStep(
                `Обратное ребро ${v}-${to}. Обновляем low[${v}],
                v,
            );
        } else {
            children++;
            dfs(to, v);
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);

            recordStep(
                `Возврат в ${v} из ${to}. Обновляем low[${v}],
                v,
            );

            if (currentLow[to] >= currentTin[v] && p !== ap.add(v);
                recordStep(
                    `Найдена точка сочленения: вершина ${v},
                    v,
                );
        }
    }
}

if (p === -1 && children > 1) {
    ap.add(v);
    recordStep(
        `Корень дерева DFS ${v} имеет >1 детей. Это
        v,
    );
} else if (p === -1) {
    recordStep(
        `Корень дерева DFS ${v} имеет <=1 ребенка, он
        v,
    );
}

```

```

        setCurrentStepIndex(0);
    };

    return (
      <div className="bg-slate-900 rounded-xl border border-
        <div className="bg-slate-800 p-4 border-b border-
          <h3 className="font-bold text-white flex items-
            <Info className="w-5 h-5 text-rose-400" />
              Моделирование поиска точек сочленения
            </h3>

          <div className="flex items-center gap-2 bg-slate-
            <button
              onClick={togglePlay}
              className="flex items-center gap-2 px-3 py-
              text-white"
            >
              {isPlaying ? (
                <Pause className="w-4 h-4 ml-1" />
              ) : (
                <Play className="w-4 h-4 ml-1" />
              )}
              {isPlaying ? "Пауза " : "Старт "}
            </button>

            <button
              onClick={reset}
              className="flex items-center justify-center
              ors"
            >
              <RotateCcw className="w-4 h-4" />
            </button>
          </div>
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-3 g
          <div className="lg:col-span-2 relative h-96 lg:l
            <svg className="w-full h-full" viewBox="0 0 8

```

```

        : isAp
        ? "#f43f5e"
        : isVisited
        ? "#34d399"
        : "#334155";
const r = isAp ? 24 : 20;

return (
  <g key={node.id}>
    <circle
      cx={node.x}
      cy={node.y}
      r={r}
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-all duration-1
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="14px"
      fontWeight="bold"
      className="select-none pointer-events
    >
      {node.label}
    </text>

    {isVisited && (
      <>
        <text
          x={node.x}
          y={node.y - r - 15}
          textAnchor="middle"
          fill="#94a3b8"

```

```

<div className="space-y-4 flex-1 overflow-y-auto">
  <div className="mb-4">
    <h5 className="text-xs text-slate-500 font-medium">Точка сочленения</h5>
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-slate-500"></div>
    </div>
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-rose-500"></div>
    </div>
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-teal-500"></div>
    </div>
    <div className="flex items-center gap-2">
      <div className="w-8 h-1 bg-rose-500 border-1px solid black"></div>
    </div>
  </div>

  <div>
    <h5 className="text-xs text-slate-500 font-medium">ЛОГ ДЕЙСТВИЙ:</h5>
    <div className="space-y-2">
      {steps
        .slice(0, currentStepIndex + 1)
        .reverse()
        .map((step, idx) => (
          <div
            key={idx}
            className={`text-xs p-2 rounded $
xt-slate-500`} `>
            >
              {step.desc}
            </div>
          </div>
        ))}
    </div>
  </div>
</div>

```

```

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
      { u: 'B', v: 'C', w: -2 },
      { u: 'A', v: 'C', w: 1 },
      { u: 'C', v: 'D', w: 3 },
      { u: 'C', v: 'E', w: 4 },
      { u: 'D', v: 'F', w: 2 },
      { u: 'E', v: 'F', w: 1 },
    ];
    if (cycle) {
      return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
    } else {
      return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
    }
  };

  const edges = getEdges(hasCycle);

  const getAdjList = (ed: Edge[]) => {

```

```

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
      dist = { ...dist, [edge.v]: uDist + edge.w };
      prev = { ...prev, [edge.v]: edge.u };
      updated = true;
      anyUpdate = true;
    }

    simulationSteps.push({
      iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
      distances: { ...dist }, previous: { ...prev },
      log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
        ${updated ? 'Релаксация успешна! dist[${edge.v}] = ' + dist[edge.v] : ''}
      `,
      activeCodeLine: updated ? 6 : 5,
    });
  }

  if (!anyUpdate && iter < vCount - 1) {
    simulationSteps.push({
      iteration: iter, checkingEdge: null, distances: { ...dist },
      log: `Ранний выход после итерации ${iter}.`,
      activeCodeLine: 10,
    });
    break;
  }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
  let cycleFound = false;
  for (const edge of edges) {
    if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
      cycleFound = true;
      simulationSteps.push({
        iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
        distances: { ...dist }, previous: { ...prev },
        log: `ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} → ${edge.v}
          образует цикл, так как dist[${edge.u}] + w < dist[${edge.v}].`
      });
    }
  }
  if (cycleFound) {
    console.log('Граф содержит цикл!');
  } else {
    console.log('Граф не содержит циклов!');
  }
}

```

```

    <div className="p-2.5 bg-blue-600 text-white"
      <span className="text-2xl"> </span>
    </div>
    <div>
      <h3 className="text-2xl font-bold text-white">
        <p className="text-sm text-blue-300">Полный
      </p>
    </div>
  </div>

  <div className="flex items-center gap-3">
    <div className="bg-slate-900 p-1 rounded-lg b
      <button onClick={() => setHasCycle(false)}
        bg-blue-600 text-white' : 'text-slate-400'
      <button onClick={() => setHasCycle(true)} c
        r gap-1 ${hasCycle ? 'bg-rose-600 text-white
    </div>
    <button onClick={() => { setIsPlaying(false);
      g-slate-600 text-white px-3 py-2 rounded-lg
    <button onClick={() => setIsPlaying(!isPlaying
      sition-colors text-white ${isPlaying ? 'bg-
    <button onClick={() => { setIsPlaying(false);
      ; }} className="flex items-center gap-1 bg-l
      s">⏮ar    </button>
    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6 m
    {/* Граф */}
    <div className="w-full lg:w-2/3 bg-blue-950/20
      y-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-blue
        ld shadow-sm">
        Итерация {currentStep.iteration} (⏮ar {curr
      </div>

      {currentStep.hasNegativeCycle && (
        <div className="absolute top-3 right-3 bg-r
          animate-pulse shadow-lg shadow-rose-600/50

```

```

    });
  }}}

  {nodes.map((node) => {
    const dist = currentStep.distances[node.id];
    const isCheckingNode = currentStep.checkingNode ===
      node.id);

    return (
      <g key={node.id} className="transition-...
        <circle cx={node.x} cy={node.y} r={isC
        ckingNode ? '#7dd3fc' : '#64748b'} strokeWid
        <text x={node.x} y={node.y + 5} fill=
        <text x={node.x} y={node.y - 28} fill=
        ight="bold" textAnchor="middle" className="
        <text x={node.x} y={node.y + 35} fill=
        split(' ')[0]}</text>
      </g>
    );
  }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
  <p className="font-semibold text-amber-400 m
  <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
  <h4 className="text-lg font-bold text-emerald
  <div className="space-y-2 text-sm font-mono t
    {Object.keys(adjList).map(u => {
      <div key={u} className="flex flex-wrap g
      -colors">
        <span className="font-bold text-white
        {adjList[u].map((edge, idx) => {
          const isChk = currentStep.checkingE

```



```
      <h4 className="text-lg font-bold text-slate-600">
      <div className="bg-slate-950/80 rounded-lg p-4">
        {codeLines.map(item => {
          <div key={item.line} className={`py-1.5 px-4 ${
            item.line === 1 ? 'bg-blue-900/80 text-amber-400' : 'text-slate-600'
          }`} >
            <span className="text-slate-600 w-5 flex-shrink-0">{item.line}</span>
            <span className="whitespace-pre flex-grow-1">{item.code}</span>
          </div>
        })}
      </div>
    </div>
  </div>
</div>
);
}
```

ФАЙЛ 29/82: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect, useCallback } from 'react';
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';
```

```
const NODES = [
  { id: 0, label: '0', x: 300, y: 50 },
  { id: 1, label: '1', x: 150, y: 150 },
  { id: 2, label: '2', x: 450, y: 150 },
  { id: 3, label: '3', x: 75, y: 250 },
  { id: 4, label: '4', x: 225, y: 250 },
  { id: 5, label: '5', x: 375, y: 250 },
  { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
  { from: 0, to: 1 }, { from: 0, to: 2 },
  { from: 1, to: 3 }, { from: 1, to: 4 },
  { from: 2, to: 5 }, { from: 2, to: 6 },
  { from: 3, to: 4 }, { from: 4, to: 5 },
  { from: 5, to: 6 },
];
```

```
});
```

```
while (queue.length > 0) {
  steps.push({
    activeLine: 4,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: `Очередь не пуста, элементов: ${queue.length}`
  });
```

```
  const curr = queue.shift()!;
  steps.push({
    activeLine: 5,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: curr,
    logs: `Достаем из очереди (${curr}).`
  });
```

```
  const neighbors = ADJ[curr];
  if (neighbors.length > 0) {
    for (const n of neighbors) {
      if (!visited.has(n)) {
        steps.push({
          activeLine: 7,
          queue: [...queue],
          visited: Array.from(visited),
          currentNode: curr,
          logs: `Рассматриваем соседа: ${n}. Он еще не`
        });
```

```
        visited.add(n);
        queue.push(n);
```

```
        steps.push({
          activeLine: 9,
          queue: [...queue],
```

```

    }, []));

    // @ts-expect-error зарезервировано под кнопку «назад»
    const handlePrev = useCallback(() => {
      setStepIdx(prev => Math.max(prev - 1, 0));
    }, []);

    useEffect(() => {
      if (!isPlaying) return;
      if (stepIdx >= STEPS.length - 1) {
        setIsPlaying(false);
        return;
      }
      const timer = setTimeout(handleNext, 1200);
      return () => clearTimeout(timer);
    }, [isPlaying, stepIdx, handleNext]);

    return (
      <div className="bg-slate-900 border-2 border-slate-700" >
        <div className="flex flex-col md:flex-row items-center" >
          <div>
            <h3 className="text-2xl font-bold text-white" >
              <span className="text-blue-400" > </span> Ал
            </h3>
            <p className="text-slate-400 text-sm mt-1">Ви
          </div>
          <div className="flex bg-slate-800 p-1.5 rounded" >
            <button
              onClick={() => { setIsPlaying(false); setSt
              className="p-2 text-slate-400 hover:text-wh
              title="C6poc"
            >
              <RotateCcw className="w-5 h-5" />
            </button>
            <button
              onClick={() => setIsPlaying(p => !p)}
              className="p-2 text-blue-400 hover:text-bl
              title={isPlaying ? 'Пауза' : 'Воспроизведен

```

```

const isInQueue = step.queue.includes(n.id)
const isVisited = step.visited.includes(n.id)

let fill = '#1e293b'; // slate-800
let stroke = '#475569'; // slate-600
let scale = 1;
void scale;

if (isCurrent) {
  fill = '#3b82f6'; // blue-500
  stroke = '#60a5fa'; // blue-400
  scale = 1.3;
} else if (isInQueue) {
  fill = '#059669'; // emerald-600
  stroke = '#34d399'; // emerald-400
  scale = 1.1;
} else if (isVisited) {
  fill = '#0f172a'; // slate-900
  stroke = '#3b82f6'; // blue-500
}

return (
  <g key={n.id} className="transition-tran
    {isCurrent && {
      <circle cx={n.x} cy={n.y} r="30" fi
    }}
    <circle
      cx={n.x} cy={n.y}
      r="20"
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-colors duration
  />
  <text
    x={n.x} y={n.y}
    textAnchor="middle"
    dy=".3em"

```

```

        });
      }}}
    </div>
  </div>

  <div className="bg-slate-800/80 border border-
    <div className="text-sm font-bold text-eme
      <span>Состояние Очереди (Queue):</span>
      <span className="text-xs text-slate-500
    </div>
    <div className="flex items-center gap-2 mi
      {step.queue.length === 0 ? {
        <span className="text-slate-500 text-x
      } : {
        step.queue.map(({qId, i}) => {
          <span key={`${qId}-${i}`} className=
        enter justify-center rounded-md shadow-lg s
          {qId}
        </span>
      })
    }}
  </div>
  <div className="mt-2 text-sm text-amber-300
    ms-center justify-center">
    "{step.logs}"
  </div>
</div>
</div>
</div>
</div>
);
}

```

```
      <p className={`text-center text-xs mt-2 font-bold`} >
    </div>
  );
}
```

ФАЙЛ 31/82: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  compact?: boolean;
}
```

```
/**
```

```
 * Переходы «предыдущая / следующая тема» — как на Codecademy
 * Компактный вариант живёт в шапке главы, крупный — по центру
 */
```

```
export const ChapterNav: React.FC<Props> = ({ prev, next, index, total, onGo, compact }) => {
  if (compact) {
    return (
      <div className="flex items-center gap-2 text-xs">
        <button
          disabled={!prev}
          onClick={() => prev && onGo(prev.id)}
          title={prev ? prev.title : "Это первая тема"}
          className="flex items-center gap-1 px-2.5 py-1.5 rounded-md
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
          <ChevronLeft />
        </button>
        <span>{index} / {total}</span>
        <button
          disabled={!next}
          onClick={() => next && onGo(next.id)}
          title={next ? next.title : "Это последняя тема"}
          className="flex items-center gap-1 px-2.5 py-1.5 rounded-md
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
          <ChevronRight />
        </button>
      </div>
    );
  } else {
    return (
      <div className="flex items-center gap-2 text-sm">
        <button
          disabled={!prev}
          onClick={() => prev && onGo(prev.id)}
          title={prev ? prev.title : "Это первая тема"}
          className="flex items-center gap-1 px-3 py-1.5 rounded-md
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
          <ChevronLeft />
        </button>
        <span>{index} / {total}</span>
        <button
          disabled={!next}
          onClick={() => next && onGo(next.id)}
          title={next ? next.title : "Это последняя тема"}
          className="flex items-center gap-1 px-3 py-1.5 rounded-md
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
          <ChevronRight />
        </button>
      </div>
    );
  }
};
```

```

        {next && (
          <button
            onClick={() => onGo(next.id)}
            className="group text-right p-4 rounded-xl bg-
              transition-all sm:col-start-2"
          >
            <div className="flex items-center justify-end
              text-indigo-400 mb-1">
              Следующая тема <ChevronRight className="w-3
            </div>
            <div className="text-sm font-bold text-slate-
          </button>
        )}
      </nav>
    );
  };
};

```

ФАЙЛ 32/82: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useCallback, useEffect, useRef, useState } from 'react';
import { Check, Download, Loader2, Maximize2, MousePointer } from 'lucide-react';
import type { Chapter } from '../types';
import { useTermHints } from '../hooks/useTermHints';
import { TermPopover, type HintAnchor } from '../hints/TermHint';
import { getViz } from '../vizRegistry';
import { ChapterNav } from '../ChapterNav';
import { downloadChapterHtml } from '../utils/exportHtml';

```

```

interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
}

```

```

    } catch {
      setSaving("idle");
    }
  }, [chapter]));

const open = useCallback(
  (el: HTMLElement) => {
    const termId = el.dataset.termId;
    if (!termId) return;
    cancelHide();
    setAnchor({ termId, rect: el.getBoundingClientRect() });
  },
  [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
  if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
  if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
  if (!el) return;
  e.preventDefault();
  if (anchor && anchor.termId === el.dataset.termId)
    else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
  if (el) open(el);
};

```



```

        className="prose prose-invert max-w-none"
        onMouseOver={handlePointerOver}
        onMouseOut={handlePointerOut}
        onClick={handleClick}
        onFocus={handleFocus}
        dangerouslySetInnerHTML={{ __html: chapter.content }}
      />

    <p className="mt-6 flex items-start gap-2 text-
      ed-lg px-3 py-2">
      <MousePointerClick className="w-4 h-4 shrink-0" />
      <span>
        Подчёркнутые термины – интерактивные: наведи
        объяснение на пальцах, мини-анимацию и кноп
      </span>
    </p>

    {viz && (
      <section id="chapter-viz" className="mt-8 scr
        <div className="flex items-center justify-b
          <h3 className="flex items-center gap-2 te
            <Sparkles className="w-4 h-4 text-indigo
              Демонстрация: {viz.title}
            </h3>
            <button
              onClick={() => onOpenSimulator(chapter.
              className="flex items-center gap-1.5 te
            ate-300 hover:text-white hover:border-indigo
              >
                <Maximize2 className="w-3.5 h-3.5" /> P
              </button>
            </div>
            {viz.hint && <p className="text-xs text-sla
              <viz.Component />
            </section>
          )}

    <ChapterNav prev={prev} next={next} index={inde

```

```

{ line: "          if (visited[to]) {", highlight: "normal" },
{ line: "              low[v] = min(low[v], tin[to]);", highlight: "normal" },
{ line: "          } else {", highlight: "normal" },
{ line: "              dfs(to, v);", highlight: "normal" },
{ line: "              low[v] = min(low[v], low[to]);", highlight: "normal" },
{ line: "          }, highlight: "normal" },
{ line: "          if (low[to] > tin[v]) {", highlight: "normal" },
{ line: "              // ЭТО МОСТ!", highlight: "normal" },
{ line: "          }, highlight: "normal" },
{ line: "      }", highlight: "normal" },
{ line: "  }", highlight: "normal" },
{ line: "}", highlight: "normal" },
];

```

```

interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
  tin: Record<number, number>;
  low: Record<number, number>;
  stack: number[];
  bridges: string[];
  log: string;
  activeEdge: [number, number] | null;
  backEdge: [number, number] | null;
  activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
  { id: 0, label: "A", x: 100, y: 120 },
  { id: 1, label: "B", x: 260, y: 120 },
  { id: 2, label: "C", x: 180, y: 200 },
  { id: 3, label: "D", x: 180, y: 300 },
  { id: 4, label: "E", x: 100, y: 370 },
  { id: 5, label: "F", x: 260, y: 370 },
  { id: 6, label: "G", x: 340, y: 200 }
];

```

```

const GRAPH_EDGES = [

```

```

    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, sta
        activeEdge: [1,2], backEdge: null,
        log: "Из В идём в С. tin[C]=4, low[C]=4.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
        activeEdge: null, backEdge: [2,0],
        log: "Видим соседа А (уже посещён)! Обратное ребро
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3],
        tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
        activeEdge: [2,3], backEdge: null,
        log: "Идём С→D. tin[D]=5, low[D]=5.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
        activeEdge: [3,4], backEdge: null,
        log: "D→E. tin[E]=6, low[E]=6.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
        activeEdge: [4,5], backEdge: null,
        log: "E→F. tin[F]=7, low[F]=7.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],

```

```

    activeCodeLine: [11,13,16,17,18]
  },
  {
    currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1},
    activeEdge: null, backEdge: null,
    log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм закончен.",
    activeCodeLine: []
  },
];

```

```

export function DfsBridgesSimulator() {
  const [dfsIdx, setDfsIdx] = useState(0);
  const [dfsAuto, setDfsAuto] = useState(false);
  const dfsTimer = useRef<NodeJS.Timeout | null>(null);

  useEffect(() => {
    if (dfsAuto) {
      dfsTimer.current = setInterval(() => {
        setDfsIdx(prev => {
          if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
          return prev + 1;
        });
      }, 2500);
    }
    return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
  }, [dfsAuto]);

```

```

const step = DFS_STEPS[dfsIdx];

```

```

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">Решение задачи
      <div className="flex items-center gap-1 bg-slate-800 p-1">
        <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
          disabled={dfsIdx === 0}
          className="p-1.5 hover:bg-slate-700 rounded"

```

```

    let sc = "#475569", sw = 2, dash = "none"
    if (bridge) { sc = "#eab308"; sw = 4; }
    else if (activeTree) { sc = "#10b981"; sw
    else if (activeBack) { sc = "#f43f5e"; sw
    else if (step.visitedIndices.includes(e.u
    return <line key={e.key} x1={u.x} y1={u.y
        stroke={sc} strokeWidth={sw} strokeDash
        className="transition-all duration-500"
    }}}
    {step.currentNode !== null && step.currentN
        const n = GRAPH_NODES.find(x => x.id ===
        if (!n) return null;
        return <circle cx={n.x} cy={n.y} r={18} f
    >;
    }}())}
    {GRAPH_NODES.map(n => {
        const cur = step.currentNode === n.id;
        const vis = step.visitedIndices.includes(n
        const st = step.stack.includes(n.id);
        return <g key={n.id}>
            <circle cx={n.x} cy={n.y} r={12}
                fill={cur ? "#10b981" : st ? "#064e3b
                stroke={cur ? "#fff" : st ? "#34d399"
                strokeWidth={2.5}
                className="transition-all duration-300
            <text x={n.x} y={n.y+3} textAnchor="mid
label}</text>
            {vis && (
                <
                    <rect x={n.x-18} y={n.y-28} width={
"transition-all duration-300" />
                    <text x={n.x} y={n.y-20} textAnchor:
                } l:{step.low[n.id]||"?"}</text>
            </>
            )}
        </g>;
    }}}
</svg>

```

```

        {step.stack.length === 0
        ? <div className="text-[10px] text-slate-400"
        : step.stack.map((id, i) => {
            <div key={id} className={`px-2 py-0
            i === step.stack.length-1
            ? "bg-emerald-600/20 text-emerald-500"
            : "bg-slate-900 text-slate-400"
            }`} >Вершина {GRAPH_NODES.find(n=>n.id === id).name}
            <span className="text-slate-500 ml-2" >{step.stack[i].name}
            </div>
        ))
    }
  </div>
</div>

{/* Log */}
<div className="bg-indigo-950/10 border border-indigo-500 p-4" >
  <p className="text-[11px] text-slate-300 leading-tight" >Шаг {step.id}
  <div className="mt-2 pt-2 border-t border-slate-700" >
    <span className="text-slate-400" >Мосты:
    <span className="font-mono font-bold text-slate-300" >{
      {step.bridges.length > 0 ? step.bridges.map(b => b.name).join(' ') : ''}
    }
    </span>
  </div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 34/82: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect } from 'react';
```

```
const adjList: { [key: string]: { v: string; w: number } } =
  nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
  adjList[e.u].push({ v: e.v, w: e.w });
});
```

```
const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push([0, start])' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbors(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push([dist[v], v])' },
  { line: 11, text: '    return dist // Все кратчайшие расстояния найдены' },
];
```

```
const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);
```

```
useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };
  const visited: Record<string, boolean> = { A: false, B: false };
  const prev: Record<string, string | null> = { A: null, B: null };

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ' Дейкстра готов к доставке. Начинаем из Пикет-Парка',
    activeCodeLine: 2,
  });
```

```
for (let iter = 0; iter < nodes.length; iter++) {
```

```

        log: `    Улучшение (Релаксация)! Новое кратчайшее
        activeCodeLine: 9,
    });
  }
}
}

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: `    Дейкстра всё доставил! Все возможные кратчайшие
  activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border
    {/* Шапка симулятора */}
    <div className="flex flex-wrap items-center justify
      <div className="flex items-center gap-3">

```



```

<div className="w-full lg:w-2/3 bg-indigo-950/20
  justify-center min-h-[400px]">
  <div className="absolute top-3 left-3 bg-indigo-950/20
    font-bold shadow-sm">
    Шаг {currentStepIndex + 1} из {steps.length}
  </div>

  <svg className="w-full h-auto max-h-[500px]" >
    <defs>
      <marker id="arrow-dh-normal" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
      </marker>
      <marker id="arrow-dh-active" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
      </marker>
      <marker id="arrow-dh-opt" markerWidth="6"
        <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
      </marker>
    </defs>
    {/* Рисуем */}
    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id === edge.u)
      const vNode = nodes.find((n) => n.id === edge.v)
      const isChecking =
        currentStep.checkingEdge === edge.id
      const isOptimal = currentStep.previousEdge === edge.id

      let marker = 'url(#arrow-dh-normal)';
      if (isChecking) marker = 'url(#arrow-dh-active)';
      else if (isOptimal) marker = 'url(#arrow-dh-opt)';

      return (
        <g key={idx}>
          <line
            x1={uNode.x} y1={uNode.y}
            x2={vNode.x} y2={vNode.y}
            stroke={marker}
          />
        </g>
      )
    })}
  </svg>

```

```

        <text x={node.x} y={node.y + 5} fill=
            {node.id}
        </text>
        <text x={node.x} y={node.y - 28} fill=
"middle" className="bg-indigo-950 px-1 py-0
            {dist === 999 ? '∞' : `d=${dist}`}
        </text>
        <text x={node.x} y={node.y + 35} fill=
            {node.name.split(' ')[0]}
        </text>
    </g>
  );
  }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
  <p className="font-semibold text-amber-400">
  <p className="min-h-[40px] flex items-center">
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50">
  <h4 className="text-lg font-bold text-emerald-400">
  <div className="space-y-2.5 flex-1 overflow-y-hidden">
    {Object.keys(adjList).map((u) => {
      const isCurrentNode = currentStep.currentNode === u;
      return (
        <div key={u} className={`p-3 rounded-lg ${
          isCurrentNode ? 'bg-emerald-900/60 text-white' : 'text-slate-300'
        }`} >
          <div className="flex items-center gap-2">
            <span className={`px-2 py-0.5 rounded ${
              isCurrentNode ? 'bg-emerald-900/60 text-white' : 'text-slate-300'
            }`} >
              {u}
            </span>
            <span className="text-slate-400">→</span>
          </div>
        </div>
      );
    })}
  </div>

```

```

        const dist = currentStep.distances[n.id]
        const prev = currentStep.previous[n.id]
        const vis = currentStep.visited[n.id]
        const isCurr = currentStep.currentNode === n.id

        return (
          <tr key={n.id} className={`border-1 ${
            isCurr ? 'bg-indigo-950/60 font-medium' : ''
          }`} >
            <td className="py-2.5 px-3 flex items-center">
              <span className={`w-2 h-2 rounded-full ${vis ? 'bg-indigo-950/60' : 'bg-slate-900/60'}`}>
                {n.name}
              </span>
            </td>
            <td className="py-2.5 px-3 font-medium">
              {dist === 999 ? '∞' : dist}
            </td>
            <td className="py-2.5 px-3 text-slate-400">
              {prev || '-'}
            </td>
          </tr>
        );
      }
    }
  </tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
  <div>
    <h4 className="text-lg font-bold text-slate-400">Алгоритм
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map((item) => {
        const isActive = currentStep.activeCodeLine === item.line
        return (
          <div key={item.line} className={`py-1 px-2 ${
            isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-400'
          }`} >
            {item.line}
          </div>
        )
      })}
    </div>
  </div>
</div>

```

```
-----
  /* 4 */ "      memo[n] = fibMemo(n - 1, memo) + fibMemo
  /* 5 */ "      return memo[n];",
  /* 6 */ "}"
];
```

```
export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |
```

```
useEffect(() => {
  const generatedSteps: DPStep[] = [];
  let stepId = 0;
  const memoMap: { [key: number]: number } = {};

  function simulateFib(n: number): number {
    generatedSteps.push({
      id: stepId++,
      n,
      action: 'call',
      desc: `Вызов fibMemo(${n}). Проверяем наличие в
      codeLine: 2,
      memoState: { ...memoMap }
    });
```

```
    if (n in memoMap) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'memo_hit',
        desc: `✂ Кэш-хит! fibMemo(${n}) уже посчитано
        codeLine: 2,
        memoState: { ...memoMap }
      });
      return memoMap[n];
```

```

    const finalRes = simulateFib(targetN);

    generatedSteps.push({
      id: stepId++,
      n: targetN,
      action: 'done',
      desc: `  Расчет завершен! fibMemo(${targetN}) = $
      codeLine: 6,
      memoState: { ...memoMap }
    });

    setSteps(generatedSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
  }, [targetN]);

  useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length -
      timer = setTimeout(() => {
        setCurrentStepIndex(prev => prev + 1);
      }, speed);
    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearTimeout(timer);
  }, [isPlaying, currentStepIndex, steps, speed]);

  const currentStep = steps[currentStepIndex] || null;
  const memoState = currentStep?.memoState || {};

  return (
    <div className="bg-slate-900 rounded-2xl border bor
      /* Header & Controls */
    <div className="flex flex-col lg:flex-row lg:item
      <div>
        <div className="flex items-center gap-3 mb-2">

```

```

        title="Шаг назад"
      >
        <ChevronLeft className="w-5 h-5" />
      </button>
      <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1.5 px-2 py-2
          ${isPlaying
            ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
            : 'bg-emerald-600 text-white hover:bg-emerald-700'}
        `}
      >
        {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
        {isPlaying ? 'Пауза' : 'Пуск'}
      </button>
      <button
        onClick={() => { setIsPlaying(false); setSteps([0]); }}
        disabled={currentStepIndex === steps.length - 1}
        className="p-2 text-slate-400 hover:text-slate-300"
        title="Шаг вперед"
      >
        <ChevronRight className="w-5 h-5" />
      </button>
    </div>
  </div>
</div>

```

```

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded-lg" >
  <h4 className="text-xs font-bold uppercase tracking-wide" >DP Table View
  <div className="flex flex-wrap items-center gap-2" >
    {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
      const isCached = num in memoState || num <= 2;
      const val = num <= 2 ? 1 : memoState[num];
      const isCurrent = currentStep?.n === num;

      return (
        <div

```

```

        ? 'border-emerald-500 text-emerald-400 bg
        : 'border-transparent text-slate-400 hove
    }` }
  >
    <Eye className="w-4 h-4" />
    Стен-бай-стен состояние
  </button>
  <button
    onClick={() => setActiveTab('code')}
    className={`flex items-center gap-2 px-6 py-3
      activeTab === 'code'
        ? 'border-emerald-500 text-emerald-400 bg
        : 'border-transparent text-slate-400 hove
    }` }
  >
    <Code className="w-4 h-4" />
    Интерактивный код
  </button>
</div>

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify
  {activeTab === 'table' && {
    <div className="bg-slate-950 p-6 rounded-2xl
      <h5 className="font-bold text-white text-sm
        <span className="text-emerald-400"> </span>
      </h5>
      <p className="text-sm text-slate-400">
        В классической рекурсии для N={targetN} п
        вычисленное значение в таблице, мы момента
      </p>
      <div className="p-4 bg-slate-900 rounded-xl
        t-slate-300">
        <span>Всего шагов симуляции: {steps.length}
        <span className="text-emerald-400 font-bo
      </div>
    </div>
  }
})

```

```
        <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
      </div>
      <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
        <div
          className="h-full bg-gradient-to-r from-emerald-500 to-teal-500"
          style={{ width: `${(currentStepIndex + 1) * 100 / steps.length}%` }}
        ></div>
      </div>
    </div>
  </div>
);
}
```

ФАЙЛ 36/82: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },
  { id: 1, label: "B", x: 280, y: 140 },
  { id: 2, label: "C", x: 100, y: 140 },
  { id: 3, label: "D", x: 280, y: 250 },
  { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];

export function EulerSimulator() {
```



```

    } else {
      setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! C
    }
    setC1Done(true);
  } else {
    setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $
  }
};

```

```

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
      <button onClick={resetC1} className="flex items
        old border border-slate-700 transition-all">
        <Undo className="h-3.5 w-3.5" /> C6poc
      </button>
    </div>

    <div className="bg-slate-950 rounded-xl p-4 borde
      -hidden">
      <div className="absolute inset-0 bg-[radial-gra
      <svg className="w-full h-[260px] z-10 relative"
        {C1_EDGES.map((e, i) => {
          const u = C1_NODES.find(n => n.id === e.u)!
          const v = C1_NODES.find(n => n.id === e.v)!
          const key = `${Math.min(e.u,e.v)}-${Math.ma
          const used = c1VisitedEdges.includes(key);
          return <line key={i} x1={u.x} y1={u.y} x2={
            stroke={used ? "#6366f1" : "#475569"}
            strokeWidth={used ? 4 : 2}
            className="transition-all duration-300" />
        })}
      {C1_NODES.map(n => {
        const isLast = c1Path[c1Path.length-1] === n
        const visited = c1Path.includes(n.id);
        return <g key={n.id} className="cursor-poin
          {isLast && <circle cx={n.x} cy={n.y} r={1

```

```
    id: number;
    question: string;
    options: string[];
    correctAnswer: number;
    explanation: string;
}

const quizQuestions: Question[] = [
  {
    id: 1,
    question: 'Что мы делаем в алгоритме Краскала, если  
ер, красный) цвет?',
    options: [
      'Мы радуемся и добавляем его в минимальное остовное дерево',
      'Мы перекрашиваем их в синий цвет и делим граф по  
компонентам связности',
      'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
      'Мы вызываем алгоритм Дейкстры для поиска нового ребра'
    ],
    correctAnswer: 2,
    explanation: 'Верно! Если вершины уже в одном клане  
и добавим ребро, это нарушит структуру дерева.'
  },
  {
    id: 2,
    question: 'Почему алгоритм Дейкстры впадает в ступор?',
    options: [
      'Потому что он написан только для положительных весов',
      'Он жадный и считает, что каждый следующий шаг дает  
наилучший результат, не учитывая уже закрытые вершины.',
      'Отрицательные ребра создают гравитационный коллапс',
      'Дейкстра просто не любил отрицательные балансы на счетах'
    ],
    correctAnswer: 1,
    explanation: 'Именно! Дейкстра уверен: если ты приедешь  
раньше, чем у тебя есть план, то тебе не нужен алгоритм Беллмана-Форда.'
  },
  {
    id: 3,
```

```
export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<number>(0);
  const [selectedOption, setSelectedOption] = useState<string>('');
  const [isAnswered, setIsAnswered] = useState<boolean>(false);
  const [score, setScore] = useState<number>(0);
  const [showResults, setShowResults] = useState<boolean>(false);

  const currentQ = quizQuestions[currentQIndex];

  const handleSelectOption = (index: number) => {
    if (isAnswered) return;
    setSelectedOption(index);
    setIsAnswered(true);
    if (index === currentQ.correctAnswer) {
      setScore(prev => prev + 1);
    }
  };

  const handleNext = () => {
    if (currentQIndex + 1 < quizQuestions.length) {
      setCurrentQIndex(prev => prev + 1);
      setSelectedOption('');
      setIsAnswered(false);
    } else {
      setShowResults(true);
    }
  };

  const handleRestart = () => {
    setCurrentQIndex(0);
    setSelectedOption('');
    setIsAnswered(false);
    setScore(0);
    setShowResults(false);
  };

  if (showResults) {
    return (

```

```

    <span className="bg-purple-600 text-white px-4">
      Экспресс-тест
    </span>
    <h3 className="text-xl font-bold text-white">
  </div>
  <span className="text-sm font-mono text-slate-400">
    Bonpoc {currentQIndex + 1} / {quizQuestions.length}
  </span>
</div>

<div className="mb-8">
  <h4 className="text-xl md:text-2xl font-bold text-slate-800">
    {currentQ.question}
  </h4>

  <div className="space-y-4">
    {currentQ.options.map((option, idx) => {
      const isSelected = selectedOption === idx;
      const isCorrect = idx === currentQ.correctAnswer;

      let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
      if (isAnswered) {
        if (isCorrect) {
          optionStyle = "bg-emerald-950/80 border-emerald-500";
        } else if (isSelected) {
          optionStyle = "bg-rose-950/80 border-rose-500";
        } else {
          optionStyle = "bg-slate-800/40 border-slate-700";
        }
      }
    })}

    return (
      <div
        key={idx}
        onClick={() => handleSelectOption(idx)}
        className={"flex items-start gap-4 p-4"}
      >
        <div className="mt-0.5 shrink-0">

```

```
        (!isAnswered
          ? "bg-slate-800 text-slate-500 border border-slate-800"
          : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700")
      }
    >
    {currentQIndex + 1 < quizQuestions.length ? 'next' : 'finish'}
  </button>
</div>
</div>
);
};
```

ФАЙЛ 38/82: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect } from 'react';

interface Step {
  k: number; i: number; j: number;
  matrix: number[][]; updated: boolean;
  log: string; activeCodeLine: number;
}

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5'];
  const nodesSvg = [
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
  ];
```

```

simulationSteps.push({
  k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
  log: '  Инициализация матрицы прямых путей,  $\infty$  (999)',
  activeCodeLine: 2,
});

const N = 7;
for (let k = 0; k < N; k++) {
  simulationSteps.push({
    k, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: `+ Внешний цикл. Разрешаем пути через пром. узлы`,
    activeCodeLine: 3,
  });

  for (let i = 0; i < N; i++) {
    for (let j = 0; j < N; j++) {
      if (i === j || i === k || j === k) continue;

      const oldVal = dist[i][j];
      const viaVal = dist[i][k] + dist[k][j];
      if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
        dist[i][j] = viaVal;
        simulationSteps.push({
          k, i, j, matrix: dist.map(r => [...r]),
          log: `  Улучшение пути [${i}]→[${j}] через [${k}].`,
          activeCodeLine: 7,
        });
      }
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
  log: '  Все пары отработаны. Алгоритм Флойда завершён.',
  activeCodeLine: 8,
});

```

```

        <button onClick={() => { setIsPlaying(false);
            ; }} className="px-3 py-2 bg-emerald-600 te
    </div>
</div>

<div className="flex flex-col lg:flex-row gap-6 ml
    { /* Граф */ }
    <div className="w-full lg:w-2/3 bg-indigo-950/2
        stify-center min-h-[400px]">
        <div className="absolute top-3 left-3 bg-indi
            ont-bold shadow-sm">
            {currentStep.k >= 0 && currentStep.k < 7 ?
        </div>

    <svg className="w-full h-auto max-h-[500px]" >
        <defs>
            <marker id="arrow-fw-normal" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
            <marker id="arrow-fw-active" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
            <marker id="arrow-fw-via" markerWidth="6"
            th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
        </defs>
        {Object.keys(adjList).flatMap((uStr) => {
            const u = parseInt(uStr);
            return adjList[u].map((edge) => {
                const uNode = nodesSvg.find((n) => n.id
                const vNode = nodesSvg.find((n) => n.id
                const isTarget = currentStep.i === u &&
                const isVia = (currentStep.i === u && c
                let strokeColor = '#475569'; let marker
                if (isTarget) { strokeColor = '#10b981'
                else if (isVia) { strokeColor = '#818cf8
                return (
                    <g key={` ${u} - ${edge.v} `}>
                        <line x1={uNode.x} y1={uNode.y} x2=
                    markerEnd={marker} strokeDasharray={isTarg
                        <circle cx={({uNode.x + vNode.x) / 2

```

```

const u = parseInt(uStr);
const isI = currentStep.i === u;
const isK = currentStep.k === u;
return (
  <div key={u} className={`flex flex-wrap
    isI ? 'bg-amber-900/40 border-amber-
    isK ? 'bg-indigo-900/40 border-ind
  `}>
    <span className="font-bold text-white
e-400">→</span>
    {adjList[u].map((edge, idx) => {
      const isUpd = currentStep.i === u
      const isVia = currentStep.k !== -
rentStep.j === edge.v);
      return (
        <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
        {edge.v} ({edge.w})
        </span>
      )
    })}
  </div>
);
}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  /* Матрица расстояний */
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div className="flex justify-between items-center">
      <h4 className="text-lg font-bold text-rose-100">
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-center border-collapse">
        <thead><tr className="bg-rose-950/40 text-rose-100">
          <th className="p-2 border-r border-rose-100">

```



```
        </div>
      )))
    </div>
  </div>
</div>
</div>
</div>
);
}
```

ФАЙЛ 39/82: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```

        steps.push({ type: 'backtrack', node: v, desc:
            ueOrStack: [...stack], visitedNodes: Array.from(
        }

    dfs(start);
    return steps;
}

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Нача
        ray.from(vis)] }));

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Д
            q], visitedNodes: [...Array.from(vis)] }));
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
                q.push(u);
                steps.push({ type: 'visit', node: u, de
                    tack: [...q], visitedNodes: [...Array.from(
                }
            }
        }

        steps.push({ type: 'backtrack', node: '', desc: `Оч
            om(vis)] }));

    return steps;
}

```

```

        <button onClick={() => setMode("dfs")}
            className={`px-4 py-2 text-xs font-
text-slate-400 hover:text-slate-300`} `>
            DFS (В глубину)
        </button>
        <button onClick={() => setMode("bfs")}
            className={`px-4 py-2 text-xs font-
"text-slate-400 hover:text-slate-300`} `>
            BFS (В ширину / Волна)
        </button>
    </div>

    <div className="flex flex-col md:flex-row g
        {/* SVG Graph View */}
        <div className="bg-slate-900 border bor
ive min-h-[300px]">
            <svg className="w-full h-full max-
                {/* Edges */}
                {edges.map((e, i) => {
                    const u = nodes.find(n => n
                    const v = nodes.find(n => n
                    // Check if edge is part of
                    const edgeTraversed = isVis

                    return (
                        <line key={i} x1={u.x}
                            className={`stroke
500' : 'stroke-emerald-500') : 'stroke-slate
                        });
                    }}}

                {/* Nodes */}
                {nodes.map(n => {
                    const visited = isVisited(n
                    const current = isCurrent(n

                    let fillColor = "fill-slate
                    let strokeColor = "stroke-s

```



```

    emoji: string;
    symptom: string;
    priority: number; // 1 to 99 (Severity)
    animState: 'in' | 'idle' | 'winner' | 'out';
}

```

// Max-heap helpers for Patients

```

function siftUp(arr: Patient[]): Patient[] {
    const a = arr.map(x => ({ ...x }));
    let idx = a.length - 1;
    while (idx > 0) {
        const parent = Math.floor((idx - 1) / 2);
        if (a[parent].priority < a[idx].priority) {
            [a[parent], a[idx]] = [a[idx], a[parent]];
            idx = parent;
        } else break;
    }
    return a;
}

```

```

function siftDown(arr: Patient[]): Patient[] {
    const a = arr.map(x => ({ ...x }));
    const n = a.length;
    let idx = 0;
    while (true) {
        let largest = idx;
        const l = 2 * idx + 1;
        const r = 2 * idx + 2;
        if (l < n && a[l].priority > a[largest].priority) largest = l;
        if (r < n && a[r].priority > a[largest].priority) largest = r;
        if (largest !== idx) {
            [a[largest], a[idx]] = [a[idx], a[largest]];
            idx = largest;
        } else break;
    }
    return a;
}

```

```
})();
```

```
let uid = 300;
```

```
export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>([]);
  const [customName, setCustomName] = useState('');
  const [customPriority, setCustomPriority] = useState(0);
  const [log, setLog] = useState<string[]>([]);
  const [shakeEmpty, setShake] = useState(false);
  const [busy, setBusy] = useState(false);
  const [healingPatient, setHealing] = useState<Patient>({});

  const addLog = (msg: string) => setLog(prev => [...prev, msg]);

  // --- INSERT: Новый пациент поступает в приемный покой
  const insertPatient = useCallback((name: string, priority: number) => {
    if (busy || heap.length >= 15) {
      addLog('⚠ Все палаты переполнены! Дождитесь выписки');
      setShake(true);
      setTimeout(() => setShake(false), 400);
      return;
    }
    setBusy(true);

    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const finalName = name || preset.name.split(' ')[0];
    const finalEmoji = preset.emoji;
    const finalSymptom = preset.symptom;

    const newPatient: Patient = {
      id: `p${uid++}`,
      name: finalName,
      emoji: finalEmoji,
      symptom: finalSymptom,
      priority,
      animState: 'in',
    };
  }, [busy, heap, setShake, setLog, setBusy, setHealing]);
```

```

const topPatient = heap[0];
setHealing(topPatient);

addLog(` СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

// Step 1: Root lights up as winner
setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 'winning' } : x));

setTimeout(() => {
  // Step 2: Root exits the tree, tree reorganizes
  setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 'leaving' } : x));

  setTimeout(() => {
    // Run binary heap extraction
    setHeap(prev => {
      if (prev.length === 0) return [];
      const a = prev.map(x => ({ ...x }));
      a[0] = { ...a[a.length - 1] };
      a.pop();
      return siftDown(a).map(x => ({ ...x, animStat: 'sifting' }));
    });

    // Patient gets healed in the bed
    setTimeout(() => {
      setHealing(prev => prev ? { ...prev, animStat: 'healing' } : null);
      addLog(` Успех! Пациент ${topPatient.name} излечен!`);
      setTimeout(() => {
        setHealing(null);
        setBusy(false);
      }, 600);
    }, 800);

    }, 350);
  }, 650);
}, [busy, heap]);

const reset = () => {
  setHeap(INITIAL PATIENTS.map(x => ({ ...x, animStat: 'initial' })));
}

```

```

<div className="text-3xl"> </div>
<div>
  <h3 className="font-black text-emerald-400 text-center">
    Priority Queue</h3>
  <p className="text-xs text-slate-300 mt-1 leading-relaxed">
    В реанимации не важно, кто пришел раньше, а кто в
    остояние</b> (максимальный приоритет).
    Куча (Heap) автоматически перестраивает пац
    оказывался на самом верху дерева (в корне).
  </p>
</div>
</div>

{/* УПРАВЛЕНИЕ */}
<div className="flex flex-col lg:flex-row items-start gap-4">

  {/* Добавить случайного */}
  <button
    onClick={handleRandomInsert}
    disabled={busy || heap.length >= 15}
    className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-400 to-slate-500
      opacity-40 disabled:cursor-not-allowed text-white font-medium
      rounded-lg shadow-sm transition-all cursor-pointer focus:outline-none"
  >
    <span> Привезти по Скорой (INSERT)</span>
  </button>

  {/* Добавить кастомного */}
  <form onSubmit={handleCustomInsert} className="flex-1">
    <input
      type="text"
      required
      value={customName}
      onChange={e => setCustomName(e.target.value)}
      placeholder="Имя пациента"
      className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:text-slate-400"
    />
  </form>
</div>

```



```

    </button>
  </div>

  {/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
  <div className="grid grid-cols-1 lg:grid-cols-12" >

    {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
    <div className={`lg:col-span-8 bg-slate-900 rounded-[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`}>
      <div className="absolute top-4 left-4 text-[18px] font-medium">
        Сортировка по тяжести (Двоичная куча / Max-Heap)
      </div>

      <div className="relative w-full h-full min-h-[400px]">
        <svg className="absolute inset-0 w-full h-full" >
          {edges}
        </svg>

        {heap.map(({patient, idx}) => {
          const pos = nodePos(idx);
          const isRoot = idx === 0;
          return (
            <div
              key={patient.id}
              className={`
                absolute flex flex-col items-center
                -translate-x-1/2 -translate-y-1/2 transition-all
                ${patient.animState === 'in' ? 'animate-in' : ''}
                ${patient.animState === 'winner' ? 'animate-winner' : ''}
                ${patient.animState === 'out' ? 'animate-out' : ''}
                ${isRoot ? 'bg-rose-950/80 border-rose-500' : ''}
              `}
              style={{
                left: `${pos.x}%`,
                top: `${pos.y}%`,
                width: isRoot ? 60 : 52,
                height: isRoot ? 60 : 52,
              }}
            </div>
          )
        })}
      </div>
    </div>
  </div>

```

```

        <span>Пациент: {healingPatient.name}
      </h4>
      <p className="text-emerald-400 text-xl">
        Состояние: Стабилизация ({healingPatient.status})
      </p>
      <div className="flex justify-center gap-4">
        <span className="w-2 h-2 rounded-full bg-emerald-400">
          <Heart className="w-4 h-4 text-rose-500"/>
        </span>
      </div>
    </div>
  </div>
) : (
  <div className="text-center text-slate-600">
    <span className="text-5xl mb-2 block">Ждём вас</span>
    <p className="text-xs font-bold font-mono">Ваш врач</p>
    <p className="text-[10px] mt-1">Ждём вас</p>
  </div>
)}
</div>

<div className="w-full h-3 bg-gradient-to-r from-emerald-400 to-emerald-500">
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4">
  <div className="text-[10px] font-mono text-slate-400">
    {log.map((entry, idx) => (
      <div
        key={idx}
        className={`text-xs font-mono flex items-center justify-between p-2`}
      >
        {idx === 0 ? <span className="text-emerald-400 font-weight-bold">Начало</span> : }
        <span>{entry}</span>
      </div>
    ))}
  </div>
</div>

```

```

    return () => clearInterval(t);
  }, [steps, period, paused]);

  return step;
}

export const C = {
  idle: "#334155",
  idleStroke: "#475569",
  text: "#e2e8f0",
  dim: "#94a3b8",
  active: "#6366f1",
  done: "#10b981",
  warn: "#f59e0b",
  bad: "#f43f5e",
  edge: "#475569",
};

export const Svg: React.FC<{ children: React.ReactNode;
const paused = useContext(DemoPauseContext);
return (
  <div className="w-full">
    <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
      {children}
    </svg>
    {caption && (
      <div className="text-[10px] text-slate-400 text-center">
        {caption}
      </div>
    )}
    <div className="text-[9px] text-slate-600 text-center">
      {paused ? "пауза — курсор на карточке" : "наведи"}
    </div>
  </div>
);
};

export const Node: React.FC<{

```

```
import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
  const step = useLoop(3, 1200);
  // i -> j напрямую 9; через k=1 получаем 3+4=7
  const pos: Record<string, [number, number]> = { i: [4, 1], j: [4, 4], k: [3, 4] };
  const viaOpen = step >= 1;
  const relaxed = step >= 2;
  return (
    <Svg
      caption={
        step === 0
          ? "k ещё запрещена: путь i → j = 9"
          : step === 1
            ? "Разрешаем пересадку через k: 3 + 4 = 7"
            : "7 < 9 → D[i][j] = 7"
        }
    >
      <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.ok} />
      <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.ok} />
      <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.ok} />

      <text x={130} y={112} textAnchor="middle" fontSize=
        9
      </text>
      <text x={72} y={54} textAnchor="middle" fontSize=
        3
      </text>
      <text x={188} y={54} textAnchor="middle" fontSize=
        4
      </text>
    </Svg>
  );
};
```

```

-----
/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
  const step = useLoop(3, 1300);
  const pos: Record<string, [number, number]> = { A: [50, 50], B: [150, 50], C: [250, 50], D: [350, 50], E: [450, 50], F: [550, 50], G: [650, 50], H: [750, 50], I: [850, 50], J: [950, 50] };
  const edges: [string, string, number][] = [
    ["A", "B", 1],
    ["B", "C", 1],
    ["C", "D", 1],
    ["D", "E", 1],
    ["E", "F", 1],
    ["F", "G", 1],
    ["G", "H", 1],
    ["H", "I", 1],
    ["I", "J", 1],
    ["A", "D", 2],
    ["B", "E", 2],
    ["C", "F", 2],
    ["D", "G", 2],
    ["E", "H", 2],
    ["F", "I", 2],
    ["G", "J", 2],
    ["A", "E", 3],
    ["B", "F", 3],
    ["C", "G", 3],
    ["D", "H", 3],
    ["E", "I", 3],
    ["F", "J", 3]
  ];
  return (
    <Svg
      caption={
        step === 0
          ? "Вершины – объекты (города, слова, состояния)"
          : step === 1
            ? "Рёбра – связи между ними"
            : "Вес ребра – цена перехода"
      }
    >
      {edges.map(([u, v, w], i) => {
        const a = pos[u];
        const b = pos[v];
        return (
          <g key={i}>
            <Edge a={a} b={b} color={step >= 1 ? C.active : C.inactive}>
              {step >= 2 && (
                <text
                  x={{a[0] + b[0]} / 2}
                  y={{a[1] + b[1]} / 2 - 4}
                  textAnchor="middle"
                  fontSize={10}
                  fontWeight="700"
                  fill={C.warn}
                >
                  {w}
                </text>
              )}
            </g>
          )}
      )}
    </Svg>
  );
}
}}}
{Object.entries(pos).map(([k, [x, y]]) => (
  <Node key={k} x={x} y={y} label={k} fill={C.idle} />
))}

```

```

        </text>
      </g>
    )))

    {step >= 2 &&
      raw.map(({v, i}) => {
        <g key={`c${i}`}>
          <rect x={startX + i * (boxW + 4)} y={98} wi
          <text x={startX + i * (boxW + 4) + boxW / 2
            {sorted.indexOf(v)}
          </text>
        </g>
      )))
    </Svg>
  );
};

/* ----- Splay: три случая поворота ---

type Pt = [number, number];

const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, st
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : i

  return (
    <Svg caption={captions[step]}>
      {f.edges.map(([a, b], i) => {
        <line
          key={i}
          x1={f.pos[a][0]}
          y1={f.pos[a][1]}

```

```

    }}
  />
};

/** Zig-Zag – x и p с разных сторон, крутим сначала ниже
export const ZigZagDemo: React.FC = () => {
  <SplayFrames
    captions={["Зигзаг: g влево, x вправо", "Поворот НИЗКО"]
    frames=[
      { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
      { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
      { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
    ]}
  />
};

```

ФАЙЛ 43/82: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "A"],
];

export const BfsDemo: React.FC = () => {
  const order = ["A", "B", "C", "D", "E", "F"];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order[step - 1] : null;
  return (

```

```

const edges: [string, string][] = [["труссы", "штаны"]
const order = ["труссы", "носки", "штаны", "боты"];
const step = useLoop(order.length + 1, 800);
const placed = order.slice(0, step);
const pos = Object.fromEntries(nodes.map((n) => [n.id
return (
  <Svg caption={`Порядок: ${placed.join(" → ") || "..."}
    {edges.map(([u, v], i) => {
      <Edge key={i} a={pos[u]} b={pos[v]} color={plac
    })}
    {nodes.map((n) => {
      const idx = placed.indexOf(n.id);
      return (
        <g key={n.id}>
          <rect x={n.x - 34} y={n.y - 12} width={68} h
            " }} />
          <text x={n.x} y={n.y} textAnchor="middle" d
            {idx >= 0 && <circle cx={n.x + 30} cy={n.y
            {idx >= 0 && {
              <text x={n.x + 30} y={n.y - 12} textAncho
              x + 1}</text>
            }}
          </g>
        );
      });
    })}
    <text x={224} y={62} textAnchor="middle" fontSize=
    <text x={224} y={74} textAnchor="middle" fontSize=
  </Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Пут
      <Edge a={[60, 70]} b={[200, 70]} color={improved

```



```

const edges: [string, string][] = [["A", "B"], ["A",
const cut = step === 1;
return (
  <Svg caption={cut ? "Убрали вершину C → две отдельные"
    {edges.map(([u, v], i) => {cut && (u === "C" || v
    {Object.entries(pos).map(([k, [x, y]]) =>
      cut && k === "C" ? (
        <circle key={k} cx={x} cy={y} r={12} fill="no
      ) : (
        <Node key={k} x={x} y={y} label={k} fill={k =
        } />
      )
    )}
  </Svg>
);
};

```

```

export const MstDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];
  const chosen = ["A-D", "A-B", "B-C", "D-E"];
  const step = useLoop(chosen.length + 1, 800);
  const taken = new Set(chosen.slice(0, step));
  return (
    <Svg caption={`Жадно берём самые лёгкие рёбра без ц
      {edges.map((e, i) => {
        const on = taken.has(`${e.u}-${e.v}`);
        return (
          <g key={i}>
            <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
            <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y
              textAnchor="middle" fontSize={9} fontWeigh
          </g>

```

```

const step = useLoop(states.length, 1000);
const heap = states[step];
const pos: [number, number][] = [[130, 24], [80, 68],
const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
return (
  <Svg caption="Новый элемент всплывает наверх, пока"
    <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b=
    <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
    {heap.map((v, i) => {
      <Node key={i} x={pos[i][0]} y={pos[i][1]} label=
    }}}
  </Svg>
);
};

export const StackDemo: React.FC = () => {
  const states = [["A"], ["A", "B"], ["A", "B", "C"], [
  const step = useLoop(states.length, 800);
  const items = states[step];
  const popping = step >= 3;
  return (
    <Svg caption={popping ? "поп: забираем ВЕРХНИЮЮ тапе
    <rect x={90} y={22} width={80} height={96} rx={6}
    {items.map((v, i) => {
      <g key={v} style={{ transition: "all .3s" }}>
        <rect x={94} y={100 - i * 26} width={72} height=
        fill={i === items.length - 1 ? (popping ? C
        <text x={130} y={111 - i * 26} textAnchor="mi
      </g>
    }}}
    <text x={130} y={16} textAnchor="middle" fontSize=
  </Svg>
);
};

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C
  const step = useLoop(states.length, 800);

```

```

    </Svg>
  );
};

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
  { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
  { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
  { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
  { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
  { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

export const TrieDemo: React.FC = () => {
  const step = useLoop(TRIE_NODES.length + 1, 700);
  return (
    <Svg caption="Путь от корня по буквам = слово; общий"
      {TRIE_NODES.filter((n) => n.parent !== null).map(
        (n) => {
          const p = TRIE_NODES[n.parent as number];
          return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
        }
      )}
      {TRIE_NODES.map((n) => <Node key={n.id} x={n.x} y={n.y} ch={n.ch} />)}
    </Svg>
  );
};

export const TrieBfsDemo: React.FC = () => {
  const levels = [[0], [1, 2], [3, 4, 5]];
  const step = useLoop(levels.length + 1, 950);
  const doneIds = new Set(levels.slice(0, step).flat());
  const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];
  return (
    <Svg caption={`BFS по бору: уровень ${Math.max(0, Math.min(step - 1, levels.length - 1))}`}
      {TRIE_NODES.filter((n) => n.parent !== null).map(
        (n) => {
          const p = TRIE_NODES[n.parent as number];
          return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
        }
      )}
      {TRIE_NODES.map((n) => <Node key={n.id} x={n.x} y={n.y} ch={n.ch} />)}
    </Svg>
  );
};

```

```

    { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
    { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
    { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
    { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
  ];
  return (
    <Svg caption="Запрос собирается из  $O(\log n)$  готовых"
      {nodes.map((n, i) => (
        <g key={i}>
          <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={n.h}
            fill={n.lvl === step ? C.active : C.idle} stroke={C.dim} />
          <text x={n.x} y={n.y} textAnchor="middle" domClass={C.dim}>{n.label}</text>
        </g>
      )}}
    </Svg>
  );
};

export const PrefixSumDemo: React.FC = () => {
  const a = [3, 1, 4, 1, 5, 9];
  const pref = a.reduce<number[]>((acc, v) => [...acc, v], []);
  const step = useLoop(4, 1200);
  const l = 1, r = 3;
  const show = step >= 1;
  return (
    <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -`
      : "о префиксные суммы P"}>
      <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
      {a.map((v, i) => (
        <g key={i}>
          <rect x={20 + i * 39} y={24} width={34} height={12}
            fill={show && i >= l && i <= r ? C.active : C.idle} stroke={C.dim} />
          <text x={37 + i * 39} y={36} textAnchor="middle" domClass={C.dim}>{v}</text>
        </g>
      )}}
      <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
      {pref.map((v, i) => (

```

```

    <Svg caption={`Сделано ${step} операций, суммарно $
      {costs.map((c, i) => (
        <rect key={i} x={14 + i * 30} y={104 - c * 10}
          fill={i < step ? (c > 2 ? C.bad : C.done) : C
        )}}
      <line x1={8} y1={106} x2={252} y2={106} stroke={C
      <text x={130} y={20} textAnchor="middle" fontSize=
    </Svg>
  );
};

```

```

export const NpDemo: React.FC = () => {
  const step = useLoop(2, 1400);
  return (
    <Svg caption={step === 0 ? "Найти решение – долго (1
      <ellipse cx={130} cy={64} rx={112} ry={48} fill="
      <ellipse cx={92} cy={64} rx={52} ry={32} fill="#3
      <text x={92} y={64} textAnchor="middle" dominantB
      <text x={206} y={34} textAnchor="middle" fontSize=
      <circle cx={196} cy={84} r={13} fill={step === 1
      <text x={196} y={84} textAnchor="middle" dominant
        ?`}</text>
      <text x={130} y={124} textAnchor="middle" fontSiz
    </Svg>
  );
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} hei

```

```
-----
    <text x={130} y={122} textAnchor="middle" fontSize=
      каждая клетка считается один раз и переиспользуе
    </text>
  </Svg>
);
};
-----
```

ФАЙЛ 45/82: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
-----
import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
  AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
  SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"
  | "segment-tree" | "bridge" | "articulation" | "mst"
  | "prefix-function" | "dp" | "floyd" | "components" |
  | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,
  heap: HeapDemo,
  stack: StackDemo,
  queue: QueueDemo,
  dsu: DsuDemo,
  "prefix-sum": PrefixSumDemo,
  "suffix-link": SuffixLinkDemo,
  "segment-tree": SegmentTreeDemo,
  "bridge": BridgeDemo,
  "articulation": ArticulationDemo,
  "mst": MstDemo,
  "dp": DpDemo,
  "floyd": FloydDemo,
  "components": ComponentsDemo,
  "zig": ZigDemo,
  "zig-zig": ZigZigDemo,
  "zig-zag": ZigZagDemo,
};
-----
```

```
        </div>
      </DemoPauseContext.Provider>
    );
  };
};
```

ФАЙЛ 46/82: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
```

```
interface Props {
  vizId: string | null;
  onClose: () => void;
}
```

```
/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
```

```
  useEffect(() => {
    if (!vizId) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      window.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [vizId, onClose]);
```

```
const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
```

```

-----
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}

interface Props {
  anchor: HintAnchor | null;
  onClose: () => void;
  onOpenSimulator: (vizId: string) => void;
  onCardEnter: () => void;
  onCardLeave: () => void;
}

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor,
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: number}>({
    top: 0, left: 0,

  useLayoutEffect(() => {
    if (!anchor) {
      setPos(null);
      return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300;

    let left = anchor.rect.left + anchor.rect.width / 2;
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;

```



```

<div className="bg-slate-900 border border-indigo-500" >
  <div className="flex items-start gap-2 px-3 pt-3" >
    <h4 className="text-sm font-bold text-indigo-500" >
      <button
        onClick={onClose}
        className="text-slate-500 hover:text-white"
        aria-label="Заккрыть подсказку"
      >
        <X className="w-4 h-4" />
      </button>
    </div>

    <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
      <p className="text-[13px] leading-relaxed text-slate-300" >
        {entry.demo && <MiniDemo kind={entry.demo} />}
      </p>
      {entry.formal && (
        <p className="text-[11.5px] leading-relaxed text-slate-300" >
          {entry.formal}
        </p>
      )}
      {entry.complexity && (
        <div className="text-[11px] font-mono text-slate-300" >
          Сложность: {entry.complexity}
        </div>
      )}
      {viz && simulatorId && (
        <button
          onClick={() => onOpenSimulator(simulatorId)}
          className="w-full flex items-center justify-center text-white rounded-lg py-2 transition-colors"
        >
          <Play className="w-3.5 h-3.5" /> Показать
        </button>
      )}
    </div>
  </div>

```

```

        return true;
    };

    const getPotential = (id: string) => {
        if (step >= 2 && step < 7) {
            if (id === 'S') return 0;
            if (id === 'A') return 0;
            if (id === 'B') return -2;
            if (id === 'C') return -1;
        }
        return null;
    };

    const getEdgeColorClass = (e: any) => {
        if (e.u === 'S') {
            if (step === 0 || step === 7) return "opacity-0";
            if (step === 1) return "stroke-amber-500 stroke-width-2px";
            if (step === 2) return "stroke-pink-500 stroke-width-2px";
            return "stroke-slate-700 stroke-dasharray-4 4 stroke-width-2px";
        }

        if (e.id === 'AB') {
            if (step === 4) return "stroke-amber-400 stroke-width-2px";
            if (step > 4) return "stroke-emerald-500";
        }
        if (e.id === 'BC') {
            if (step === 5) return "stroke-amber-400 stroke-width-2px";
            if (step > 5) return "stroke-emerald-500";
        }
        if (e.id === 'CA') {
            if (step === 6) return "stroke-amber-400 stroke-width-2px";
            if (step > 6) return "stroke-emerald-500";
        }

        if (step >= 4) return "stroke-slate-600"; // other edges
        return "stroke-slate-500";
    };

```

```

};

const isEdgeTextHighlighted = (e: any) => {
  if (e.id === 'AB' && step === 4) return true;
  if (e.id === 'BC' && step === 5) return true;
  if (e.id === 'CA' && step === 6) return true;
  return false;
};

const isEdgeTextEmerald = (e: any) => {
  if (e.id === 'AB' && step > 4) return true;
  if (e.id === 'BC' && step > 5) return true;
  if (e.id === 'CA' && step > 6) return true;
  return false;
};

const getDesc = () => {
  switch (step) {
    case 0: return (
      <div>
        <b>Исходный граф.</b> Имеется ребро
        Если мы запустим быстрый алгоритм Д
      </div>
    );
    case 1: return (
      <div>
        <b>Шаг 1: Точка отсчета.</b><br/>
        Добавляем фиктивную вершину S. Соед
ми</span>.
        Это наша база для расчета потенциал
      </div>
    );
    case 2: return (
      <div>
        <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b>
        Запускаем медленный, но мощный алгор
ных вершин:
        <span className="text-pink-300 ml-1

```

```

        );
        default: return "";
    }
};

return (
    <div className="bg-slate-800 p-4 rounded-xl border"
        <h4 className="text-sm md:text-base font-bo
            <span className="w-2.5 h-2.5 bg-amber-400
                Визуализация: Перевзвешивание алгоритмов
            </h4>

        <div className="flex flex-col md:flex-row gap-4"
            <div className="bg-[#0f172a] flex-1 min-h-100
                nter p-4">
                    <svg width="400" height="300" class="
                        <defs>
                            <marker id="arrow-slate" viewbox="0 0 10 5"
                                start-reverse">
                                    <path d="M 0 0 L 10 5 L 10 0" />
                                </marker>
                            <marker id="arrow-slate-dim" viewbox="0 0 10 5"
                                uto-start-reverse">
                                    <path d="M 0 0 L 10 5 L 10 0" />
                                </marker>
                            <marker id="arrow-amber" viewbox="0 0 10 5"
                                start-reverse">
                                    <path d="M 0 0 L 10 5 L 10 0" />
                                </marker>
                            <marker id="arrow-pink" viewbox="0 0 10 5"
                                tart-reverse">
                                    <path d="M 0 0 L 10 5 L 10 0" />
                                </marker>
                            <marker id="arrow-emerald" viewbox="0 0 10 5"
                                o-start-reverse">
                                    <path d="M 0 0 L 10 5 L 10 0" />
                                </marker>
                        </defs>

```

```

        {/* Nodes */}
        {nodes.map(n => {
          if (!isNodeVisible(n.id)) return null
          const pot = getPotential(n.id)

          return (
            <g key={n.id} className={
              <circle
                cx={n.x} cy={n.y}
                className={`stroke-1
                  ${n.id === 'S'
                    ? 'fill-[#2e8b57]
                    : 'fill-slate-500
                />
              <text x={n.x} y={n.y}
                -white">
                  {n.label}
                </text>

                {pot !== null && (
                  <g transform={`
                    fade-in zoom-in duration-500">
                      <rect x="-100" y="-100"
                        k-500 stroke-1" />
                      <text x="0" y="0"
                        font-bold fill-pink-400">
                        {pot}
                      </text>
                    </g>
                  )}
                </g>
              )}
            )}
          </svg>
        </div>

        <div className="w-full md:w-[320px] flex

```

```
    );  
}
```

ФАЙЛ 49/82: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";  
import {  
  Play,  
  Pause,  
  RotateCcw,  
  Layers,  
} from "lucide-react";
```

```
interface GNode {  
  id: number;  
  x: number;  
  y: number;  
  label: string;  
}
```

```
interface GEdge {  
  u: number;  
  v: number;  
}
```

```
const initialNodes: GNode[] = [  
  { id: 0, x: 150, y: 100, label: "0" },  
  { id: 1, x: 300, y: 100, label: "1" },  
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0  
  { id: 3, x: 450, y: 160, label: "3" },  
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se  
];
```

```
const initialEdges: GEdge[] = [
```

```

    ) => {
      generatedSteps.push({
        phase,
        desc,
        visited: new Set(visited),
        currentNode,
        order: [...order],
        transposed,
        sccMap: { ...sccMap },
        currentSccId,
        activeEdges: transposed
          ? initialEdges.map((e) => ({ u: e.v, v: e.u }
            : [...initialEdges],
      ));
    };
  };

  recordStep(
    "init",
    "Инициализация алгоритма Косарайю. Начинаем первый",
    null,
    false,
    -1,
  );

  // Adj list
  const adj: number[][] = Array.from({ length: 5 }, () => []);
  initialEdges.forEach((e) => adj[e.u].push(e.v));

  // DFS 1: post-order list
  const dfs1 = (u: number) => {
    visited.add(u);
    recordStep(
      "dfs1",
      `DFS 1: зашли в вершину ${u}, помечаем посещенной`,
      u,
      false,
      -1,
    );
  };

```

```
// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
  visited.add(u);
  sccMap[u] = currentScc;
  recordStep(
    "dfs2",
    `DFS 2: заходим в ${u} на транспонированном графе`,
    u,
    true,
    currentScc,
  );

  for (const to of adjT[u]) {
    if (!visited.has(to)) {
      dfs2(to, currentScc);
    }
  }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
  const u = order[i];
  if (!visited.has(u)) {
    sccId++;
    recordStep(
      "dfs2",
      `DFS 2: берем следующую непосещенную из стека`,
      u,
      true,
      sccId,
    );
    dfs2(u, sccId);
  } else {
    recordStep(
      "dfs2",

```



```

    } else {
      setIsPlaying(false);
    }
    return () => clearInterval(timer);
  }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getScColor = (id: number) => {
  if (id === 1) return "fill-emerald-600 stroke-emera
  if (id === 2) return "fill-indigo-600 stroke-indigo
  return "fill-slate-800 stroke-slate-500";
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Layers className="w-5 h-5 text-emerald-400"
          Визуализатор Алгоритма Косарайю (Поиск SCC)
        </h3>

      <div className="flex items-center gap-2 bg-slate
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            d-500 text-white"
        >
          {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
          ) : (
            <Play className="w-4 h-4 ml-1" />
          )}
          {isPlaying ? "Пауза " : "Старт "}

```

```

        <path d="M 0 1 L 10 5 L 0 9 z" fill="#1f77b4" stroke="#1f77b4" stroke-width="1" />
    </marker>
    <marker
      id="arrow-transposed"
      viewBox="0 0 10 10"
      refX="22"
      refY="5"
      markerWidth="6"
      markerHeight="6"
      orient="auto-start-reverse"
    >
      <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666" stroke="#666666" stroke-width="1" />
    </marker>
  </defs>

```

```

  { /* Edges */ }
  {initialEdges.map((edge, idx) => {
    const u = initialNodes.find((n) => n.id === edge.u);
    const v = initialNodes.find((n) => n.id === edge.v);

    // If transposed, draw reversed coordinates
    const x1 = step.transposed ? v.x : u.x;
    const y1 = step.transposed ? v.y : u.y;
    const x2 = step.transposed ? u.x : v.x;
    const y2 = step.transposed ? u.y : v.y;

    const isSccEdge =
      step.sccMap[edge.u] &&
      step.sccMap[edge.v] &&
      step.sccMap[edge.u] === step.sccMap[edge.v];
    let stroke = "#475569";
    let markerId = "arrow";

    if (step.transposed) {
      stroke = isSccEdge ? "#818cf8" : "#4f46e5";
      markerId = "arrow-transposed";
    } else {
      if (

```

```

    if (sccId) {
        classes = getSccColor(sccId);
    } else if (isCurrent) {
        classes = "fill-amber-600 stroke-amber-600";
    } else if (isVisited && step.phase === "done") {
        classes = "fill-emerald-700 stroke-emerald-700";
    }

    return (
        <g key={node.id}>
            <circle
                cx={node.x}
                cy={node.y}
                r="20"
                className={`transition-all duration-300`}
                strokeWidth="3"
            />
            <text
                x={node.x}
                y={node.y}
                dy=".3em"
                textAnchor="middle"
                fill="white"
                fontSize="13px"
                fontWeight="bold"
                className="select-none"
            >
                {node.label}
            </text>

            {sccId && (
                <text
                    x={node.x}
                    y={node.y - 28}
                    textAnchor="middle"
                    fill="#10b981"
                    fontSize="10px"
                    fontWeight="bold"
                >
                    {sccId}
                </text>
            )}
        </g>
    );
}

```

```

        <span className="text-slate-600 text-
    ) : {
        [...step.order].reverse().map(({val, i
        <div
            key={idx}
            className="bg-indigo-950 border b
ex items-center gap-1"
        >
            {val}
        </div>
        ))
    })
</div>
</div>

{/* List of actions log */}
<div>
    <span className="text-[10px] text-slate-5
        ЛОГ ОПЕРАЦИЙ:
    </span>
    <div className="space-y-1.5 max-h-[150px]
        {steps
            .slice(0, currentStepIdx + 1)
            .reverse()
            .slice(0, 5)
            .map(({s, idx}) => {
                <div
                    key={idx}
                    className={`text-[11px] p-1.5 roun
                >
                    {s.desc}
                </div>
            }}}
        </div>
    </div>
</div>

<div className="mt-4 pt-3 border-t border-sla

```

```

    label: string;
    x: number;
    y: number;
    color: string; // 'none', 'red', 'blue', 'purple', 'm
}

```

```

interface Edge {
    id: string;
    u: number;
    v: number;
    weight: number;
    status: 'pending' | 'inspecting' | 'included' | 'reje
    color: string;
}

```

```

interface StepLog {
    title: string;
    description: string;
    type: 'info' | 'success' | 'warning' | 'error';
}

```

// 9 Vertices layout

```

const initialVertices: Vertex[] = [
    { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
    { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
    { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
    { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
    { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
    { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
    { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
    { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
    { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

// 12 Edges connecting the 9 vertices

```

const initialEdges: Edge[] = [
    { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending'
    { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending'

```

```

    }, ...prev]));
  },
  // Step 2: Include A-B
  () => {
    setVertices(prev => prev.map(v => v.id === 0 || v.id === 1));
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
      weight: 1,
      type: 'info'
    }, ...prev]));
  },
  // Step 3: Inspect D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
      weight: 3,
      type: 'info'
    }, ...prev));
  },
  // Step 4: Include D-E
  () => {
    setVertices(prev => prev.map(v => v.id === 3 || v.id === 4));
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
      weight: 3,
      type: 'info'
    }, ...prev));
  },
  // Step 5: Inspect B-C (4)
  () => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
      weight: 4,
      type: 'info'
    }, ...prev));
  },

```

```

        type: 'warning'
      }, ...prev]));
    },
    // Step 10: Reject B-E
    () => {
      setEdges(prev => prev.map(e => e.id === '1-4' ? {
        setLogs(prev => [{
          title: 'Отказ: Цикл обнаружен!',
          description: 'Мы выбрасываем ребро B-E. Иначе по
          type: 'error'
        }, ...prev]));
    },
    // Step 11: Inspect E-F (7)
    () => {
      setEdges(prev => prev.map(e => e.id === '4-5' ? {
        setLogs(prev => [{
          title: 'Шаг 6: Берем ребро E-F (вес 7)',
          description: 'Оно соединяет фиолетовую вершину
          type: 'info'
        }, ...prev]));
    },
    // Step 12: Include E-F
    () => {
      setVertices(prev => prev.map(v => v.id === 5 ? {
        setEdges(prev => prev.map(e => e.id === '4-5' ? {
        setLogs(prev => [{
          title: 'Успех: Ребро E-F в осто́ве',
          description: 'Вершина F окрашена в фиолетовый ц
          type: 'success'
        }, ...prev]));
    },
    // Step 13: Inspect D-G (8)
    () => {
      setEdges(prev => prev.map(e => e.id === '3-6' ? {
        setLogs(prev => [{
          title: 'Шаг 7: Берем ребро D-G (вес 8)',
          description: 'Оно соединяет фиолетовую вершину
          type: 'info'

```

```

    },
    // Step 18: Include G-H
    () => {
        setVertices(prev => prev.map(v => v.id === 7 ? {
        setEdges(prev => prev.map(e => e.id === '6-7' ? {
        setLogs(prev => [{
            title: 'Успех: Ребро G-H в острове',
            description: 'Вершина H перекрашена в фиолетовый',
            type: 'success'
        }, ...prev]));
    },
    // Step 19: Inspect E-H (11) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
            title: 'Шаг 10: Берем ребро E-H (вес 11)',
            description: 'Оба конца E и H уже фиолетовые. Соединяет фиолетовую E и фиолетовую H',
            type: 'warning'
        }, ...prev]));
    },
    // Step 20: Reject E-H
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
            title: 'Отказ: Цикл обнаружен!',
            description: 'Выбрасываем ребро E-H.',
            type: 'error'
        }, ...prev]));
    },
    // Step 21: Inspect H-I (12)
    () => {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: 'Шаг 11: Берем ребро H-I (вес 12)',
            description: 'Соединяет фиолетовую H и последнюю фиолетовую G',
            type: 'info'
        }, ...prev]));
    },

```



```

    setVertices(prev => prev.map(v => v.id === 3 ? {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
        ..e, status: 'in_heap' } : e));
    setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D
    setLogs(prev => [{
        title: 'Успех: Плесень захватила вершину D',
        description: 'В кучу добавлены новые пути из D:
        type: 'success'
    }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
        description: 'Плесень расширяется в сторону верш
        type: 'info'
    }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));
    setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
    setLogs(prev => [{
        title: 'Успех: Плесень поглотила вершину A',
        description: 'Новое ребро A-B(2) добавлено в куч
        type: 'success'
    }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к верши
        type: 'info'

```

```

        setLogs(prev => [{
            title: 'Шаг 6: Куча выдает ребро В-Е (вес 6)',
            description: 'ВНИМАНИЕ! Плесень проверяет вершины',
            type: 'warning'
        }, ...prev]));
    },
    // Step 11: Reject В-Е
    () => {
        setEdges(prev => prev.map(e => e.id === '1-4' ? {
            id: '1-4', weight: 4, type: 'tree'
        }, ...e.edges), ...prev);
        setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E-H (10)'], ...prev);
        setLogs(prev => [{
            title: 'Отказ: Игнорируем внутреннее ребро',
            description: 'Ребро В-Е отброшено. Плесень не р',
            type: 'error'
        }, ...prev]));
    },
    // Step 12: Pop E-F (7)
    () => {
        setEdges(prev => prev.map(e => e.id === '4-5' ? {
            id: '4-5', weight: 5, type: 'tree'
        }, ...e.edges), ...prev);
        setLogs(prev => [{
            title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
            description: 'Плесень из центра Токио (Е) тянет',
            type: 'info'
        }, ...prev]));
    },
    // Step 13: Include E-F, add F's edges
    () => {
        setVertices(prev => prev.map(v => v.id === 5 ? {
            id: 5, name: 'F', type: 'vertex'
        }, ...v.edges), ...prev);
        setEdges(prev => prev.map(e => e.id === '4-5' ? {
            id: '4-5', weight: 5, type: 'tree'
        }, ...e.edges), ...prev);
        setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (10)'], ...prev);
        setLogs(prev => [{
            title: 'Успех: Вершина F захвачена',
            description: 'Ребро F-I(13) добавлено в кучу. Р',
            type: 'success'
        }, ...prev]));
    },
    // Step 14: Pop D-G (8)

```

```

    },
    // Step 18: Pop G-H (10)
    () => {
        setEdges(prev => prev.map(e => e.id === '6-7' ? {
            setLogs(prev => [{
                title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
                description: 'Плесень ползет к вершине H.',
                type: 'info'
            }, ...prev]));
    },
    // Step 19: Include G-H, add H's edges
    () => {
        setVertices(prev => prev.map(v => v.id === 7 ? {
            setEdges(prev => prev.map(e => e.id === '6-7' ? {
                eap' } : e));
        setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
        setLogs(prev => [{
            title: 'Успех: Вершина H захвачена',
            description: 'В кучу добавлено H-I(12).',
            type: 'success'
        }, ...prev]));
    },
    // Step 20: Pop E-H (11) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
            setLogs(prev => [{
                title: 'Шаг 11: Куча выдает E-H (вес 11)',
                description: 'Обе вершины E и H уже в плесени.',
                type: 'warning'
            }, ...prev]));
    },
    // Step 21: Reject E-H
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
            setHeapQueue(['H-I (12)', 'F-I (13)']));
        setLogs(prev => [{
            title: 'Отказ: Игнорируем E-H',
            description: 'Выбрасываем из кучи.',

```

```

// Node 7(H) -> G-H(10)
// Node 8(I) -> H-I(12)
setEdges(prev => prev.map(e =>
  ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
setLogs(prev => [{
  title: 'Первая пятилетка: Каждая деревня выбирает
description: 'Каждая из 9 деревень одновременно
  А выбирает A-B(2), D выбирает D-E(3), а G вы
  !',
  type: 'info'
}, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Components)
() => {
  // Concurrently build chosen roads. Merge into:
  // Component 1: {A, B, C} (colored red)
  // Component 2: {D, E, F, G, H, I} (colored gold/olive)
  setVertices(prev => prev.map(v =>
    [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
  ));
  setEdges(prev => prev.map(e => {
    if (['0-1', '1-2'].includes(e.id)) {
      return { ...e, status: 'included', color: 'red' }
    }
    if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
      return { ...e, status: 'included', color: 'blue' }
    }
    return e;
  }));
  setLogs(prev => [{
    title: 'Слияние: Деревни объединились в колхозы
description: 'Все выбранные дороги построены ра
    оз {D, E, F, G, H, I}.',
    type: 'success'
  }, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between

```

```

    activeAlgo === 'kruskal' ? kruskalSteps :
    activeAlgo === 'prim' ? primSteps :
    boruvkaSteps;

```

```

const handleNextStep = () => {
  if (stepIndex < currentSteps.length) {
    currentSteps[stepIndex]();
    setStepIndex(stepIndex + 1);
  }
};

```

```

const handleReset = (algo: 'kruskal' | 'prim' | 'boruvka') => {
  setActiveAlgo(algo);
  setVertices(initialVertices);
  setEdges(initialEdges);
  setStepIndex(0);
  setHeapQueue([]);
  if (algo === 'kruskal') {
    setLogs([
      {
        title: 'Введение в раскраску (Краскал)',
        description: 'Представь, что все 9 вершин графа  
уже покрашены. Выбираем старшую (Heap) и захватываем соседей!',
        type: 'info',
      },
    ]);
  } else if (algo === 'prim') {
    setLogs([
      {
        title: 'Введение в Плесень (Прима)',
        description: 'Логика «Плесени»: выбираем старшую  
вершину (Heap) и захватываем соседей!',
        type: 'info',
      },
    ]);
  } else {
    setLogs([
      {
        title: 'Введение в Коллективизацию (Борувка)'

```

```

    { /* 3-in-1 MST Simulator Box */ }
    <div className="bg-slate-900/90 border border-slate-900">
      <div className="flex flex-col lg:flex-row justify-between">
        <div>
          <div className="flex items-center gap-2 mb-2">
            <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-weight-bold uppercase tracking-wider">
              Супер-Интерактив 3 в 1
            </span>
            <h3 className="text-2xl font-bold text-white">
              3 в 1
            </h3>
          </div>
          <p className="text-slate-400 text-sm">
            Наблюдай за работой Краскала (Раскраска),
          </p>
        </div>

        { /* Algorithm Tabs */ }
        <div className="flex flex-wrap items-center gap-2">
          <div className="flex items-center bg-slate-900/90 border border-slate-900">
            <button
              onClick={() => handleReset('kruskal')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
                +
                (activeAlgo === 'kruskal'
                  ? 'bg-indigo-600 text-white shadow-md'
                  : 'text-slate-400 hover:text-slate-300')
              }
            >
              <GitGraph className="w-3.5 h-3.5" />
              <span>Краскал (Билет 18)</span>
            </button>
            <button
              onClick={() => handleReset('prim')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
                +
                (activeAlgo === 'prim'

```

```

        ? "bg-slate-800 text-slate-500 border
        : activeAlgo === 'kruskal'
        ? "bg-indigo-600 hover:bg-indigo-500
        : activeAlgo === 'prim'
        ? "bg-emerald-600 hover:bg-emerald-500
        : "bg-rose-600 hover:bg-rose-500 text
    }
  >
    <SkipForward className="w-4 h-4" />
    <span>{stepIndex === 0 ? 'Начать' : stepI
  </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3
  { /* Graph Canvas */ }
  <div className="lg:col-span-2 bg-slate-950 ro
    ative min-h-[480px] overflow-hidden">

    { /* Legend */ }
    <div className="absolute top-4 left-4 bg-sl
      -center gap-3 text-xs">
      <div className="flex items-center gap-1.5
        <div className="w-3 h-3 rounded-full bg
      </div>
      {activeAlgo === 'kruskal' ? (
        <
          <div className="flex items-center gap
            <div className="w-3 h-3 rounded-ful
          </div>
          <div className="flex items-center gap
            <div className="w-3 h-3 rounded-ful
          </div>
          <div className="flex items-center gap
            <div className="w-3 h-3 rounded-ful
          </div>
        </>
      ) : activeAlgo === 'prim' ? (

```

```

        return (
          <g key={edge.id}>
            <line
              x1={u.x + "%"}
              y1={u.y + "%"}
              x2={v.x + "%"}
              y2={v.y + "%"}
              stroke={strokeColor}
              strokeWidth={isInspecting ? 4.5 : 1}
              strokeDasharray={isRejected ? '4,4' : ''}
              className={"transition-all duration-300"}
            />
            <text
              x={({u.x + v.x} / 2 + "%")}
              y={({u.y + v.y} / 2 + "%")}
              fill={isInspecting ? '#f59e0b' : '#94a3b8'}
              : '#94a3b8'}
              fontSize="4.5"
              fontFamily="monospace"
              fontWeight="bold"
              textAnchor="middle"
              dominantBaseline="middle"
              className="select-none filter drop-shadow"
              dy="-1.5"
            >
              {edge.weight}
            </text>
          </g>
        );
      }
    }
  </svg>

```

```

  { /* HTML Overlay for Vertices */ }
  <div className="absolute inset-0 w-full h-full" >
    {vertices.map(vertex => {
      <div
        key={vertex.id}

```



```

    ✂ Приоритетная очередь (Min-Heap):
  </p>
  <div className="flex flex-wrap gap-1.5">
    {heapQueue.length === 0 ? (
      <span className="text-xs text-slate-500">
        Очередь пуста
      </span>
    ) : (
      heapQueue.map((item, idx) => (
        <span key={idx} className={"px-2 py-1 shadow-md animate-pulse" : "bg-slate-800 text-white"}>
          {item}
        </span>
      ))
    )}
  </div>
</div>
))}

<div className="flex-1 p-4 overflow-y-auto">
  {logs.map((log, idx) => (
    <div
      key={idx}
      className={
        "p-4 rounded-xl border transition-all" +
        (log.type === 'success'
          ? 'bg-emerald-950/40 border-emerald-500'
          : log.type === 'warning'
            ? 'bg-amber-950/40 border-amber-500'
            : log.type === 'error'
              ? 'bg-rose-950/40 border-rose-500'
              : 'bg-slate-900/60 border-slate-700')
      }
    >
      <div className="flex items-center gap-2">
        {log.type === 'success' && <CheckCircle className="text-emerald-500"/>
        {log.type === 'warning' && <HelpCircle className="text-amber-500"/>
        {log.type === 'error' && <XCircle className="text-rose-500"/>
        {log.type === 'info' && <Play className="text-slate-500"/>
      </div>
      <h5 className="font-bold text-sm text-white">{log.message}</h5>
    >
  )}
</div>

```



```
        for u, v, w in edges:
            ru, rv = find(u), find(v)
            if ru != rv:
                if cheapest[ru] == -1 or cheapest[ru][2] < w:
                    cheapest[ru] = (u, v, w)
                if cheapest[rv] == -1 or cheapest[rv][2] < w:
                    cheapest[rv] = (u, v, w)
# Объединяем все колхозы по выбранным мостам
for bridge in cheapest:
    if bridge != -1 and union(bridge[0], bridge[1]):
        mst.append(bridge)`}
</pre>
</div>
</div>
})
</div>
};
```

ФАЙЛ 51/82: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
  {
    img: dijkstraImg,
    alt: 'Добрый робот Дейкстра',
    title: 'Добрый (Дейкстра)',
    desc: 'Быстрый, жадный, но работает только с',
    highlight: 'положительными',
    highlightColor: 'text-emerald-400',
```



```

    {open && (
      <div className="lg:hidden fixed inset-0 z-[90]">
        <div className="absolute inset-0 bg-slate-950">
          <div className="absolute inset-y-0 left-0 w-[100px] flex flex-direction-column justify-center items-center text-white">
            <div className="flex items-center justify-between p-2">
              <span className="font-bold text-white text-sm">Закреть</span>
              <button
                onClick={() => setOpen(false)}
                className="p-2 -mr-2 text-slate-400 hover:text-white"
                aria-label="Закреть содержание"
              >
                <X className="w-5 h-5" />
              </button>
            </div>
            <div className="flex-1 min-h-0">
              <Sidebar
                selectedChapterId={selectedChapterId}
                setSelectedChapterId={setSelectedChapterId}
                onNavigate={() => setOpen(false)}
              />
            </div>
          </div>
        </div>
      </div>
    )}
  </div>
);
};

```

ФАЙЛ 53/82: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C

```

```

<div className="flex items-center w-full lg:w-a
  >
    <button
      id="tab-guide-btn"
      onClick={() => setActiveTab('guide')}
      className={`flex-1 lg:flex-none flex items-
        uration-200 whitespace-nowrap ${
          activeTab === 'guide'
            ? 'bg-indigo-600 text-white shadow-lg s
            : 'text-slate-400 hover:text-white'
        }`}
    >
      <BookOpen className="w-4 h-4" /> Учебное по
    </button>
    <button
      id="tab-simulator-btn"
      onClick={() => setActiveTab('simulator')}
      className={`flex-1 lg:flex-none flex items-
        uration-200 whitespace-nowrap ${
          activeTab === 'simulator'
            ? 'bg-indigo-600 text-white shadow-lg s
            : 'text-slate-400 hover:text-white'
        }`}
    >
      <Cpu className="w-4 h-4" /> Тренажёры
    </button>
    <a
      id="download-pdf-btn"
      href="/export/full_code_all_pages.pdf"
      download="full_code_all_pages.pdf"
      target="_blank"
      rel="noreferrer"
      className="flex items-center justify-center
        over:bg-emerald-600/20 border border-emeral
      title="Скачать полный PDF сборник всех стра
    >
      <FileDown className="w-4 h-4" /> Скачать PD

```

```

<h4 className="text-base font-bold text-white">K5
<p className="text-xs text-slate-400">Просто смотри

<div className="grid grid-cols-1 md:grid-cols-2 gap-4" style={{
  /* K5 */
  <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
    <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
    <svg className="w-full h-full absolute inset-0" style={{
      (() => {
        const pts = [
          {id:0,x:160,y:40},
          {id:1,x:270,y:100},
          {id:2,x:230,y:230},
          {id:3,x:90,y:230},
          {id:4,x:50,y:100}
        ];
        const edges = [
          [0,1],[0,2],[0,3],[0,4],
          [1,2],[1,3],[1,4],
          [2,3],[2,4],
          [3,4]
        ];
        function findPt(id:number) { return pts.find(p => p.id === id); }
        function intersect(a:number,b:number,c:number,d:number) {
          if (a===c||a===d||b===c||b===d) return true;
          const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
          function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
          }
          return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
        }
        const crosses: number[] = [];
        for(let i=0;i<edges.length;i++) {
          for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
              if (!crosses.includes(i)) crosses.push(i);
          }
        }
      })
    }}
  </div>
</div>

```



```

        [0,3],[0,4],[0,5],
        [1,3],[1,4],[1,5],
        [2,3],[2,4],[2,5]
    ];
    function findPt(id:number) { return pts.f
    function intersect(a:number,b:number,c:nu
        if (a===c||a===d||b===c||b===d) return
        const p1=findPt(a),p2=findPt(b),p3=find
        if (!p1||!p2||!p3||!p4) return false;
        function ccw(ax:number,ay:number,bx:num
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses:number[]=[];
    for(let i=0;i<edges.length;i++) for(let j=
        if (intersect(edges[i][0],edges[i][1],e
            if (!crosses.includes(i)) crosses.push
            if (!crosses.includes(j)) crosses.push
        }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(e
        const xing=crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x
            stroke={xing?"#f43f5e":"#4f46e5"} str
        }
    const labels = ["\u04141","\u04142","\u04
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill=
            <text x={p.x} y={p.y+3} textAnchor="m
            </g>);
    }
    return <>{lines}{circles}</>;
  })()}

```

```

    рцию борща!' },
    { name: 'Кот Борис',      emoji: ' ', color: 'from-amber',
      'плиз.' },
    { name: 'Бабушка Люба',   emoji: ' ', color: 'from-rose',
      ' ' },
    { name: 'Инопланетянин',  emoji: ' ', color: 'from-emerald',
      'пливо для нашего НЛО!' },
    { name: 'Бизнесмен',      emoji: ' ', color: 'from-slate',
      'тартапы.' },
  ];

```

```
let counter = 3;
```

```

export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: ' ', color: 'from-amber',
      'голода после матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён',   emoji: ' ', color: 'from-rose',
      'ишу ИИ за порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис',     emoji: ' ', color: 'from-slate',
      'ез сметаны, плиз.', animState: 'idle' },
  ]));

```

```

const [servedHistory, setServedHistory] = useState<string[]>([]);
const [isServing, setIsServing]         = useState(false);
const [shakeEmpty, setShake]             = useState(false);
const [log, setLog]                      = useState<string[]>([]);

```

```
const addLog = (msg: string) => setLog(prev => [...prev, msg]);
```

```
// ENQUEUE: Встать в конец очереди (хвост / Tail)
```

```

const enqueue = useCallback(() => {
  if (isServing) return;
  if (queue.length >= 6) {
    addLog('⚠ Хвост очереди уже на улице, повару тяжело!');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
  setQueue(prev => [...prev, queue[queue.length - 1]]);
}, [queue, isServing]);

```

```

// Step 2: Guest leaves with happy face, queue sh
setQueue(prev => prev.map((c, idx) => idx === 0 ?

setServedHistory(prev => [`${headGuest.emoji} ${h

setTimeout(() => {
  setQueue(prev => prev.slice(1));
  setIsServing(false);
  addLog(`  ${headGuest.name} поел и ушел сытым.
}, 400);
}, 900);
}, [queue, isServing]));

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: ' ', col
      т голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: ' ', col
      апишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: ' ', col
      без сметаны, плиз.', animState: 'in' },
  ]));
  setServedHistory([]);
  setIsServing(false);
  setLog([' Столовая сброшена. Снова голодная толпа!
  setTimeout(() => {
    setQueue(prev => prev.map(c => ({ ...c, animState
  }, 500);
};

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-indigo-950/40 border border-in
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-indigo-400 tex

```

```

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12">

  {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
  <div className={`md:col-span-9 bg-slate-900 rounded-3xl
    -[300px] overflow-hidden ${shakeEmpty ? 'animate-pulse' : ''}}
    <div className="absolute top-4 left-4 text-[1.2em] font-mono">
      Очередь голодных гостей (Буфер FIFO / BFS)
    </div>

    <div className="flex-1 flex flex-col justify-between">
      <div className="flex items-end justify-between">

        {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
        <div className="flex flex-col items-center">
          <div className="relative">
            <div className="absolute -top-10 left-1/2 -translate-x-1/2">
              <div className="w-1.5 h-6 bg-slate-400 rounded">
            <span className="text-3xl block"> </span>
            <span className="text-[10px] font-mono font-bold">
              ase font-mono tracking-wider">
                ПОВАР
              </span>
            </div>
          </div>

          {/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
          <div className="flex flex-col items-center">
            <span className="text-2xl animate-pulse"> </span>
            <span className="text-[8px] font-mono t
          </div>

```

```

        ${customer.color}
        ${isHead ? 'ring-4 ring-amber-500' : ''}
      `}>
      <span className="text-3xl">{customer.name}</span>
    </div>

    {/* Индексы HEAD/TAIL */}
    {(isHead || isTail) && (
      <span className={`
        absolute -bottom-2 text-[10px]
        ${isHead ? 'bg-amber-500' : 'bg-slate-900'}
      `}>
        {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}
      </span>
    )}
  </div>
</div>
);
})
})
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-amber-500 to-slate-900">
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
  <div className="absolute top-4 left-4 text-[10px] font-mono">
    Сытые гости (Архив FIFO)
  </div>

  <div className="absolute top-4 right-4 flex items-center justify-end gap-2">
    <div className="order-emerald-800/80 text-[9px] font-mono">
      <Sparkles className="w-3.5 h-3.5 animate-pulse">
        <span>СЫТЫ</span>
      </div>
    </div>
  </div>
</div>

```

```
        <span>{entry}</span>
      </div>
    )))
  </div>
</div>
);
};
```

ФАЙЛ 56/82: src/components/SalmonAutomatonWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
  depth: number;
  x?: number;
  y?: number;
}
```

```
export const SalmonAutomatonWidget: React.FC = () => {
  const [words, setWords] = useState<string[]>(['CAR',
  const [newWord, setNewWord] = useState('');
  const [nodes, setNodes] = useState<{ [key: number]: T
  const [simulationStep, setSimulationStep] = useState<
  const [bfsQueue, setBfsQueue] = useState<number[]>([
  const [currentNode, setCurrentNode] = useState<number
```

```

        id: count,
        char: c,
        parent: curr,
        children: {},
        fail: 0,
        term_link: 0,
        is_terminal: false,
        words: [],
        depth: newNodes[curr].depth + 1
    };
}
curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
    newNodes[curr].words.push(word);
    newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
        newNodes[u].x = 40 + currentX * paddingX;
        currentX++;
    } else {
        let leftX = -1;
        let rightX = -1;
        childKeys.forEach(v => {
            layoutDFS(v, depth + 1);
            if (leftX === -1) leftX = newNodes[v].x!;
            rightX = newNodes[v].x!;
        });
    }
};

```

```

        return;
    }
    const updated = words.filter((w) => w !== wordToRemove);
    setWords(updated);
    buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
    const rootChildren = Object.values(nodes[0].children);
    const queue = [...rootChildren];
    const updatedNodes = { ...nodes };
    rootChildren.forEach((childId) => {
        updatedNodes[childId].fail = 0;
    });

    setNodes(updatedNodes);
    setBfsQueue(queue);
    setSimulationStep(1);
    setCurrentNode(null);
    setSpeechText(
        'Начинаем обход в ширину (BFS)! Все вершины на главном уровне  

        уже суффикса просто нет. Жми «Следующий шаг»!'
    );
};

// Один шаг BFS
const stepBFS = () => {
    if (bfsQueue.length === 0) {
        setSimulationStep(2);
        setCurrentNode(null);
        setSpeechText(
            'Ура! Очередь пуста! Мы успешно построили полную таблицу переходов!  

            Теперь можно полюбоваться таблицей переходов!'
        );
        return;
    }

```



```

<div className="bg-slate-800/90 rounded-2xl p-6 border-2 border-slate-900" style={{width: 1000px, height: 100px}}
  {/* Шапка виджета */}
  <div className="flex flex-wrap items-center justify-between" style={{width: 1000px, height: 100px}}
    <div className="flex items-center space-x-3" style={{width: 1000px, height: 100px}}
      <span className="text-3xl" style={{width: 1000px, height: 100px}}> </span>
      <div>
        <h4 className="text-2xl font-bold text-white" style={{width: 1000px, height: 100px}}>
          Интерактивный конструктор: Говорящий Эхо-
        </h4>
        <p className="text-sm text-slate-400" style={{width: 1000px, height: 100px}}>
          Построение дерева и расчет суффиксных ссы
        </p>
      </div>
    </div>
  <div className="flex items-center gap-2" style={{width: 1000px, height: 100px}}
    <button
      onClick={() => buildInitialTrie(words)}
      className="flex items-center gap-1 bg-slate-700 text-white px-4 py-2 rounded-md transition-colors"
      title="Сбросить автомат"
    >
      <RotateCcw className="w-4 h-4" /> Сбросить
    </button>
  </div>
</div>

```

```

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 gap-4" style={{width: 1000px, height: 100px}}
  {/* Аватар Лосося (Сеня) с анимацией моргания и
  <div className="flex flex-col items-center justify-center" style={{width: 1000px, height: 100px}}
    <div className="w-40 h-40 relative flex items-center justify-center" style={{width: 1000px, height: 100px}}
      full border border-blue-500/30 shadow-inner
      {/* SVG Лосося */}
      <svg
        viewBox="0 0 200 200"
        className={`w-36 h-36 transition-transform duration-200ms ${
          isTalking ? 'scale-105 drop-shadow-[0_0_10px_#007bff]' : ''
        }`}
      >

```

```

        { /* Улыбка / жабры */ }
        <path d="M 130 80 Q 125 100 130 120" stroke="black" />
        <path d="M 120 85 Q 115 100 120 115" stroke="black" />
    </svg>

    { /* Декоративные пузыри */ }
    <div className="absolute -top-1 right-4 text-blue-400 font-size-20" >
    <div className="absolute top-8 right-1 text-blue-400 font-size-20" >
    </div>
    <span className="mt-3 bg-rose-600/30 border-b-rose-600/30 border-b-1px" >
        Эхо-Лосось Сеня
    </span>
</div>

{ /* Пузырь с текстом (Баббл) */ }
<div className="md:col-span-2 bg-slate-800 border-1px border-slate-700
  transition min-h-[140px]" >
    { /* Треугольник баббла */ }
    <div className="absolute -left-3 top-1/2 -transform rotate(-90deg)
      bg-slate-800 border-b-[12px] border-b-transparent border-l-[12px]
      border-l-transparent border-r-[12px] border-r-transparent" >

    <div className="flex items-center space-x-2 text-white" >
        <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
        <span>Прямое вещание из аквариума:</span>
    </div>

    <p className="text-slate-200 text-base leading-relaxed" >
        {speechText}
    </p>

    <div className="mt-4 pt-3 border-t border-slate-700" >
        <div className="text-xs text-slate-400 flex justify-between" >
            <Info className="w-3.5 h-3.5 text-blue-400" />
            Статус: {simulationStep === 0 ? 'Борется' : 'Побежден!'}
        </div>
    </div>

```

</h5>

```

{(() => {
  let maxX = 0;
  let maxY = 0;
  Object.values(nodes).forEach(n => {
    if (n.x && n.x > maxX) maxX = n.x;
    if (n.y && n.y > maxY) maxY = n.y;
  });
  const svgWidth = Math.max(maxX + 80, 400);
  const svgHeight = Math.max(maxY + 60, 200);

  return (
    <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
      <defs>
        <marker id="arrowhead" markerWidth="6"
          <polygon points="0 0, 6 3, 0 6" fill=
        </marker>
        <marker id="fail-arrow" markerWidth="6"
          <polygon points="0 0, 6 3, 0 6" fill=
        </marker>
      </defs>

    {/* Draw Edges */}
    {Object.values(nodes).map(node => {
      if (node.id === 0) return null;
      const parent = nodes[node.parent];
      if (!parent.x || !parent.y || !node.x ||

      const isCurrent = currentNode === node..

      return (
        <g key={`edge-${node.id}`}>
          <line
            x1={parent.x}
            y1={parent.y + 16}
            x2={node.x}
            y2={node.y - 16}

```

```

        strokeDasharray="4 4"
        markerEnd="url(#fail-arrow)"
    />
  );
  }}}

  { /* Draw Nodes */
  Object.values(nodes).map(node => {
    if (!node.x || !node.y) return null;
    const isRoot = node.id === 0;
    const isCurrent = currentNode === node;
    const isInQueue = bfsQueue.includes(node);

    let fill = '#1e293b';
    let stroke = '#475569';

    if (isCurrent) {
      fill = '#2563eb';
      stroke = '#60a5fa';
    } else if (isInQueue) {
      fill = '#b45309';
      stroke = '#fbbf24';
    } else if (node.is_terminal) {
      fill = '#059669';
      stroke = '#34d399';
    }

    return (
      <g key={`node-${node.id}`}>
        <circle
          cx={node.x}
          cy={node.y}
          r="16"
          fill={fill}
          stroke={stroke}
          strokeWidth="2"
        />
        <text

```

```

        className="bg-slate-800 border border-
er gap-1 group"
      >
        {w}
        <button
          onClick={() => removeWord(w)}
          className="text-slate-500 hover:tex
          title="Удалить"
        >
          ×
        </button>
      </span>
    )))
  </div>
</div>

```

```

<form onSubmit={handleAddWord} className="fle
  <input
    type="text"
    value={newWord}
    onChange={(e) => setNewWord(e.target.valu
    placeholder="НОВОЕ СЛОВО..."
    maxLength={8}
    className="bg-slate-800 border border-sla
    outline-none focus:border-blue-500"
  />
  <button
    type="submit"
    className="bg-slate-700 hover:bg-slate-60
    ition-colors"
  >
    <Plus className="w-3.5 h-3.5" /> Добавить
  </button>
</form>
</div>

```

```

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p

```

```

<td className="py-2.5 px-3 flex items-center"
  <span
    className={`w-2 h-2 rounded-full`}
    node.id === 0
      ? 'bg-purple-500'
      : isCurrent
      ? 'bg-blue-400 animate-pulse'
      : isInQueue
      ? 'bg-amber-400'
      : 'bg-slate-600'
    >`>
  ></span>
  <span>#{node.id}</span>
  {node.id === 0 && <span className="font-size-0.8"
</td>
<td className="py-2.5 px-3">
  <span className="bg-slate-800 border-1px solid black"
    {node.char}
  </span>
</td>
<td className="py-2.5 px-3 text-slate-400"
  {node.id === 0 ? '-' : `#${node.id}`}
</td>
<td className="py-2.5 px-3"
  {node.id === 0 ? {
    <span className="text-slate-500"
  } : {
    <span className="bg-indigo-950 text-white"
      #{node.fail}
    </span>
  }}
</td>
<td className="py-2.5 px-3"
  {node.id === 0 || node.term_line === 0
    <span className="text-slate-500"
  } : {
    <span className="bg-rose-950 text-white"
  >
0 transition-colors cursor-help" title={`Click to toggle the

```

Универсальное пособие • полный исходный код

```
-----  
import React, { useState, useEffect, useCallback, useMe  
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRi
```

```
// ===== TYPES =====
```

```
interface Step {  
  id: number;  
  desc: string;  
  codeLine: number;  
  treeState: Record<number, number>;           // node ind  
  highlightNodes: number[];                    // current  
  mergeChildren?: [number, number];           // pair be  
  arrayHighlight?: [number, number];           // [L, R]  
  result?: number;                             // partial  
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
function buildTreeFull(arr: number[]): Record<number, number> {  
  const n = arr.length;  
  const tree: Record<number, number> = {};  
  function go(v: number, l: number, r: number) {  
    if (l === r) { tree[v] = arr[l]; return; }  
    const m = (l + r) >> 1;  
    go(2 * v, l, m);  
    go(2 * v + 1, m + 1, r);  
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);  
  }  
  go(1, 0, n - 1);  
  return tree;  
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {  
  const n = arr.length;  
  const steps: Step[] = [];  
  let id = 0;  
  const tree: Record<number, number> = {};
```

```

}

// 2. Top-Down Query steps
function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
  const n = arr.length;
  const tree = buildTreeFull(arr);
  const steps: Step[] = [];
  let id = 0;

  function go(v: number, l: number, r: number, ql: number, qr: number) {
    if (ql > r || qr < l) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
        codeLine: 2, treeState: tree, highlightNodes: [v] });
      return 0;
    }
    if (ql <= l && r <= qr) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
        codeLine: 4, treeState: tree, highlightNodes: [v] });
      return tree[v];
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
      codeLine: 6, treeState: tree, highlightNodes: [v] });
    const left = go(2 * v, l, m, ql, qr);
    const right = go(2 * v + 1, m + 1, r, ql, qr);
    const res = left + right;
    steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
      codeLine: 8, treeState: tree, highlightNodes: [v],
      rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [l, r] });
    return res;
  }
  const ans = go(1, 0, n - 1, qL, qR);
  steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]`,
    codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
  return steps;
}

// 3. Bottom-Up Query steps (iterative on implicit tree)
function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {

```



```

    steps.push({ id: id++, desc: ` Запрос sum([${qL}..${qR}]`
    highlightNodes: [1], arrayHighlight: [qL, qR], result: 0
    return steps;
}

```

// ===== TREE RENDERING HELPER =====

```

interface VNode { v: number; l: number; r: number; val: number; }

```

```

function buildVisualTree(treeState: Record<number, number>) {
  if (l === r) return { v, l, r, val: treeState[v] ?? null };
  const m = (l + r) >> 1;
  return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
    treeState, v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
  "build(v, l, r) {",
  "  if (l === r) { tree[v] = A[l]; return; }",
  "  // -----",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  build(2*v, l, mid);",
  "  build(2*v+1, mid+1, r);",
  "  tree[v] = tree[2*v] + tree[2*v+1];",
  "}"
];

```

```

const QUERY_TD_CODE = [
  "query(v, l, r, ql, qr) {",
  "  if (ql > r || qr < l) return 0;",
  "  // -----",
  "  if (ql <= l && r <= qr) return tree[v];",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  // спускаемся в обоих детей",
  "  return query(left) + query(right);",
  "}"
];

```

```
-----
  return { step, idx, setIdx, playing, setPlaying, speed
}
```

```
// ===== SIMULATION PANEL =====
```

```
function SimPanel({ title, icon, steps, code, color, arr
  | 'emerald' | 'amber'; arr: number[] }) {
  const p = Player({ steps, color });
  if (!p) return null;
  const { step, idx, setIdx, playing, setPlaying, speed
  const n = arr.length;
```

```
  const vTree = useMemo(() => buildVisualTree(step.tree
```

```
  const renderNode = useCallback((nd: VNode): React.Rea
    const isHigh = step.highlightNodes.includes(nd.v);
    const isChild = step.mergeChildren?.includes(nd.v);
    const has = nd.val !== null;
```

```
    let cls = 'bg-slate-800 border-slate-700 text-slate
    if (isHigh) cls = `${clr.ring} text-white ring-2 sc
    else if (isChild) cls = `${clr.childRing} text-ambe
    else if (has) cls = `${clr.filled} text-slate-200`;
```

```
    return (
      <div key={nd.v} className="flex flex-col items-ce
        <div className={`flex flex-col items-center jus
          ${cls}`}>
          <span className="text-[7px] opacity-60 font-m
          <span className="text-[9px] font-bold">[{nd.l
          <span className="text-xs font-extrabold font-i
        </div>
        {(nd.left || nd.right) && (
          <div className="flex flex-col items-center mt
            <div className="w-px h-2 bg-slate-700" />
            <div className="flex justify-around w-full
              {nd.left && renderNode(nd.left)}
              {nd.right && renderNode(nd.right)}
            </div>
```

```

x">
{code.map((line, i) => {
  const ln = i + 1;
  const active = step.codeLine === ln;
  return (
    <div key={ln} className={`px-2 py-0.5 rounded-
2 border-indigo-500 text-indigo-200` : color
' : 'bg-amber-600/20 border-l-2 border-amber-
    <span className="inline-block w-4 text-slate-
    </div>
  );
})}
</pre>

```

```

{/* Description + result */}
<div className="px-3 py-2.5 bg-slate-900/60 border-t-2 border-l-2 border-r-2 border-pink]">
  <span className={`mt-0.5 inline-block w-2 h-2 rounded-sm bg-${step.codeLine === ln ? 'emerald' : 'pink'}-500`} />
  <span className="leading-relaxed">{step.desc}</span>
  {step.result !== undefined && (
    <span className={`ml-auto shrink-0 font-mono text-${step.codeLine === ln ? 'emerald' : 'pink'}-300`} />
    {step.result}
  )}
</div>

```

```

{/* Controls */}
<div className="px-3 py-2.5 bg-slate-950 border-t-2 border-l-2 border-r-2 border-pink]">
  <div className="flex items-center bg-slate-900/60 border-t-2 border-l-2 border-r-2 border-pink]">
    <button onClick={() => { setPlaying(false); setStep(0); }} className="p-2 rounded-sm text-white transition-all hover:bg-slate-800">C6poc</button>
    <button onClick={() => { setPlaying(false); setStep(0); }} className="p-2 rounded-sm text-white transition-all hover:bg-slate-800">C6poc</button>
    <button onClick={() => setPlaying(!playing)} className="p-2 rounded-sm text-white transition-all ${playing ? 'bg-rose-500' : 'bg-slate-800'}">{playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}</button>
  
```

```

    const parsed = customInput.split(',').map(s => pars
    if (parsed.length >= 2 && parsed.length <= 16) {
      setArr(parsed);
    }
  };

  const randomize = () => {
    const size = arr.length;
    const a = Array.from({ length: size }, () => Math.f
    setArr(a);
    setCustomInput(a.join(', '));
  };

  const buildSteps = useMemo(() => genBuildSteps(arr),
  const queryTDSteps = useMemo(() => genQueryTopDownSte
  const queryBUSteps = useMemo(() => genQueryBottomUpSt

  const totalSum = arr.reduce((a, b) => a + b, 0);

  return (
    <div className="bg-slate-900 rounded-2xl border bor
      /* ---- TOP BAR: array editor + query range ----
      <div className="mb-6 space-y-4">
        <div className="flex flex-col lg:flex-row gap-4
          /* Array input */
          <form onSubmit={handleApply} className="flex-
            <div className="flex items-center justify-b
              <label className="text-[10px] font-bold up
                label>
                  <button type="button" onClick={randomize}
                "><RefreshCw className="w-3.5 h-3.5" /></bu
              </div>
            <div className="flex gap-2">
              <input
                value={customInput}
                onChange={e => setCustomInput(e.target.
                className="flex-1 bg-slate-900 border b
              one focus:border-indigo-500"

```

```

        onChange={e => setQR(Math.min(Number(
          className="w-14 bg-slate-900 border b
        focus:border-indigo-500"
      />
    </div>
  </div>
  <div className="text-[10px] text-slate-500 p
    Запрос: sum(A[{qL}..{qR}]) = {arr.slice(q
  </div>
</div>
</div>
</div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1
  <button onClick={() => setView('build')} classN
    -colors ${view === 'build' ? 'border-indigo-5
    -slate-200'}`}>
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} clas
    on-colors ${view === 'queries' ? 'border-emerg
    er:text-slate-200'}`}>
    <Search className="w-3.5 h-3.5" /> Запрос: св
  </button>
  <button onClick={() => setView('theory')} class
    n-colors ${view === 'theory' ? 'border-blue-5
    te-200'}`}>
    <Info className="w-3.5 h-3.5" /> Почему / Сра
  </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
  <SimPanel
    key={`build-${arr.join('-')}`}
    title="Построение дерева (рекурсия)"
    icon=" "

```

```

к получает <code className="bg-slate-900 px-
-slate-900 px-1 rounded font-mono text-emera
евая часть забирает на 1 больше (округление
</p>
<div className="bg-slate-900 p-4 rounded-lg
  <div className="text-slate-400">Корень: L
  <div className="text-slate-300">mid = |(0
  <div className="text-indigo-300">→ Левый:
  <div className="text-emerald-300">→ Правы
h - 1) >> 1)} эл.)</div>
  <div className="text-slate-500 mt-2">Всег
} внутренних.</div>
</div>
</div>

{/* Comparison cards */}
<div className="grid grid-cols-1 xl:grid-cols
  <div className="bg-slate-950 p-5 rounded-xl
    <div className="absolute -top-3 left-4 bg
e border border-emerald-500 shadow-md">
      Запрос Сверху-вниз (Рекурсия)
    </div>
    <p className="text-xs text-slate-300 mb-3
      Начинаем с корня. Если отрезок узла полн
наче спускаемся в обоих детей.
    </p>
    <div className="space-y-1.5 text-xs">
      <div className="flex justify-between bo
span className="text-rose-400 font-mono">0(
        <div className="flex justify-between bo
><span className="text-rose-400 font-mono">
          <div className="flex justify-between"><
d-400 font-bold">✓ легко</span></div>
        </div>
      </div>
    </div>
  <div className="bg-slate-950 p-5 rounded-xl
    <div className="absolute -top-3 left-4 bg
rder border border-amber-500 shadow-md">

```



```

        }` }
      >
      {hasViz && {
        <span
          className="w-1.5 h-1.5 rounded-
          title="Есть интерактивная визуа
        />
      }}
      <span className="font-semibold text
      <ChevronRight
        className={`w-4 h-4 shrink-0 mt-0
          isSelected ? "text-indigo-400"
        }` }
      />
    </button>
  );
  }}}
</div>
</div>
  )})
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-
  ] text-slate-300 space-y-1.5 hidden lg:block">
  <div className="font-bold text-indigo-400 flex
    <Sparkles className="w-3.5 h-3.5" /> Как поль
  </div>
  <p className="leading-relaxed">
    Подчёркнутые термины в тексте раскрываются по
    симулятора.
  </p>
  <p className="text-slate-400 italic">Стрелки ←
  </div>
</aside>
  );
};

```



```

const [queue, setQueue] = useState<Person[]>([
  { id: '1', name: 'Студент Вася', icon: ' ' },
  { id: '2', name: 'Голодный кодер', icon: ' ' },
  { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный"',
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный"',
  { id: '3', text: 'Кандидат: "Привет, я испанский ин
]);
const [newCustomText, setNewCustomText] = useState('')
const [newCustomProb, setNewCustomProb] = useState(0.)
const [heapMessage, setHeapMessage] = useState<string>('')

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: ' ' },
    { name: 'Кастрюля от супа', icon: ' ' },
    { name: 'Чашка от кофе', icon: '☑' },
    { name: 'Миска с остатками салата', icon: ' ' },
    { name: 'Тарелка от тако', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setStack(prev => [...prev, newPlate]);
  setPopMessage(`Положили поверх стопки: ${newPlate.name}`);
};

const popPlate = () => {
  if (stack.length === 0) {

```

```

        return;
    }
    const firstPerson = queue[0];
    setQueue(prev => prev.slice(1));
    setDequeueMessage(`Выдан суп первому в очереди (F
});

const resetQueue = () => {
    setQueue([
        { id: '1', name: 'Студент Вася', icon: ' ' },
        { id: '2', name: 'Голодный кодер', icon: ' ' },
        { id: '3', name: 'Кот Борис', icon: ' ' },
    ]);
    setDequeueMessage("Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
    e.preventDefault();
    if (!newCustomText.trim()) return;
    const item: Hypothesis = {
        id: Date.now().toString(),
        text: `Кандидат: "${newCustomText}"`,
        probability: Number(newCustomProb)
    };
    setHeap(prev => [...prev, item]);
    setNewCustomText('');
    setHeapMessage(`Добавлена гипотеза с вероятностью
});

const popHeap = () => {
    if (heap.length === 0) {
        setHeapMessage("⚠ Куча пуста! Нет кандидатов для
        return;
    }
    // Find highest probability
    let maxIdx = 0;
    for (let i = 1; i < heap.length; i++) {

```

```

        activeTab === 'stack'
          ? 'bg-blue-600/20 border-blue-500 text-blue-50'
          : 'bg-slate-800/60 border-slate-700/60 text-slate-50'
      }` }
    >
    <div className="flex items-center gap-3">
      <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'bg-blue-600/20 border-blue-500 text-blue-50' : 'bg-slate-800/60 border-slate-700/60 text-slate-50'} `}>
        <div className="text-left">
          <div className="font-bold text-sm text-white">
            <div className="text-xs opacity-80">LIFO
          </div>
        </div>
      </div>
      <span className="text-xs bg-blue-500/20 text-white">
        DFS
      </span>
    </button>

    <button
      onClick={() => setActiveTab('queue')}
      className={`flex items-center justify-between gap-3 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-50' : 'bg-slate-800/60 border-slate-700/60 text-slate-50'} `}
    >
      <div className="flex items-center gap-3">
        <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-50' : 'bg-slate-800/60 border-slate-700/60 text-slate-50'} `}>
          <div className="text-left">
            <div className="font-bold text-sm text-white">
              <div className="text-xs opacity-80">FIFO
            </div>
          </div>
        </div>
        <span className="text-xs bg-indigo-500/20 text-white">
          BFS
        </span>
      </button>

    <button

```

```

rounded-xl text-sm font-bold shadow-lg tra
>
  <Plus className="w-4 h-4" /> Поставить
</button>
<button
  onClick={popPlate}
  className="flex-1 md:flex-none flex item
border-blue-500/40 px-4 py-2.5 rounded-xl
>
  Помыть верхнюю
</button>
<button
  onClick={resetStack}
  title="Сбросить"
  className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
>
  <RotateCcw className="w-5 h-5" />
</button>
</div>
</div>

{popMessage && {
  <div className="bg-blue-950/60 border borde
imate-fade-in">
    <span className="inline-block w-2 h-2 roun
    {popMessage}
  </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
>
  {stack.length === 0 ? {
    <div className="text-center py-12 text-sl
    <div className="text-5xl mb-3"> </div>
    <p className="text-base font-bold">Стор
    <p className="text-xs mt-1">Добавьте гр

```

```

        <div className="mt-6 text-xs text-slate-500" style={{
          position: 'relative',
          height: 100px,
          border: 1px solid #ccc,
          padding: 10px,
          background: 'linear-gradient(to bottom, #f0f0f0 49%, #fff 49%, #fff 51%, #f0f0f0 51%)',
          backgroundSize: '100% 100%',
          backgroundRepeat: 'repeat-y',
        }}>
          ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
        </div>
      </div>
    </div>
  )}

```

```

{ /* QUEUE TAB */ }
{activeTab === 'queue' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция очереди за супом</span>
          <span className="text-xs font-mono bg-i
            /span>
          </h3>
          <p className="text-slate-400 text-xs mt-1" style={{
            position: 'relative',
            height: 100px,
            border: 1px solid #ccc,
            padding: 10px,
            background: 'linear-gradient(to bottom, #f0f0f0 49%, #fff 49%, #fff 51%, #f0f0f0 51%)',
            backgroundSize: '100% 100%',
            backgroundRepeat: 'repeat-y',
          }}>
            Кто первым пришел, тот первым получил суп
          </p>
        </div>
        <div className="flex flex-wrap items-center gap-2">
          <button
            onClick={addPerson}
            className="flex-1 md:flex-none flex item
              -2.5 rounded-xl text-sm font-bold shadow-lg" style={{
                position: 'relative',
                height: 100px,
                border: 1px solid #ccc,
                padding: 10px,
                background: 'linear-gradient(to bottom, #f0f0f0 49%, #fff 49%, #fff 51%, #f0f0f0 51%)',
                backgroundSize: '100% 100%',
                backgroundRepeat: 'repeat-y',
              }}>
            <Plus className="w-4 h-4" /> Встать в очередь
          </button>
          <button
            onClick={servePerson}
            className="flex-1 md:flex-none flex item
              er border-indigo-500/40 px-4 py-2.5 rounded" style={{
                position: 'relative',
                height: 100px,
                border: 1px solid #ccc,
                padding: 10px,
                background: 'linear-gradient(to bottom, #f0f0f0 49%, #fff 49%, #fff 51%, #f0f0f0 51%)',
                backgroundSize: '100% 100%',
                backgroundRepeat: 'repeat-y',
              }}>
            Налить суп первому
          </button>
          <button
            onClick={clearQueue}
            className="flex-1 md:flex-none flex item
              er border-indigo-500/40 px-4 py-2.5 rounded" style={{
                position: 'relative',
                height: 100px,
                border: 1px solid #ccc,
                padding: 10px,
                background: 'linear-gradient(to bottom, #f0f0f0 49%, #fff 49%, #fff 51%, #f0f0f0 51%)',
                backgroundSize: '100% 100%',
                backgroundRepeat: 'repeat-y',
              }}>
            Очистить очередь
          </button>
        </div>
      </div>
    </div>
  }
}

```

```

    {queue.map((person, index) => {
      const isFirst = index === 0;
      return (
        <div
          key={person.id}
          className={`flex flex-col items-center
            isFirst
              ? 'bg-gradient-to-b from-indigo-500 to-white
relative'
              : 'bg-slate-900/90 border-slate-800'}
          >
          {isFirst && (
            <span className="absolute -top-10 left-0
se tracking-wider shadow">
              Первый (Head)
            </span>
          )}
          <span className="text-4xl mb-2">{
            <span className={`font-bold text-2xl
              {person.name}
            </span>
            <span className={`text-[10px] font-mono
              isFirst ? 'bg-indigo-500/30 text-white' : ''
            }`}>
              {isFirst ? 'Получает сун' : `В
            </span>
          </div>
        );
      )}
    )}

```

```

    <div className="flex flex-col items-center
-600 min-w-[120px] h-[120px]">
      <Plus className="w-6 h-6 mb-1" />
      <span className="text-xs font-mono up">
    </div>
  </div>

```

```

        </button>
      </div>
    </div>

    {/* Add Form */}
    <form onSubmit={addHypothesis} className="bg-
      s-12 gap-4 items-end">
      <div className="md:col-span-6">
        <label className="block text-xs font-bold">
          <input
            type="text"
            value={newCustomText}
            onChange={e => setNewCustomText(e.target.value)}
            placeholder="например: "Привет, я лучший!"
            className="w-full bg-slate-900 border border-
              rder-emerald-500 transition-colors"
          />
        </div>
        <div className="md:col-span-3">
          <label className="block text-xs font-bold">
            Вероятность: <span className="text-emerald-500">
              {newCustomProb}
            </span>
          </label>
          <input
            type="range"
            min="0.01"
            max="0.99"
            step="0.01"
            value={newCustomProb}
            onChange={e => setNewCustomProb(Number(e.target.value))}
            className="w-full accent-emerald-500 cursor-pointer"
          />
        </div>
        <div className="md:col-span-3">
          <button
            type="submit"
            disabled={!newCustomText.trim()}
            className="w-full flex items-center justify-center gap-2
              erald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
          >
            Добавить гипотезу
          </button>
        </div>
      </form>
    </div>
  </div>
</div>

```

```

        isTop
        ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]'
        : 'bg-slate-900/80 border-slate-900'
      }` }
    }
  }
  >
  <div className="flex items-center justify-between" >
    <span className={`flex items-center justify-center gap-2`} >
      isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900 text-white'
    </span>
    <div>
      <div className={`font-bold text-white`} >
        {item.text}
      </div>
      {isTop && (
        <span className="text-[10px] font-mono text-white" >
          Вершина Кучи (Root) • Будущее
        </span>
      )}
    </div>
  </div>

  <div className="flex items-center justify-between" >
    { /* Custom progress bar */ }
    <div className="hidden sm:block" >
      <div
        className={`h-full rounded-full`}
        style={{ width: `${percent}%` }}
      ></div>
    </div>
    <span className={`font-mono font-mono text-white`} >
      isTop ? 'text-emerald-400 text-white' : 'text-white'
    </span>
  </div>

```



```
* теперь это витрина: видно всё, что вообще можно поты
*/
export const SimulatorHub: React.FC<Props> = ({ onOpen,
  const [query, setQuery] = useState("");

  const items = useMemo(() => {
    const titleById = new Map(chapters.map((c) => [c.id, c.title]));
    const all = Object.entries(VIZ_REGISTRY).map(([id, entry], index) => ({
      id,
      entry,
      chapterTitle: titleById.get(id),
    }));
    const q = query.trim().toLowerCase();
    if (!q) return all;
    return all.filter(
      (i) =>
        i.entry.title.toLowerCase().includes(q) ||
        (i.entry.hint ?? "").toLowerCase().includes(q) ||
        (i.chapterTitle ?? "").toLowerCase().includes(q)
    );
  }, [query]);

  return (
    <div>
      <div className="mb-6">
        <h2 className="text-xl sm:text-2xl font-extrabold">Витрина</h2>
        <p className="text-sm text-slate-400 leading-relaxed">
          Все интерактивные демонстрации в одном месте.
          Здесь их можно открыть напрямую.
        </p>
      </div>

      <div className="relative mb-6 max-w-md">
        <Search className="w-4 h-4 text-slate-500 absolute left-0 top-0">
          <input
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Поиск тренажёра..."
          />
        </Search>
      </div>
    </div>
  );
}
```

```
        className="flex items-center gap-1.5
0 hover:text-white transition-colors min-w-1
    >
        <BookOpen className="w-3.5 h-3.5 shrink-0
        <span className="truncate max-w-[9rem]
    </button>
    })
  </div>
</div>
  )})
</div>
</div>
);
};
```

ФАЙЛ 61/82: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useCallback, useMemo, useState } from "
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, U
```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерев
```

```
 * поднять отмеченный узел в корень. Клик по узлу = оди
```

```
 * Порядок поворотов выбираешь сам; правильный ли он по
```

```
 * только после того, как узел окажется наверху (или по
```

```
 */
```

```
export interface TNode {
```

```
  key: number;
```

```
  left: TNode | null;
```

```
  right: TNode | null;
```

```
}
```

```

    hintCase: "Zig-Zag, затем Zig-Zig",
    target: 5,
    build: () => {
        const n5 = leaf(5);
        const n6: TNode = { key: 6, left: n5, right: leaf(7) };
        const n4: TNode = { key: 4, left: leaf(3), right: n6 };
        return { key: 8, left: n4, right: leaf(9) };
    },
},
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] => {
    const path: TNode[] = [];
    let cur = root;
    while (cur) {
        path.push(cur);
        if (key === cur.key) return path;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
    const path = findPath(root, key);
    if (path.length < 2) return root;

    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const gg = path.length >= 3 ? path[path.length - 3] : null;

    if (p.left === x) {
        p.left = x.right;
        x.right = p;
    } else {
        p.right = x.left;
        x.left = p;
    }

    if (gg) {
        gg.left = x;
        x.left = p;
    }

    return root;
}

```

```

const walk = (n: TNode | null, depth: number) => {
  if (!n) return;
  walk(n.left, depth + 1);
  nodes.push({ key: n.key, x: order++, y: depth, depth });
  if (n.left) edges.push([n.key, n.left.key]);
  if (n.right) edges.push([n.key, n.right.key]);
  walk(n.right, depth + 1);
};
walk(root, 0);

const cols = Math.max(1, order);
const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
const colW = 46;
const rowH = 62;
return {
  nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
  edges,
  width: cols * colW,
  height: rows * rowH + 20,
};
}

/* ----- КОМПОНЕНТ ----- */

export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.root);
  const [history, setHistory] = useState<TNode[]>([]);
  const [moves, setMoves] = useState<{ node: number; count: number }>({ node: 0, count: 0 });
  const [showAnswer, setShowAnswer] = useState(false);

  const target = preset.target;
  const solved = tree.key === target;

  const reset = useCallback(

```

```

    <div>
      <h4 className="text-sm sm:text-base font-bold">
        <p className="text-[12px] text-slate-400 lead">
          Клик по узлу = один поворот с его родителем
          <span className="font-mono font-bold text-indigo-600">
            поворотов выбираешь сам, разбор покажем в к
          </span>
        </p>
      </div>
      <button
        onClick={() => reset((presetIdx + 1) % PRESETS.length)}
        className="flex items-center gap-1.5 px-3 py-1 text-sm font-medium transition-colors shrink-0"
      >
        <Dices className="w-3.5 h-3.5" /> Другое дерево
      </button>
    </div>

    <div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar">
      {PRESETS.map((p, i) => {
        <button
          key={p.name}
          onClick={() => reset(i)}
          className={`px-3 py-1.5 rounded-lg text-[11px] font-medium transition-colors
            ${i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-slate-600"}
          `}
        >
          {p.name}
        </button>
      })}
    </div>

    <div className="bg-slate-900 rounded-xl border border-slate-800 p-4">
      <svg
        viewBox={`0 0 ${width} ${height}`}
        className="h-auto block mx-auto"
        style={{ minWidth: Math.min(width * 1.5, 420) }}
        role="img"
      >

```

```

        stroke={isTarget ? "#a5b4fc" : "#475568"}
        strokeWidth={2}
      />
      <text textAnchor="middle" dominantBaseline="middle">
        {n.key}
      </text>
    </g>
  );
}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
  <button
    onClick={undo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold">
    >
    <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
  </button>
  <button
    onClick={() => reset()}
    className="flex items-center gap-1.5 px-3 py-1.5 text-white font-bold transition-colors">
    >
    <RotateCcw className="w-3.5 h-3.5" /> Заново
  </button>
  <button
    onClick={() => setShowAnswer((s) => !s)}
    className={`flex items-center gap-1.5 px-3 py-1.5 text-white ${
      showAnswer
        ? "bg-amber-600/20 border-amber-500 text-amber-500"
        : "bg-slate-800 border-slate-700 text-slate-200"
    }`}
    >
    {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : <EyeOn className="w-3.5 h-3.5" />}
    {showAnswer ? "Скрыть разбор" : "Сдаться / показать"}
  </button>
</div>

```

```

        }}
      </ul>
    }}
    {wrong === 0 && moves.length > minimal && (
      <div className="opacity-80">Порядок верный,
    }}
  </div>
  }}
</div>
);
};

```

ФАЙЛ 62/82: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";

```

```

/**
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
 *
 * Что сделано ради понятности:
 * • у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
 * сразу видно, что дерево остаётся деревом поиска;
 * • под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ;
 * это и есть доказательство, что поворот ничего не ломает;
 * • кадр «прицел» ничего не двигает, только подсвечивает;
 * • на кадре «результат» остаются призраки – пунктирные узлы в
 * местах и стрелки «откуда → куда»;
 * • по умолчанию анимация НЕ крутится сама: пользователь
 * идёт в своём темпе. Автопрокрутка включается кнопкой.
 */

```

```

type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;

```

```

const ZIG: Case = {
  key: "zig",
  label: "Zig",
  when: "родитель x — уже корень, деда нет",
  rotations: "1 поворот",
  keys: { x: 20, p: 40 },
  inorder: ["A", "20", "B", "40", "C"],
  ranges: "A < 20 · B между 20 и 40 · C > 40",
  frames: [
    {
      ...ZIG_BEFORE,
      kind: "start",
      title: "Было: 20 висит под корнем 40",
      text: "Нам нужен ключ 20, а он на глубине 1. Деду",
      ms: RESULT_MS,
    },
    {
      ...ZIG_BEFORE,
      kind: "aim",
      title: "Прицел: крутим ребро 40 — 20",
      text: "Ничего пока не двигается. Смотри на подсве",
      focus: ["p", "x"],
      ms: AIM_MS,
    },
    {
      pos: { x: [160, 50], A: [80, 130], p: [245, 130],
      edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
      kind: "result",
      title: "Стало: 20 — корень",
      text: "40 стало правым сыном двадцатки. Поддерев",
        ева от 40 — это одно и то же место.",
      moved: ["x", "p", "B"],
      ms: RESULT_MS,
    },
  ],
};

```



```

        kind: "result",
        title: "Поворот 1: 40 поднялось над 60",
        text: "40 встало в корень, 60 опустилось к нему по
            убине 1, а не 2.",
        moved: ["p", "g", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZZ_S1,
        kind: "aim",
        title: "Прицел 2: ребро 40 – 20",
        text: "А теперь обычный одиночный поворот – тот же
            focus: ["p", "x"],
            ms: AIM_MS,
    },
    {
        pos: { x: [100, 45], A: [45, 118], p: [188, 118],
        edges: [ ["x", "A"], ["x", "p"], ["p", "B"], ["p",
        kind: "result",
        title: "Стало: 20 в корне, бамбук стал кустом",
        text: "Сравни с первым кадром: А и В были на глуб
            дерево, а не только достаёт ключ.",
        moved: ["x", "p", "A", "B"],
        ms: RESULT_MS,
    },
    ],
};

```

/* ----- Zig-Zag -----

```

const ZG_S0 = {
    pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
    edges: [ ["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
    pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
    edges: [ ["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

      text: "Остался одиночный поворот вокруг деда — и",
      focus: ["g", "x"],
      ms: AIM_MS,
    },
    {
      pos: { x: [180, 50], p: [90, 130], g: [275, 130],
      edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
      kind: "result",
      title: "Стало: 20 и 60 — два сына сорока",
      text: "Дерево стало идеально симметричным: все че
        очку». ",
      moved: ["x", "p", "g", "C"],
      ms: RESULT_MS,
    },
  ],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: string }> = {
  x: { fill: "#6366f1", stroke: "#a5b4fc" },
  p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
  g: { fill: "#f59e0b", stroke: "#fcd34d" },
  sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "x — ищем его",
  p: "p — родитель",
  g: "g — дед",
};

```

```
const isSubtree = (id: NodeId) => id === "A" || id ===
```

```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: Case[] }> = ({
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(id) : true);
  const ghosts = frame.kind === "result" ? (moved ?? []) : [];

```

```

    const pa = pos[a];
    const pb = pos[b];
    if (!pa || !pb) return null;
    const hot = Boolean(focus && focus.includes(a));
    return (
      <line
        key={` ${a} - ${b} - ${i} `}
        x1={pa[0]}
        y1={pa[1]}
        x2={pb[0]}
        y2={pb[1]}
        stroke={hot ? "#fbbf24" : "#475569"}
        strokeWidth={hot ? 5 : 2}
        opacity={focus && !hot ? 0.25 : 1}
        style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
      />
    );
  });
}

```

```

{Object.keys(pos) as NodeId[]}.map((id) => {
  const p = pos[id];
  if (!p) return null;
  const sub = isSubtree(id);
  const c = COLOR[sub ? "sub" : id];
  const r = sub ? 15 : 20;
  const hot = Boolean(focus && focus.includes(id));
  return (
    <g
      key={id}
      style={{
        transform: `translate(${p[0]}px, ${p[1]}px)`,
        transition: MOVE,
        opacity: dim(id) ? 0.28 : 1,
      }}
    >
      {hot && <circle r={r + 7} fill="none" stroke="black" />}
      <circle r={r} fill={c.fill} stroke={c.stroke} />
      <text

```

```

useEffect(() => {
  if (!playing) return;
  const t = setTimeout(() => setFrame((f) => (f + 1) %
  return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (
  <div className="w-full bg-slate-950 rounded-2xl border
    <div className="flex gap-2 mb-4 overflow-x-auto no
      {CASES.map((c, i) => (
        <button
          key={c.key}
          onClick={() => setCaseIdx(i)}
          className={`px-3 sm:px-4 py-2 rounded-lg text
            i === caseIdx ? "bg-indigo-600 text-white
          }`}
        >
          {c.label}
        </button>
      )
    >
  </div>

  <div className="mb-3 text-xs sm:text-[13px] text-
    <span className="font-bold text-indigo-300">Кор
    <span className="text-slate-500">({active.rotat
  </div>

  /* шаг + пояснение стоят НАД картинкой: сначала
  <div className="mb-3 rounded-xl border border-sla
    <div className="flex items-center gap-2 mb-1 fl
      <span
        className={`text-[10px] font-bold uppercase
          current.kind === "aim"
            ? "bg-amber-500/20 text-amber-300"
            : current.kind === "start"
              ? "bg-slate-700 text-slate-300"
              : "bg-indigo-500/20 text-indigo-300"
            }`}
      >

```

```

        style={
          playing
            ? { animation: `demoProgress ${duration}ms`
              : { width: "100%", background: "#334155"
            }
        }
      />
    </div>

```

```

<div className="flex flex-wrap items-center gap-2" >
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f - 1 + total) % total);
    }}
    disabled={idx === 0}
    className="px-3 py-2 rounded-lg bg-slate-800 text-white
      dark:hover:text-slate-300 text-xs font-bold transition-colors"
  >
    ← Назад
  </button>
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f + 1) % total);
    }}
    className="px-4 py-2 rounded-lg bg-indigo-600 text-white
      w-indigo-900/40"
  >
    {last ? "⏮ Сначала" : "Дальше →"}
  </button>
  <button
    onClick={() => setPlaying((p) => !p)}
    className="flex items-center gap-1.5 px-3 py-2 text-white
      text-xs font-bold transition-colors"
  >
    {playing ? <Pause className="w-3.5 h-3.5" /> : "Пуск"}
    {playing ? "Стоп" : "Авто"}
  </button>

```

```
};
```

ФАЙЛ 63/82: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from "react";
import {
  RotateCcw,
  HelpCircle,
  BookOpen,
} from "lucide-react";
```

```
interface SplayNode {
  key: number;
  left: SplayNode | null;
  right: SplayNode | null;
  x?: number;
  y?: number;
}
```

```
// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
  if (!root) return { key, left: null, right: null };
  if (key < root.key) {
    root.left = insertBST(root.left, key);
  } else if (key > root.key) {
    root.right = insertBST(root.right, key);
  }
  return root;
};
```

```
// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
  const y = x.left;
  if (!y) return x;
```

```

const [tree, setTree] = useState<SplayNode>(createInitialTree);
const [inputValue, setInputValue] = useState<string>('');
const [log, setLog] = useState<string[]>([
  "Дерево инициализировано. Попробуйте кликнуть на узел."
]);
const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

// Splay operation that tracks steps!
const splayStepByStep = (
  root: SplayNode | null,
  key: number,
): { newRoot: SplayNode | null; steps: string[] } => {
  const steps: string[] = [];
  if (!root) return { newRoot: null, steps: ["Дерево пустое."] };

  // We implement the classic top-down or bottom-up splay operation.
  // For visualization logs, we do a bottom-up explanation.
  let path: { node: SplayNode; dir: "left" | "right" | null }[] = [];
  let current: SplayNode | null = root;

  // Find the element or the element where search failed.
  let lastNode: SplayNode = root;
  while (current) {
    lastNode = current;
    if (key === current.key) {
      path.push({ node: current, dir: null });
      break;
    } else if (key < current.key) {
      path.push({ node: current, dir: "left" });
      current = current.left;
    } else {
      path.push({ node: current, dir: "right" });
      current = current.right;
    }
  }

  const targetKey = current ? key : lastNode.key;
  const isFound = !!current;

```

```

        if (!node.left) return node;
        steps.push(
            `Финальный поворот Zig (Правое вращение корня)
        );
        return rightRotate(node);
    } else {
        if (!node.right) return node;

        if (k < node.right.key) {
            // Zag-Zig (Right-Left)
            node.right.left = splayAction(node.right.left, target);
            if (node.right.left) {
                node.right = rightRotate(node.right);
                steps.push(`Компонент Zig-Zag: правый поворот
            )
        }
        } else if (k > node.right.key) {
            // Zag-Zag (Right-Right)
            node.right.right = splayAction(node.right.right, target);
            node = leftRotate(node);
            steps.push(
                `Вращение Zag-Zag (Левый поворот родителя д
            );
        }

        if (!node.right) return node;
        steps.push(
            `Финальный поворот Zag (Левое вращение корня)
        );
        return leftRotate(node);
    }
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
    setHighlightedNode(key);

```



```
// Assign coordinate positions using a simple recursion
const computeCoordinates = (
  node: SplayNode | null,
  x: number,
  y: number,
  levelWidth: number,
): void => {
  if (!node) return;
  node.x = x;
  node.y = y;
  if (node.left) {
    computeCoordinates(node.left, x - levelWidth, y + 1);
  }
  if (node.right) {
    computeCoordinates(node.right, x + levelWidth, y + 1);
  }
};

const drawTree = cloneTree(tree);
computeCoordinates(drawTree, 300, 40, 140);

// Render lines and nodes
const renderLines = (node: SplayNode | null): React.ReactNode => {
  if (!node) return [];
  let lines: React.ReactNode[] = [];
  if (node.left && node.left.x && node.left.y) {
    lines.push(
      <line
        key={`l-${node.key}-${node.left.key}`}
        x1={node.x}
        y1={node.y}
        x2={node.left.x}
        y2={node.left.y}
        stroke="#475569"
        strokeWidth="2"
      />,
    );
  }
};
```

```

        className="transition-all duration-300 group-
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="12px"
      fontWeight="bold"
      className="select-none pointer-events-none"
    >
      {node.key}
    </text>
  </g>,
);
circles = circles.concat(renderNodes(node.left));
circles = circles.concat(renderNodes(node.right));
return circles;
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    {/* Header */}
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <BookOpen className="w-5 h-5 text-indigo-400"
        Интерактивное Splay-дерево
      </h3>
      <p className="text-xs text-slate-400">
        Кликайте на вершины дерева, чтобы «вытащить»
        Splay.
      </p>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-12
      {/* Playfield */}
      <div className="lg:col-span-7 bg-[#0f172a] p-4

```

```

        setHighlightedNode(val);
    }
  }}
  className="bg-indigo-950 hover:bg-slate-900"
>
  Вставить
</button>
</form>

<button
  onClick={resetTree}
  className="flex items-center gap-1.5 px-3"
>
  <RotateCcw className="w-3.5 h-3.5" /> Сброс
</button>
</div>
</div>

{/* Logs */}
<div className="lg:col-span-5 bg-slate-950 border-1
  to overflow-y-auto custom-scrollbar">
  <h4 className="text-xs font-bold text-slate-400">
    Ход вращения (Splay-логи)
  </h4>
  <div className="flex-1 space-y-2 mb-4 overflow-y-auto">
    {log.map((step, idx) => (
      <div
        key={idx}
        className={`text-xs p-2.5 rounded transition-colors
          ${idx === log.length - 1
            ? "border-l-2 border-indigo-500 bg-slate-900"
            : "text-slate-400 bg-slate-900/50"}
        `}
      >
        {step}
      </div>
    ))}
  </div>
</div>

```

```

<code className="text-indigo-400">[30]<
поднимет в корень{" "}
<strong className="text-white">
    последний успешно просмотренный узел
</strong>
    , на котором он споткнулся! Это оставля
сверху.
</p>
</div>
<div className="mt-3 text-[10px] text-indigo
    Поиск настраивает локальный кэш под реали
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
<div>
    <p className="font-bold text-slate-300 mb
        2. Эффект качелей (Swing)
    </p>
    <p className="text-slate-400 leading-rela
        Если агент запрашивает по очереди{" "}
        <code className="text-rose-400">[10]</c
        <code className="text-rose-400">[60]</c
        углы), дерево будет превращаться в палк
        сторону. Первая операция Splay(60) стои
        <strong className="text-white">0(n)</st
        глубину n. Вторая Splay(10) заставляет
        Постоянный свинг убивает производительн
        сложности <strong className="text-white
    </p>
</div>
<div className="mt-3 text-[10px] text-rose-
    Кэшировать свингующие пары на концах – фа
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
<div>

```

```

    isDirty: boolean;
    animState: 'in' | 'idle' | 'washing' | 'clean';
  }

const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: ' ', color: 'f' },
  { name: 'Тарелка из-под пиццы', emoji: ' ', color: 'f' },
  { name: 'Блюдец от торта', emoji: ' ', color: 'from-p' },
  { name: 'Тарелка от тако', emoji: ' ', color: 'from-y' },
  { name: 'Тарелка из-под суши', emoji: ' ', color: 'fr' },
  { name: 'Сковорода от глазуньи', emoji: ' ', color: ' ' },
];

let counter = 0;

export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>(
    [
      { id: 'p1', name: 'Тарелка из-под спагетти', emoji: ' ', state: 'idle' },
      { id: 'p2', name: 'Тарелка из-под пиццы', emoji: ' ', state: 'idle' },
      { id: 'p3', name: 'Блюдец от торта', emoji: ' ', state: 'idle' },
    ]
  );

  const [cleanRack, setCleanRack] = useState<Plate[]>(
    []
  );
  const [isWashing, setIsWashing] = useState(false);
  const [showSoap, setShowSoap] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>(
    []
  );

  const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

  // PUSH: Положить грязную тарелку сверху стопки
  const pushPlate = useCallback(() => {
    if (isWashing) return;
    if (dirtyStack.length >= 7) {
      addLog('⚠ Стопка слишком высокая, тарелки могут р

```

```

// Step 1: Start washing animation (sponge and soap
setDirtyStack(prev => prev.map(p => p.id === topPlate.id)
setShowSoap(true);

setTimeout(() => {
  // Step 2: Wash complete. Plate becomes clean and
  const cleanPlate: Plate = {
    ...topPlate,
    isDirty: false,
    animState: 'clean',
  };

  setCleanRack(prev => [cleanPlate, ...prev].slice(0, prev.length + 1);
  setDirtyStack(prev => prev.slice(0, prev.length - 1));
  setShowSoap(false);
  setIsWashing(false);
  addLog(` Готово! Чистая тарелка ${topPlate.emoji}`, 850);
}, [dirtyStack, isWashing]);

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', imState: 'in' },
    { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', mState: 'in' },
  ]);
  setCleanRack([]);
  setIsWashing(false);
  setShowSoap(false);
  setLog([' Стол сброшен. Посуда снова грязная!']);
};

const topIndex = dirtyStack.length - 1;

```

```

        onClick={reset}
        className="px-5 py-3.5 rounded-2xl bg-slate-800
            r-pointer border border-slate-700"
    >
        Сброс
    </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12" >

    {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
    <div className="md:col-span-7 bg-slate-900 rounded-3xl
        [380px]">
        <div className="absolute top-4 left-4 text-[10px] font-mono">
            Грязная стопка (Стек вызовов / LIFO)
        </div>

        {/* Визуализация раковины */}
        <div className="absolute top-4 right-4 flex items-center
            r-slate-800/80 text-[10px] font-mono">
            <span className="w-1.5 h-1.5 rounded-full bg-slate-700
                border border-slate-600" />
            <span>РАКОВИНА </span>
        </div>

        {/* Мыльные пузыри при мытье */}
        {showSoap && {
            <div className="absolute inset-0 pointer-events-none">
                <div className="absolute w-6 h-6 rounded-full
                    bg-slate-700/30 border border-slate-600" />
                <div className="absolute w-4 h-4 rounded-full
                    bg-slate-700/30 border border-slate-600" />
                <div className="absolute w-5 h-5 rounded-full
                    bg-slate-700/30 border border-slate-600" />
                <div className="absolute text-4xl anim-crown" />
            </div>
        })}
    </div>

```

```

        <span className="text-lg">{plate.em
        <span className="text-slate-800 tex
        {isTop && (
            <span className="text-[9px] bg-bl
                LIFO
            </span>
        )}
    </div>
    );
    }}}
</div>
})

{/* Столешница раковины */}
<div className="w-full h-4 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 round
    <div className="absolute top-4 left-4 text-[1
        Сушилка для чистой посуды
    </div>

    <div className="absolute top-4 right-4 flex i
        order-emerald-800/80 text-[9px] font-mono">
        <Sparkles className="w-3.5 h-3.5 animate-pu
        <span>ЧИСТО </span>
    </div>

    <div className="flex-1 flex flex-col justify-
        {cleanRack.length === 0 ? (
            <div className="flex-1 flex flex-col item
                <span className="text-5xl mb-2"> </span>
                <p className="text-xs font-bold font-mo
                <p className="text-[10px] mt-1">Здесь 6
            </div>
        ) : (
            <div className="flex flex-col gap-2 w-ful

```



```
import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

function generateKmpSteps(pattern: string, text: string) {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];

  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

  for (let i = 1; i < n; i++) {
    let j = pi[i - 1];

    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });

    while (j > 0 && s[i] !== s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
      j = pi[j - 1];
    }

    if (s[i] === s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
      j++;
    } else {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
    }

    pi[i] = j;
  }
}
```

```

        steps.push({ i, l, r, zTarget: z[i], zSource: z[i]}' разные. Стоп.` }));
    } else {
        steps.push({ i, l, r, z: [...z], desc: `Унë
    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксирываю

    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
        steps.push({ i, l, r, z: [...z], desc: ` 0
    }

    if (z[i] === pattern.length) {
        steps.push({ i, l, r, z: [...z], found: true
    }
}
return { s, steps, patternLength: pattern.length };
}

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp"
const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
const [pattern, setPattern] = useState("aba");
const [text, setText] = useState("abacaba");

```

```

// Ensure inputs are valid dynamically
const safePattern = pattern || " ";
const safeText = text || " ";

```

```

const { s, steps, patternLength } = useMemo(() => {
    if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
    else return generateZSteps(safePattern, safeText);
}, [mode, safePattern, safeText]);

```

```

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

```

```

    " : "text-slate-400 hover:text-slate-300"}`
      Z-ФУНКЦИЯ
    </button>
  </div>

  <div className="flex items-center gap-2" >
    <div className="bg-slate-800 p-1 flex items-center gap-2" >
      <span className="text-[10px] text-white" >Z-ФУНКЦИЯ</span>
      <input value={pattern} onChange={onChangePattern} type="text" />
    </div>
    <div className="bg-slate-800 p-1 flex items-center gap-2" >
      <span className="text-[10px] text-white" >Z-ФУНКЦИЯ</span>
      <input value={text} onChange={onChangeText} type="text" />
    </div>
  </div>
</div>

<div className="bg-slate-950 rounded-xl border-[1px] flex items-center justify-start" >
  <div className="flex gap-1.5 px-4 h-full" >
    {s.split("").map((char, index) => {
      const isPattern = index < pattern.length;
      const isSeparator = index === pattern.length;

      let borderColor = "border-slate-900";
      let bgColor = "bg-slate-900";
      let textColor = isSeparator ? "text-slate-400" : "text-slate-300";

      // KMP Logic
      if (mode === "kmp") {
        if (step.highlight?.include(char)) {
          borderColor = "border-slate-400";
          bgColor = step.match === char ? "bg-slate-400" : "bg-slate-800";
        }
      }
    })}
  </div>
</div>

```

```

z-20">
        { /* KMP fingers */
        {mode == "kmp" && step
            <div className="abs
                <span className
            </div>
        }}
        {mode == "kmp" && step
            <div className="abs
                <span className
            </div>
        }}
        {mode == "kmp" && step
            <div className="abs
                <span className
            </div>
        }}

20">
        { /* Z fingers */
        {mode == "z" && step.i
            <div className="abs
                <span className
            </div>
        }}
        {mode == "z" && step.z
            <div className="abs
                <span className
            </div>
        }}
        {mode == "z" && step.z
            <div className="abs
                <span className
            </div>
        }}

e z-20">
        { /* Z fingers */
        {mode == "z" && step.i
            <div className="abs
                <span className
            </div>
        }}
        {mode == "z" && step.z
            <div className="abs
                <span className
            </div>
        }}
        {mode == "z" && step.z
            <div className="abs
                <span className
            </div>
        }}
    
```

```

        overflow-hidden">
            <div className="absolute top-0 right-0">
                <span className="font-mono text-sm">
                    {steps.length}
                </span>
            </div>
            <div className="text-[10px] text-slate-400">
                {steps.map((step, index) => (
                    <div className="text-sm text-slate-400">
                        {step.desc}
                    </div>
                ))}
            </div>
        </div>

        { /* Why this matters card */ }
        <div className="bg-slate-800 p-4 rounded-xl">
            <span className="text-2xl mt-1"> </span>
            <p className="text-xs text-slate-400">
                Обрати внимание на хитрость с решёткой.
                Мы пишем <b>Шаблон + # + Текст</b>.
                Когда значени {mode === 'kmp' ? "проблема не была найдена!" : "Пройди шаги вручную, посмотри, как работает алгоритм для пропуска работы (в Z)."}
            </p>
        </div>

        { /* Code Sneak Peek */ }
        <div className="bg-slate-900 border border-slate-800 px-4 py-2">
            <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase">
                <span>Код C++ {mode === 'kmp' ? 'КМП' : 'З'}
            </div>
            <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}
` : ''}
            </pre>
        </div>
    </div>

```

```

    u: number;
    v: number;
}

const dagNodes: TNode[] = [
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
  { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
  { id: 2, x: 280, y: 230, label: "2", name: "Линейный (Линейная алгебра)" },
  { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
  { id: 4, x: 640, y: 150, label: "4", name: "Машинное (Машинное обучение)" },
];

const dagEdges: TEdge[] = [
  { u: 0, v: 1 },
  { u: 0, v: 2 },
  { u: 1, v: 3 },
  { u: 2, v: 3 },
  { u: 3, v: 4 },
];

interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
  currentNode: number | null;
  topoList: number[];
  dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const listSteps: TStep[] = [];
    const visited = new Set<number>();
    const fullyProcessed = new Set<number>();

```

```

        u,
      );
    }
  }

  fullyProcessed.add(u);
  dfsStack.pop();
  topoList.push(u); // prepend behavior: we write in reverse
  recordStep(
    `DFS: закончили обработку '${dagNodes[u].name}'`,
    u,
  );
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    recordStep(
      `Запускаем новый обход DFS от нетронутой вершины ${i}`,
      null,
    );
    dfs(i);
  }
}

recordStep(
  "Топологическая сортировка завершена! Полученный список вершин:",
  null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
  desc: "Запуск...",
  visited: new Set(),
  fullyProcessed: new Set(),
};
```

```

<div className="flex items-center gap-2 bg-slate-500" >
  <button
    onClick={togglePlay}
    className="flex items-center gap-2 px-3 py-1.5 text-white"
  >
    {isPlaying ? (
      <Pause className="w-4 h-4 ml-1" />
    ) : (
      <Play className="w-4 h-4 ml-1" />
    )}
    {isPlaying ? "Пауза " : "Старт "}
  </button>

  <button
    onClick={reset}
    className="flex items-center justify-center gap-2 px-3 py-1.5 text-white"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
  {/* Playfield SVG */}
  <div className="lg:col-span-8 bg-[#0B1120] relative" >
    <svg className="w-full h-full max-w-[650px]" >
      <defs>
        <marker
          id="dag-arrow"
          viewBox="0 0 10 10"
          refX="24"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >

```



```

{ /* Nodes */
{dagNodes.map({node} => {
  const isCurrent = step.currentNode === node.id
  const isVisited = step.visited.has(node.id)
  const isProcessed = step.fullyProcessed.has(node.id)

  let fill = "#1e293b";
  let stroke = "#334155";
  let textColor = "text-slate-400";
  void textColor;

  if (isCurrent) {
    fill = "#312e81";
    stroke = "#818cf8";
    textColor = "text-white font-bold";
  } else if (isProcessed) {
    fill = "#022c22";
    stroke = "#059669";
    textColor = "text-emerald-300";
  } else if (isVisited) {
    fill = "#3730a3";
    stroke = "#6366f1";
    textColor = "text-indigo-200";
  }

  return (
    <g key={node.id}>
      <rect
        x={node.x - 55}
        y={node.y - 22}
        width="110"
        height="44"
        rx="8"
        fill={fill}
        stroke={stroke}
        strokeWidth="2"
        className="transition-all duration-100"
      />
    </g>
  )
})}

```

```

    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-auto" >
    { /* Topological List output */ }
    <div>
      <span className="text-[10px] text-slate-500" >
        РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
      </span>
      <div className="bg-slate-900/60 p-2.5 rounded" >
        pr-1 text-slate-300">
          {step.topoList.length === 0 ? (
            <span className="text-slate-600 text-
              ожидаем обработку узлов...
            </span>
          ) : (
            getTopoListString()
          )}
        </div>
      </div>

    { /* Color key */ }
    <div className="space-y-1">
      <span className="text-[10px] text-slate-500" >
        ВЕРШИНЫ В DFS:
      </span>
      <div className="flex items-center gap-2" >
        <span className="w-3.5 h-3.5 rounded bg-
          <span className="text-slate-400 text-[10px]
            Серые (зашли)
          </span>
        </div>
        <div className="flex items-center gap-2" >
          <span className="w-3.5 h-3.5 rounded bg-
            <span className="text-slate-400 text-[10px]
              Черные (готово)
            </span>
          </div>
    </div>
  </div>

```

Универсальное пособие • полный исходный код

import React, { useEffect, useMemo, useState } from "re

/**

* Интерактивный Tgear – шесть режимов:

* build – сборка: ВИДИМАЯ сортировка точек (и чем с

* координатная плоскость НАВЕРХУ (ассоциаци

* чек-лист инвариантов (автопроверка, как у

* split – разрез по x с двумя корзинами (грамотная

* merge – склейка двух деревьев по y.

* erase – удаление ПРОИЗВОЛЬНОЙ точки: найти по x,

* layers – миф «дети в массиве по $2k/2k+1$ »: работает

* search – «отсортирую бинарным поиском»: поиск не с

* сортировка платит $\geq n \cdot \log n$ сравнений, со

*

* Канонический набор: все x и все y различны $\Rightarrow$ дерево

* Узел идентифицируется парой $id = "x:y"$.

*/

export interface Pair {

key: number;

grade: number;

}

type NodeId = string;

const pairId = (key: number, grade: number): NodeId =>

interface TNode {

id: NodeId;

key: number;

grade: number;

left: TNode | null;

right: TNode | null;

}

export const POINTS: Pair[] = [

{ key: 45, grade: 8 },

{ key: 3, grade: 6 },

{ key: 6, grade: 5 },

{ key: 2, grade: 4 },

```

-----
export interface SplitResult {
  l: TNode | null;
  r: TNode | null;
  trace: { atId: NodeId; goLeft: boolean; note: string }
}
export function splitTree(root: TNode | null, x: number) {
  const trace: SplitResult["trace"] = [];
  const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
    if (!t) return { l: null, r: null };
    if (t.key <= x) {
      const sub = go(t.right);
      trace.push({ atId: t.id, goLeft: true, note: `(${t.id} перемещено в правое поддерево.` });
      t.right = sub.l;
      return { l: t, r: sub.r };
    }
    const sub = go(t.left);
    trace.push({ atId: t.id, goLeft: false, note: `(${t.id} перемещено в левое поддерево.` });
    t.left = sub.r;
    return { l: sub.l, r: t };
  };
  const res = go(root);
  return { l: res.l, r: res.r, trace };
}

```

```

export interface MergeResult {
  root: TNode | null;
  trace: { aId: NodeId | null; bId: NodeId | null; chosen: string }
}
export function mergeTree(a: TNode | null, b: TNode | null) {
  const trace: MergeResult["trace"] = [];
  const go = (a: TNode | null, b: TNode | null): TNode | null {
    if (!a || !b) {
      trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosen: `Выбор сделан целиком.` });
      return a ?? b;
    }
  }
}

```

Универсальное пособие • полный исходный код

```
-----  
  t ? { id: t.id, key: t.key, grade: t.grade, left: clo
```

```
/* ----- валидация treap (для чек-листа и ст
```

```
export function checkTreap(root: TNode | null): { bst: B  
  let bst = true;  
  let heap = true;  
  let prev: number | null = null;  
  const walk = (n: TNode | null, min: number, max: number,  
    if (!n) return;  
    if (n.key <= min || n.key > max) bst = false;  
    if (parentGrade !== null && n.grade > parentGrade) heap = false;  
    walk(n.left, min, n.key, n.grade);  
    if (prev !== null && n.key < prev) bst = false;  
    prev = n.key;  
    walk(n.right, n.key, max, n.grade);  
  };  
  walk(root, -Infinity, Infinity, null);  
  return { bst, heap };  
}
```

```
/* ----- раскладка сцены дерева -----
```

```
const layoutTree = (root: TNode | null) => {  
  const pos = new Map<NodeId, { px: number; py: number }>();  
  let slot = 0;  
  const walk = (n: TNode | null, depth: number) => {  
    if (!n) return;  
    walk(n.left, depth + 1);  
    pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 100 });  
    slot += 1;  
    walk(n.right, depth + 1);  
  };  
  walk(root, 0);  
  return pos;  
};
```

```
const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId; }
```

```

const b = byId(e.to);
if (!a || !b) return null;
return (
  <line
    key={`te-${e.from}-${e.to}`}
    x1={a.px} y1={a.py} x2={b.px} y2={b.py}
    stroke={edgeColor ? edgeColor(e.from, e.to) : 'black'}
    strokeWidth={2.5}
    strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
    className="transition-all duration-500"
  />
);
}}}
{nodes.map((n) => {
  const p = pairParts(n.id);
  const pulse = pulseId === n.id;
  return (
    <g key={`tn-${n.id}`} transform={`translate(${n.px}, ${n.py})`}
      <circle r={pulse ? 24 : 20} fill={nodeColor(n)}
        stroke={pulse ? 'black' : 'gray'} strokeWidth={pulse ? 4 : 2} className="transition-all duration-500" />
      <text y={-2} fill="white" fontSize="13" fontWeight="bold">{n.id}</text>
      <text y={13} fill="black" fontSize="10">{n.name}</text>
    </g>
  );
})}
</svg>
{subtitle && <p className="text-xs text-slate-400">{subtitle}</p>}
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
  const [a, b] = id.split(":").map(Number);
  return [a, b];
};

/* ----- плоскость (клетчатая бумага) ----- */

```

```

{points.map((p) => {
  const done = drawnEdges.some((e) => (e.a.key == p.key))
  return (
    <g key={`pp-${p.key}:${p.grade}`} className="point" >
      <circle cx={PX(p.key)} cy={PY(p.grade)} r={radius} fill="black" stroke="black" strokeWidth={1.6} />
      <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size="10" font-family="monospace">{p.key}</text>
    </g>
  );
})}

{pending && (
  <g className="transition-all duration-500">
    <circle cx={PX(pending.key)} cy={PY(pending.grade)} r={radius} fill="black" stroke="black" strokeWidth={1.6} />
    <text x={PX(pending.key)} y={PY(pending.grade) + 3.5} font-size="10" font-family="monospace">{pending.key}</text>
  </g>
)}
</svg>
);
};

```

/* ===== РЕЖИМ BUILD ===== */

```

type BuildPhase = "sort" | "connect";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } => {
  const pairs: Pair[] = [];
  const errors: string[] = [];
  const seen = new Set<string>();
  const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
  for (const chunk of chunks) {
    const nums = chunk.split(/\sxx*/).filter((Boolean))
  }
}

```

```

    if (node.left) { es.push({ a: { key: node.key, gr
    if (node.right) { es.push({ a: { key: node.key, g
    walk(node.left);
    walk(node.right);
  };
  walk(userTree);
  return es;
}, [userTree]));
const leftRightEdges = useMemo(() => [...canonEdges].

useEffect(() => {
  if (!playing) return;
  const t = setTimeout(() => {
    if (phase === "sort") {
      if (algo === "none") { setPlaying(false); return
      if (sortIdx < 3) setSortIdx(sortIdx + 1);
      else setPlaying(false);
    } else {
      const total = order === "y-desc" ? canonEdges.l
      if (connIdx < total) setConnIdx(connIdx + 1);
      else setPlaying(false);
    }
  }, 900);
  return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, can

const startConnect = (ord: ConnectOrder) => {
  setOrder(ord);
  setPhase("connect");
  setConnIdx(0);
  setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? M
const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

```



```

<div className="flex items-center gap-2">
  {phase === "sort" ? (
    <>
      <span className="text-xs text-slate-400">
        {[ "quicksort", "counting" ] as const }.map((a) => {
          <button
            key={a}
            onClick={() => { setAlgo(a); setSort(a); }}
            disabled={n < 2}
            className={`px-3 py-1.5 rounded-lg text-white
              bg-emerald-600 text-white` : "bg-slate-700 text-slate-300"}
          >
            {a === "quicksort" ? "quicksort (рекурсия)" : "по y ↓ (алгоритм)"}
          </button>
          <button
            onClick={() => startConnect("left-right")}
            className={`px-3 py-1.5 rounded-lg text-white
              bg-emerald-600 text-white` : "bg-slate-700 text-slate-300 hover:bg-emerald-500"}
          >
            слева направо
          </button>
        </>
      </span>
    ) : (
      <>
        <span className="text-xs text-slate-400">
          <button
            onClick={() => startConnect("y-desc")}
            className={`px-3 py-1.5 rounded-lg text-white
              bg-emerald-600 text-white` : "bg-slate-700 text-slate-300"}
          >
            по y ↓ (алгоритм)
          </button>
          <button
            onClick={() => startConnect("left-right")}
            className={`px-3 py-1.5 rounded-lg text-white
              bg-emerald-600 text-white` : "bg-slate-700 text-slate-300 hover:bg-emerald-500"}
          >
            слева направо
          </button>
        </>
      </span>
    )
  ) : (
    <>
      <span className="text-xs text-slate-400">
        <button
          onClick={() => startConnect("y-desc")}
          className={`px-3 py-1.5 rounded-lg text-white
            bg-emerald-600 text-white` : "bg-slate-700 text-slate-300"}
        >
          по y ↓ (алгоритм)
        </button>
        <button
          onClick={() => startConnect("left-right")}
          className={`px-3 py-1.5 rounded-lg text-white
            bg-emerald-600 text-white` : "bg-slate-700 text-slate-300 hover:bg-emerald-500"}
        >
          слева направо
        </button>
      </span>
    </>
  )
}</div>
</div>
<Plane points={userPairs} drawnEdges={phase ===

```



```

return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center gap-3">
      <span className="text-sm text-slate-300">Удалить
      <select value={key} onChange={(e) => { setKey(N
        te-700 rounded-lg px-3 py-1.5 text-sm font-mo
        {POINTS.map((p) => <option key={p.key} value=
      </select>
      <button onClick={() => setStepIdx(Math.max(0, i
        bg-slate-700 text-white disabled:opacity-30">
      <button onClick={() => setStepIdx(Math.min(step
        font-bold bg-indigo-600 text-white disabled:op
      <span className="text-xs text-slate-400">шаг {i
    </div>

    {!done ? (
      <div className="bg-slate-900/40 rounded-xl p-4 l
        <TreeView
          root={fullTree}
          nodeColor={(id) => (pathSet.has(id) ? "#783
          nodeStroke={(id) => (pathSet.has(id) ? "#f5
          subtitle="Куча сама умеет удалять только ве
        />
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4 l
        <h4 className="text-sm font-bold text-slate-3
        <TreeView root={res.root} subtitle="Валидность
        <ul className="mt-2 space-y-1 text-sm">
          <li> обход слева-направо по-прежнему сорти
          <li> каждое ребро вниз по y (куча целая) —
        </ul>
      </div>
    ))

    <div className="text-xs text-slate-300 bg-slate-9
      {steps[idx]?.note}

```

```
</div>
```

```
{pick === "heap" ? (
  <div className="bg-slate-900/40 rounded-xl p-4" >
    <h4 className="text-sm font-bold text-slate-300" >
      4>
    <div className="flex gap-2 overflow-x-auto mb-2" >
      {heapArr.slice(1).map((v, i) => (
        <div key={i} className="px-3 py-2 rounded" >
          <div className="text-slate-500 text-[10px]" >
            <div className="text-white font-bold">{v}</div>
            <div className="text-emerald-400 text-[10px]" >
              {i}</div>
          </div>
        </div>
      ))}
    </div>
    <p className="text-xs text-slate-400">Полное дерево
      →  $2i/2i+1$  попадает в узел.</p>
  </div>
) : (
```

```
<div className="bg-slate-900/40 rounded-xl p-4" >
  <h4 className="text-sm font-bold text-rose-300" >
    4>
  <TreeView root={full} subtitle="Попробуй «посмотреть»" />
  <div className="flex gap-2 overflow-x-auto mt-2" >
    {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((v, i) => (
      const id = slots[i];
      return (
        <div key={i} className={`px-2.5 py-2 rounded border-slate-700` : `bg-rose-950/40 border-rose-950`} >
          <div className="text-slate-500 text-[10px]" >
            {id} <div className="text-white font-[10px]" >
              {v}</div>
          </div>
        </div>
      )
    )}
  </div>
  <p className="text-xs text-rose-300 mt-2">Смотри, как левая ветка ушла на 4 уровня и заняла да
```

```

    if (sortedByKey[mid].key === target) { steps.push(
      нашли за ${steps.length + 1} сравнение(й).` });
    if (sortedByKey[mid].key < target) { steps.push({
      } — вся левая половина выброшена.` }); lo = mid
    else { steps.push({ lo, hi, mid, found: false, no
      ` }); hi = mid - 1; }
  }
  return steps;
}, [sortedByKey]);

```

```

const wrongMid = Math.floor((shuffled.length - 1) / 2)
const comparisons = stage === 2 ? bidx + 1 : 0;

```

```

return (
  <div className="space-y-4">
    <div className="flex gap-2">
      {stage < 2 && (
        <button onClick={() => { setStage(stage === 0
          digo-600 text-white">
            {stage === 0 ? "Найти y для x=7 бинарным по
          </button>
        )}
        {stage === 2 && (
          <button onClick={() => setBidx(Math.min(bsStep
            -1.5 rounded-lg text-xs font-bold bg-indigo
            Шаг поиска →
          </button>
        )}
        <button onClick={() => { setStage(0); setBidx(0
          Сброс</button>
      </div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
  {(stage === 2 ? sortedByKey : shuffled).map((p,
    const highlight = stage === 1 && i === wrongMid
    const inBs = stage === 2 && (() => { const s :
    const isMid = stage === 2 && (() => { const s :
    const isFound = stage === 2 && bidx >= bsStep

```

```

    { id: "build", label: "Собрать" },
    { id: "split", label: "Split ≈ " },
    { id: "merge", label: "Merge  " },
    { id: "erase", label: "Erase   " },
    { id: "layers", label: "Миф: 2k/2k+1" },
    { id: "search", label: "Поиск ≠ сортировка" },
  ];

export const TreapBuildViz = () => {
  const [tab, setTab] = useState<Tab>("build");

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border">
      <div className="flex flex-wrap items-center justify-center">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-emerald-600 text-white">
            <span className="text-2xl"> </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white">
              <p className="text-sm text-emerald-300">Пап
            </p>
          </div>
        </div>
      </div>

      <div className="flex flex-wrap gap-2 mb-5">
        {TABS.map((t) => (
          <button
            key={t.id}
            onClick={() => setTab(t.id)}
            className={`px-4 py-2 rounded-lg text-xs font-medium
              900/60 text-slate-400 hover:text-white border
            `}
            >
            {t.label}
          </button>
        ))}
      </div>
    </div>
  );
};

```

```
-----
import { SplayRotationSandbox } from "../SplayRotationSandbox";
import { StackViz } from "../StackViz";
import { StringAlgorithmsViz } from "../StringAlgorithmsViz";
import { TopologicalSortViz } from "../TopologicalSortViz";
import { WaterfallAnimationWidget } from "../WaterfallAnimation";
import ChapterImage from "../ChapterImage";
import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumViz";
import { SparseTableViz } from "../visualizers/SparseTableViz";
import { TreapBuildViz } from "../TreapBuildViz";
```

```
/** Разбор поворота и песочница всегда идут парой: посмотри
**
```

```
 * Анимация и песочница идут друг под другом, а не в две
 * колонке дерево сжималось до нечитаемого размера, а по
 * компактности. Между ними – подпись, объясняющая перемену
 */
```

```
const SplayRotationsPair: React.FC = () => (
  <div className="space-y-4">
    <SplayRotationsViz />
    <p className="flex items-start gap-2 text-[12px] leading-3 py-2">
      <span aria-hidden="true"> </span>
      <span>
        Разобрался – проверь себя: ниже то же самое дерево
        не решишь.
      </span>
    </p>
    <SplayRotationSandbox />
  </div>
);
```

```
export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
```



```

treap: {
  title: "Treap: плоскость → дерево → Split/Merge/Era
  hint: "6 вкладок: Собрать (сортировка + соединение
    , миф про  $2k/2k+1$ , поиск  $\neq$  сортировка.",
  Component: TreapBuildViz,
},
"prefix-sums-2d": {
  title: "Префиксные суммы (1D и 2D)",
  hint: "Наведите на число – увидите, из чего оно сло
  Component: PrefixSumViz,
},
"dynamic-programming": {
  title: "Динамическое программирование",
  hint: "Таблица подзадач заполняется на глазах.",
  Component: DPVisualizer,
},
"splay-tree": {
  title: "Splay-дерево",
  hint: "Покадровый разбор трёх поворотов, песочница
  Component: () => (
    <div className="space-y-10">
      <SplayRotationsPair />
      <SplayTreeViz />
    </div>
  ),
},
"splay-rotations": {
  title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
  hint: "Сверху – покадровый разбор поворота, снизу –
  Component: () => <SplayRotationsPair />,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
"top-sort": { title: "Топологическая сортировка", Component:
"scc-kosaraju": { title: "Компоненты сильной связности", Component:
"graph-articulation": { title: "Точки сочленения", Component:
"bridges-code": { title: "Мосты: DFS + tin/low", Component:
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component:
"planarity-euler-formula": { title: "Планарность: K5

```

```
      <div className="space-y-10">
        <WaterfallAnimationWidget />
        <SalmonAutomatonWidget />
      </div>
    ),
  },
intro: { title: "Мнемокарточки", Component: MnemonicCard,
  "everyday-basics": {
    title: "Бытовой тренажёр: стек, очередь, куча",
    hint: "Тарелки, очередь в столовой и мешок гипотез",
    Component: Simulator,
  },
};

export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 69/82: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
  x: number; // Координаты для SVG водопада
  y: number;
  depth: number;
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
```

```

const [textToScan, setTextToScan] = useState<string>('');
const [currentIndex, setCurrentIndex] = useState<number>(0);
const [currentNode, setCurrentNode] = useState<number>(0);
const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>('');
  'Режим поиска: Введите текст и жмите «Следующий шаг»';
);
const [jumpPath, setJumpPath] = useState<{ from: number, to: number }>(null);
const [foundMatches, setFoundMatches] = useState<{ index: number, match: string }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(1);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const clean = e.target.value.toUpperCase().replace(/ /g, '');
  setTextToScan(clean);
  resetSearchSimulation();
};

const resetSearchSimulation = () => {
  setCurrentIndex(0);
  setCurrentNode(0);
  setIsSearchDone(false);
  setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
  setJumpPath(null);
  setFoundMatches([]);
  setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
  if (currentIndex >= textToScan.length) {
    setIsSearchDone(true);
    setSearchLog('Мы проплыли весь текст! Лосось доволен');
    setActiveSearchCodeLine(4);
    return;
  }

```

```

    let temp = curr;
    while (temp !== 0) {
        if (WATERFALL_NODES[temp].is_terminal) {
            WATERFALL_NODES[temp].words.forEach((word) => {
                currentMatches.push({ index: currentIndex, word });
                matchedWordsForLog.push(word);
            });
        }
        temp = WATERFALL_NODES[temp].term_link;
    }

    if (matchedWordsForLog.length > 0) {
        logMsg += `    БИНГО! Найдено слово: ${matchedWordsForLog[0].word}\n`;
        setActiveSearchCodeLine(4);
    }

    setCurrentNode(curr);
    setCurrentIndex(currentIndex + 1);
    setSearchLog(logMsg);
    if (currentMatches.length > 0) {
        setFoundMatches((prev) => [...prev, ...currentMatches]);
    }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Другие вершины сразу инициализируются с fail = ROOT',
        codeLine: 1,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'ø',
        checkingBranchesFrom: null,
    },

```



```

        Анимация водопада: Полное обучение автома
    </h4>
    <p className="text-sm text-slate-400">
        Визуализация пошагового построения всех f
    </p>
</div>
</div>

{/* Переключатель режимов */}
<div className="bg-slate-900 p-1.5 rounded-xl b
    <button
        onClick={() => setActiveMode('training')}
        className={`flex items-center space-x-1.5 p-
            activeMode === 'training'
                ? 'bg-indigo-600 text-white shadow-lg s
                : 'text-slate-400 hover:text-white'
            }`
        >
        <GitBranch className="w-4 h-4" />
        <span>Режим 1: Полное обучение (BFS от корн
    </button>
    <button
        onClick={() => setActiveMode('search')}
        className={`flex items-center space-x-1.5 p-
            activeMode === 'search'
                ? 'bg-blue-600 text-white shadow-lg sha
                : 'text-slate-400 hover:text-white'
            }`
        >
        <Play className="w-4 h-4" />
        <span>Режим 2: Поиск в тексте</span>
    </button>
</div>
</div>

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
    /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
```

```

        </button>
        <button
          onClick={() => setTrainStep((prev) => M
          disabled={trainStep === TRAINING_STEPS_
          className="flex-1 bg-indigo-600 hover:b
          ition-all shadow-lg shadow-indigo-900/50"
        >
          {trainStep === TRAINING_STEPS_INFO.leng
        </button>
      </div>
    </div>

    /* Блок с кодом построения fail-ссылки и опи
    <div className="lg:col-span-2 bg-slate-900/90
      >
        <div>
          <h5 className="text-white font-bold mb-3
            <span className="flex items-center gap-
              <span>❌</span> Код BFS построения fail
            </span>
            <span className="text-xs bg-indigo-950 l
              Вершина: #{targetTrainingNode.id} ({t
            </span>
          </h5>
          <div className="bg-slate-950 p-4 rounded-
            <div className={`p-1.5 rounded flex ite
              -l-4 border-indigo-500 text-indigo-300 font
                <span>1. let u = trie[r][char]; // Те
                  {currentTrainInfo.codeLine === 1 && <
                тут</span>}
            </div>
            <div className={`p-1.5 rounded flex ite
              -l-4 border-indigo-400 text-indigo-200 font
                <span>2. let v = fail[r]; // Подъем к
                  ainInfo.inspectedParentFail].label}}</span>
                  {currentTrainInfo.codeLine === 2 && <
                к родителю</span>}
            </div>

```

```

<h5 className="text-white font-bold mb-2" >
  <span>» </span> Текст для сканирования
</h5>
<p className="text-xs text-slate-400 mb-4" >
  Введи текст, чтобы лосось просканировал
</p>
<input
  type="text"
  value={textToScan}
  onChange={handleTextChange}
  maxLength={15}
  placeholder="ВВЕДИТЕ ТЕКСТ..."
  className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
/>
</div>

<div>
  <div className="mb-4 flex flex-wrap gap-1" >
    <span className="text-xs text-slate-400" >
      {textToScan.split('').map((char, idx) =>
        <span
          key={idx}
          className={`w-7 h-8 flex items-center justify-center
            ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
              idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
              'bg-slate-800 text-slate-500'}
          >
            {char}
          </span>
        )}
      )}
    </span>
  </div>
  {currentIndex >= textToScan.length && (
    <span className="bg-emerald-600 text-white text-sm font-medium" >
      ГОТОВО ✓
    </span>
  )}

```



```

        border-blue-400 text-blue-200 font-bold' :
          <span>3. curr = trie[curr].get(char)
            {activeSearchCodeLine === 3 && <span>
ем по течению</span>}
        </div>
        <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 font
          <span>4. if (is_terminal[curr]) report
            {activeSearchCodeLine === 4 && <span>
овпадение!</span>}
          </div>
        </div>
      </div>

    <div>
      <h5 className="text-slate-300 font-bold m
        <AlertCircle className="w-3.5 h-3.5 tex
      </h5>
      <div className="bg-slate-800 p-3.5 rounded
tems-center">
        {searchLog}
      </div>
    </div>
  </div>
</div>
))

```

```

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
  {/* Водопадные фоновые эффекты */}
  <div className="absolute inset-0 opacity-10 bg-
o-900 to-transparent pointer-events-none"></d
  <div className="absolute top-0 left-1/2 -transl
    <div className="w-8 bg-blue-400 h-full animat
    <div className="w-12 bg-blue-300 h-full anima
    <div className="w-6 bg-blue-500 h-full animat
  </div>

```

```

        <rect x="25" y={level.y - 12} width="14" height="10" fill="#94a0b4" />
      />
      <text x="32" y={level.y + 4} fill="#94a0b4" font-size="12" font-weight="bold">
        {level.label}
      </text>
    </g>
  )))

  /* Рендер прямых переходов (потоков воды) */
  Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
    const fromNode = WATERFALL_NODES[Number(fromId)];
    return Object.entries(children).map(([char, child]) => {
      const toNode = WATERFALL_NODES[toId];

      // Подсветка в режиме поиска
      let isHighlighted = activeMode === 'search' ? (char === toNode.label) :
      type === 'normal';

      // Подсветка в режиме обучения (когда ищем слово)
      let isTrainingExamined = activeMode === 'train' ? (char === toNode.label) :
      let isMatchBranch = isTrainingExamined & isHighlighted;

      return (
        <g key={`edge-${fromNode.id}-${toNode.id}-${char}`}>
          /* Линия течения воды */
          <line
            x1={fromNode.x}
            y1={fromNode.y}
            x2={toNode.x}
            y2={toNode.y}
            stroke={isHighlighted ? '#3b82f6' : '#ccc'}
            strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
            className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
          />
          /* Символ перехода */
          <circle cx={({fromNode.x + toNode.x}) / 2} cy={fromNode.y + 10} r="5" fill={
            isHighlighted ? (isMatchBranch ? '#10b981' : '#f43f5b') : '#ccc'
          />
          <text
            x={({fromNode.x + toNode.x}) / 2}
            y={fromNode.y + 10}
            font-size="12"
            font-weight="bold"
            fill={isHighlighted ? (isMatchBranch ? '#10b981' : '#f43f5b') : '#ccc'}
          />
        </g>
      );
    });
  });
}

export default WaterfallDiagram;

```

```

        let isJustEstablished = activeMode === 't
-ссылку

        // Вычисляем изогнутую кривую для красоты
        const dx = targetNode.x - node.x;
        const dy = targetNode.y - node.y;
        const cx = node.x + dx / 2 + (node.id % 2
        const cy = node.y + dy / 2 - 30;

        return (
          <g key={`fail-${node.id}`}>
            <path
              d={`M ${node.x} ${node.y} Q ${cx} $
              fill="none"
              stroke={isHighlighted ? '#f43f5e' :
              strokeWidth={isHighlighted || isJus
              strokeDasharray="6,6"
              className={isHighlighted || isJustE
              opacity={isHighlighted || isJustEst
            />
            {isJustEstablished && (
              <text x={cx} y={cy - 10} fill="#10b
            >
              fail[{node.label}] = {targetNode.
            </text>
          )}
        </g>
      );
    }
  }
}

{/* Рендер вершин (бассейнов водопада) */}
{Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id ===
  const isTargetChild = activeMode === 'tra
  const hasWords = node.words.length > 0;

  return (
    <g key={`node-${node.id}`} className="c

```

```

        </g>
    })

    { /* Индикатор словарных слов */
    {hasWords && {
        <g transform={`translate(${node.x +
            <circle cx="0" cy="0" r="9" fill=
            <text x="0" y="3" fontSize="10" f
                *
            </text>
        </g>
    }}
    </g>
    );
    }}}

{ /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
<g>
    { /* Круг-подложка под рыбу */
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bezi
        className="shadow-lg"
    />
    { /* Сама рыба */
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"
        style={{ transition: 'x 0.8s cubic-bezi
        className="animate-float select-none"
    >

```

```
    });  
};
```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 70/82: src/components/visualizers/PrefixSumViz.tsx
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ: 1000x1000

```
import React, { useMemo, useState } from "react";  
  
/**  
 * Префиксные суммы: 1D и 2D.  
 *  
 * Главная фишка – наведение на число:  
 * * в 1D наводим на P[i] – подсвечивается кусок массива  
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит  
 * * в 2D наводим на ячейку – подсвечивается прямоугольник  
 * а при выбранном запросе показывается формула включения-исключения  
 */  
  
const A1 = [3, 1, 4, 1, 5, 9, 2, 6];  
  
const A2 = [  
  [1, 2, 3, 4],  
  [5, 6, 7, 8],  
  [9, 1, 2, 3],  
  [4, 5, 6, 7],  
];  
  
const cellBase =  
  "flex items-center justify-center rounded-md font-mono text-sm",  
  "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";
```

```

const covered =
  hover?.kind === "pref"
    ? { from: 0, to: hover.i - 1 }
    : hover?.kind === "a"
      ? { from: hover.i, to: hover.i }
      : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
  <div className="space-y-5">
    <div className="overflow-x-auto pb-1">
      <div className="inline-block min-w-full">
        /* индексы */
        <div className="flex gap-1.5 mb-1 pl-[52px]">
          {A1.map((_, i) => {
            <div key={i} className={`${cellBase} !h-5`}>
              {i}
            </div>
          })}
        </div>

        /* массив a */
        <div className="flex gap-1.5 items-center mb-1">
          <div className="w-[46px] shrink-0 text-right">
            {A1.map((v, i) => {
              const inCover = covered && i >= covered.from && i <= covered.to;
              const inRange = i >= range.l && i <= range.r;
              return (
                <div
                  key={i}
                  onMouseEnter={() => setHover({ kind: "pref", i })}
                  onMouseLeave={() => setHover(null)}
                  onClick={() => setRange((r) => (i < r ? i : r))}
                  className={`${cellBase} cursor-pointer`}
                  inCover={inCover}
                  inRange={inRange}
                >
                  {v}
                </div>
              );
            })}
          </div>
        </div>
      </div>
    </div>
  </div>
);

```

```

        </div>
    </div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border border-slate-800" >
    {hover?.kind === "pref" ? (
        hover.i === 0 ? (
            <p className="text-slate-300">
                <b className="text-emerald-400">P[0] = 0</b> – начало
                отрезка, начинающегося с нуля.
            </p>
        ) : (
            <p className="text-slate-300">
                <b className="text-emerald-400">
                    P[{hover.i}] = {pref[hover.i]}
                </b>{" "}
                – это сумма всего, что набежало от начала
            <span className="font-mono text-indigo-300">
                a[0..{hover.i} - 1] = {A1.slice(0, hover.i)}
            </span>
            </p>
        )
    ) : hover?.kind === "a" ? (
        <p className="text-slate-300">
            <b className="text-indigo-400">
                a[{hover.i}] = {A1[hover.i]}
            </b>{" "}
            входит во все префиксы, начиная с{" "}
            <span className="font-mono text-emerald-300">
                a[0..{hover.i} - 1]
            </span>
            подвинуть границу запроса.
        </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор на любое число: у <b className="text-emerald-400">у</b>
            у <b className="text-indigo-400">a</b> – курсор на а
        </p>
    )
}
```

```

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">Исходная матрица
        <div className="inline-block">
          {A2.map((row, i) => {
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => {
                const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
                return (
                  <div
                    key={j}
                    onClick={() => setRect((r) => {
                      i < r.r1 || j < r.c1 ? { r1: rect.r1, r2: rect.r2, c1: rect.c1, c2: rect.c2 } : { r1: i, r2: i, c1: j, c2: j }
                    })}
                    className={` ${cellBase} cursor-pointer ${inRect ? "bg-indigo-500 text-white" : ""}`}
                  >{v}</div>
                );
              })}
            <div>{i}</div>
          })}
        </div>
      </div>
    </div>
  )
)

```



```

        ht">
            {corner}
        </span>
    })
</div>
);
}}
</div>
)}}
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border
    {hover ? {
        <p className="text-slate-300">
            <b className="text-amber-400">
                 $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hov$ 
            </b>{" "}
            – сумма всего прямоугольника от левого верх
        </p>
    } : {
        <p className="text-slate-500">Наведите курсор
    }}
</div>

<div className="bg-slate-900 border border-indigo
    <p className="font-mono text-sm sm:text-base te
        сумма = <span className="text-emerald-400">A<
        <span className="text-rose-400">C</span> + <s
        <span className="text-indigo-300 font-bold">{
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
        Вычли два «хвоста» и вернули дважды вычитенный
    </p>
</div>
</div>
);

```

```

    [25, 41, 16, 92, 9, 17, 56, 31],
    [59, 18, 77, 24, 61, 82, 35, 12],
    [73, 8, 38, 85, 47, 95, 19, 58],
    [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][] = Array.from({length: 8}, () =>
  Array.from({length: 8}, () =>
    Array.from({length: 4}, () =>
      Array(4).fill(0)
    )
  )
);

for (let r = 0; r < 8; r++) {
  for (let c = 0; c < 8; c++) {
    ST2D[r][c][0][0] = val2D[r][c];
  }
}

for (let kx = 0; kx <= 3; kx++) {
  for (let ky = 0; ky <= 3; ky++) {
    if (kx === 0 && ky === 0) continue;
    for (let r = 0; r + (1 << kx) <= 8; r++) {
      for (let c = 0; c + (1 << ky) <= 8; c++) {
        if (kx > 0) {
          ST2D[r][c][kx][ky] = Math.min(
            ST2D[r][c][kx-1][ky],
            ST2D[r + (1 << (kx-1))][c][kx-1][ky]
          );
        } else {
          ST2D[r][c][kx][ky] = Math.min(
            ST2D[r][c][kx][ky-1],
            ST2D[r][c + (1 << (ky-1))][kx][ky-1]
          );
        }
      }
    }
  }
}

```

```

        <Viz2DQuery key="2d_query" />
      )}
    </AnimatePresence>
  </div>
);
};

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }

  const maxStep = computeSteps.length;
  const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

  return (
    <div className="flex flex-col gap-5">
      <div className="flex items-center justify-between">
        <div>
          <h3 className="text-lg font-bold text-indigo-600">
            Построение Sparse Table (Min)
          </h3>
          <p className="text-sm text-slate-400">
            Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$ 
          </p>
        </div>
        <div className="flex gap-2">
          <button
            onClick={() => setStep(Math.max(0, step - 1))}
            className="p-2 bg-slate-800 hover:bg-slate-700"
            disabled={step === 0}
          />
        </div>
      </div>
    </div>
  );
};

```

```

</div>
<div className="mt-2 text-xs min-h-[36px]">
  {hover && covered ? (
    <p className="text-slate-300">
      <b className="text-sky-400">
        ST[{hover.i}][{hover.j}] = {stID[hover.i]}
      </b>{" "}
      – это минимум на отрезке{" "}
      <span className="font-mono text-sky-300">
        [{covered.from}, {covered.to}]
      </span>{" "}
      длины  $2^{\{hover.j\}} = \{1 \ll hover.j\} : \{ " " \}$ 
      <span className="font-mono text-slate-400">
        min({arrID.slice(covered.from, covered.to)}
      </span>
    </p>
  ) : (
    <p className="text-slate-500">
      Наведите курсор (или тапните) на любое число
    </p>
  )}
</div>
</div>

<div className="overflow-x-auto pb-2">
  <div className="inline-block min-w-full bg-slate-100">
    <div className="grid grid-cols-[auto_repeat(4,1fr)]">
      {/* Headers */}
      <div className="text-slate-500 font-mono text-sm">
        i \ j
      </div>
      <div className="text-center font-bold text-sm">
         $2^0$  (L=1)
      </div>
      <div className="text-center font-bold text-sm">
         $2^1$  (L=2)
      </div>
    </div>
  </div>

```

```

    )
    isSrc2 = true; // wait, no. Src2
    // Let's re-eval exact source:
    // We are asking if THIS cell [i][j]
    if (currentStep.j - 1 === j) {
      if (currentStep.i === i) isSrc1 =
      if (currentStep.i + (1 << (current
        isSrc2 = true;
    }
  }
}

return (
  <div key={j} className="relative fl
    {!isValid ? (
      <div className="w-[clamp(30px,9
    ) : (
      <motion.div
        onMouseEnter={() => isVisible
        onMouseLeave={() => setHover(
        onClick={() => isVisible && s
        initial={{ opacity: 0, scale:
        animate={{
          opacity: isVisible ? 1 : 0,
          scale: isVisible ? 1 : 0.8,
        }}
        className={`w-[clamp(30px,9vw
clamp(11px,3vw,17px)] font-bold transition-
        hover && hover.i === i && h
          ? "bg-sky-500 text-white
          : isActive
          ? "bg-indigo-500 text-whi
          : isSrc1
          ? "bg-emerald-500 text-r
          : isSrc2
          ? "bg-amber-500 text-r
          : isVisible
          ? "bg-slate-800 tex
          : "bg-transparent t

```

```

<div className="bg-slate-900 border border-slate-800" style={{
  {step > 0 && step <= maxStep ? (
    () => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][{c.j}]
          </span>{" "}
          = min(
            <span className="text-emerald-400 font-l">
              ST[{c.i}][{c.j - 1}]
            </span>
            ,
            <span className="text-amber-400 font-bo">
              ST[{c.i + half}][{c.j - 1}]
            </span>
          ) &rarr; min(
            <span className="text-emerald-400">{st1D[
            <span className="text-amber-400">
              {st1D[c.i + half][c.j - 1]}
            </span>
          ) ={" "}
          <span className="text-indigo-400 font-b">
            {st1D[c.i][c.j]}
          </span>
        </p>
      );
    })()
  ) : (
    <p className="text-slate-500 italic">
      Нажмите далее, чтобы увидеть, как блоки сво
    </p>
  )}
</div>
</div>
);

```



```

        </div>
      </React.Fragment>
    )
  }
}
</div>
</div>

<div className="flex-1 min-w-0 flex flex-col gap-4" >
  <div className="bg-slate-900 p-4 sm:p-6 rounded" >
    <h3 className="text-lg font-bold text-slate-300" >
      {k === 0 ? (
        <div className="text-slate-400 text-sm leading-6" >
          <p className="mb-4">При <span className="font-bold text-slate-300">
            ы имеют размер <span className="font-bold text-slate-300">
              <div className="bg-[#0a0f1e] rounded-xl p-4" >
                <span className="text-slate-500">// Ба
                <span className="text-indigo-400">ST</span>
              </div>
                <p className="mt-6 italic text-indigo-400">
                  <Play size={14} className="fill-indigo-400" />
                </p>
              </div>
            ) : (
              <div className="flex flex-col gap-4 animate-in" >
                <div className="bg-[#0a0f1e] rounded-xl p-4 text-slate-300 shadow-inner overflow-x-auto" >
                  <span className="text-slate-500">// Те
                  <span className="text-purple-400">int<
                    <span className="text-amber-300">1</span></span>);<br/><br/>
                  <span className="text-slate-500">// Кв
                  <span className="text-indigo-400">ST</span>
                  <span className="text-rose-400">
                    <span className="text-amber-300">1</span>],
                    <span className="text-blue-400">
                      <span className="text-amber-300">1</span>[k-
                      <span className="text-amber-300">1</span>]
                    </span>
                  <span className="text-emerald-400">
                    <span className="text-amber-300">1</span>[k-

```



```

        <div className="flex item
            <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
            <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
            </div>
            <div className="flex item
            <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
            <span className="text
[activeCell.r + prevL][activeCell.c + prevL
            </div>
        </>
    )
    }}{()}}
</div>

    <div className="pt-3 mt-3 border-t
    ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>
    ) : {
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
        Наведите курсор на матрицу {L}x{L
        </div>
    }}
</div>
    }}
</div>
</div>
</div>
);
};

```

```

    )))

    <div
      className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
      style={{
        top: `${(r1 / 8) * 100}%`,
        left: `${(c1 / 8) * 100}%`,
        marginTop: '8px',
        marginLeft: '8px',
        height: 'calc(' + (H/8*100) +
        width: 'calc(' + (W/8*100) +
      }}
    />

    {/* Показываем 4 угла в самой матрице */}
    <div className="absolute pointer-e
      shadow-[0_0_10px_#f43f5e]" style={{ top: `
      lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
    <div className="absolute pointer-e
      shadow-[0_0_10px_#3b82f6]" style={{ top: `
      width: `calc(`${(Ly / 8) * 100}% - 16px)`,
    <div className="absolute pointer-e
      ld-400 shadow-[0_0_10px_#10b981]" style={{
      8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
    <div className="absolute pointer-e
      00 shadow-[0_0_10px_#f59e0b]" style={{ top:
      00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
    </div>

    <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
    <label className="flex items-cen
      <span className="text-slate-
      <div className="flex gap-2">
        <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
        <input type="number" min={

```

```
"text-slate-500">// Выс: {Lx}</span><br/>
  <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b>
```

```
  <span className="text-slate-500">// 2.
  <span className="text-indigo-400">int<
  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">В запроса)</span><br/>
```

```
  &nbsp;&nbsp;&nbsp;<span className="text-rose-900">r1</span>][<span className="text-white bg-r
```

```
  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">во, чтобы правый край уперся в c2)</span><b>
  &nbsp;&nbsp;&nbsp;<span className="text-blue-500">r1</span>][<span className="text-white bg-b
```

```
  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">рх, чтобы нижний край уперся в r2)</span><b>
  &nbsp;&nbsp;&nbsp;<span className="text-emerald-500">nded">r2 - Lx + 1</span>][<span className="
```

```
  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">и влево от c2)</span><br/>
```

```
  &nbsp;&nbsp;&nbsp;<span className="text-amber-500">">r2 - Lx + 1</span>][<span className="text
    {'}}''));
  </div>
```

```
<div className="w-full flex justify-cent
  <div className="flex flex-col items-ce
    <h4 className="text-slate-400 font-bo
      Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{{kx}}][{{k
  </h4>
```

```
  <div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>
  {Array.from({length: matRows * ma
    const r = Math.floor(idx / ma
```

```

        <span className="text-blue-400 m
n>
        <span className="text-emerald-400
span>
            <span className="text-amber-400 m
            &nbsp;<span className="text-slate-400 m
            <span className="ml-3 text-emerald-400 m
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
        </div>
    </div>
</div>
)
}
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 72/82: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРOK: 137 | РАЗМЕР

```

        </div>
    </main>
</div>

<script>
    function setMode(mode) {
        document.getElementById('btn-mode-normal').cla
            ? 'px-3 py-1 rounded text-sm font-bold'
            : 'px-3 py-1 rounded text-sm font-bold'

        document.getElementById('btn-mode-wtf').cla
```

```

        if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
        if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
    }

    // Initialize theme and mode
    setTheme('dark');

    // Setup intersection observer for TOC highlighting
    document.addEventListener('DOMContentLoaded', () => {
        const mobileMenuBtn = document.getElementById('mobile-menu-btn');
        const mobileDrawer = document.getElementById('mobile-drawer');
        const closeDrawerBtn = document.getElementById('close-drawer-btn');
        const drawerOverlay = document.getElementById('drawer-overlay');

        function toggleDrawer() {
            const isClosed = mobileDrawer.classList.contains('hidden');
            if (isClosed) {
                mobileDrawer.classList.remove('hidden');
                if (drawerOverlay) {
                    drawerOverlay.classList.remove('hidden');
                    // Timeout allows display:block to take effect
                    setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
                }
                document.body.style.overflow = "hidden";
            } else {
                mobileDrawer.classList.add('hidden');
                if (drawerOverlay) {
                    drawerOverlay.classList.add('hidden');
                }
                document.body.style.overflow = "auto";
            }
        }

        mobileMenuBtn.addEventListener('click', toggleDrawer);
        closeDrawerBtn.addEventListener('click', toggleDrawer);
        drawerOverlay.addEventListener('click', toggleDrawer);
    });

```

```
                if (link.getAttribute('href') === href) {
                    link.classList.add('bg-');
                } else {
                    link.classList.remove('bg-');
                }
            });
        }
    });
}, observerOptions);

document.querySelectorAll('section[id^="questions"]')
    .forEach(section => {
        observer.observe(section);
    });
});
</script>
</body>
</html>
```

ФАЙЛ 73/82: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 10.5 КБ

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Пособие по алгебре: От пиццы к предикатам</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js"></script>
  <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.2/es5/tex-mml-chtml.js"></script>
  <script>
    window.MathJax = {
      tex: {
        inlineMath: [['$', '$'], ['\\(', '\\)']],
        displayMath: [['$$', '$$'], ['\\[', '\\]']],
      },
    };
  </script>
</head>
```

```

    }
    mjx-container[display="true"] {
      overflow-x: auto;
      overflow-y: hidden;
      max-width: 100%;
      scrollbar-width: none;
      -ms-overflow-style: none;
      -webkit-overflow-scrolling: touch;
    }
    mjx-container[display="true"]::-webkit-scrollbar
      display: none;
  }

  @media (max-width: 640px) {
    html { font-size: 13px; }

    .uno-card { width: 40px; height: 60px; font-size: 12px; }
    .uno-card::before, .uno-card::after { font-size: 12px; }
    .uno-card::before { top: 1px; left: 2px; }
    .uno-card::after { bottom: 1px; right: 2px; }

    .uno-card.mini { width: 28px !important; height: 28px !important; }
    .uno-card.mini::before, .uno-card.mini::after { font-size: 10px; }

    .uno-equation .text-2xl { font-size: 1.2rem; }
    .uno-equation.gap-4 { gap: 0.5rem !important; }

    /* Allow horizontal scroll for equations in formula scroll
    .formula-scroll { flex-wrap: nowrap !important; }
  }

  .uno-inner {
    position: absolute; width: 110%; height: 75%;
    border-radius: 50%; transform: rotate(-30deg);
    box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
  }

  .uno-val { position: relative; z-index: 2; }

```

```
        font-family: "Comic Sans MS", "Caveat", "Serif";
    }
    body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
        background-color: rgba(255, 255, 255, 0.7);
        border-color: #cbd5e1 !important;
        box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
    }
    body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #cbd5e1 !important; }
    body.theme-notebook .text-slate-400 { color: #546e7a; }
    body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 { border-color: #94a3b8 !important; }
    body.theme-notebook .font-mono { font-family: "Monospace"; }
    body.theme-notebook #top-controls { background-color: #f1f3f4; }

    /* CREEPY BLINKING */
    @keyframes creepy-blink {
        0%, 95%, 98%, 100% { transform: scaleY(1); }
        96.5% { transform: scaleY(0.1); }
    }
    .creepy-eye {
        transform-origin: center;
        transform-box: fill-box;
        animation: creepy-blink 4s infinite;
    }
    .creepy-eye-alt {
        transform-origin: center;
        transform-box: fill-box;
        animation: creepy-blink 5.2s infinite;
        animation-delay: 1.5s;
    }
    .creepy-emoji {
        display: inline-block;
        transform-origin: center;
        animation: creepy-blink 3.8s infinite;
    }
```



```

        <path d="M0,0 L6,3 L0,6 Z" fill="#f472b"
    </marker>
    <marker id="arrow-cyan" markerWidth="6" mar
        <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4b"
    </marker>
    <marker id="arrow-indigo" markerWidth="6" m
        <path d="M0,0 L6,3 L0,6 Z" fill="#818cf"
    </marker>
    <marker id="arrow-lime" markerWidth="6" mar
        <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63"
    </marker>
    <marker id="arrow-green" markerWidth="6" ma
        <path d="M0,0 L6,3 L0,6 Z" fill="#4ade8"
    </marker>

    <!-- ЛЕГО БЛОКИ -->
    <g id="lego-yellow">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <g id="lego-red">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <g id="lego-rem">
        <rect x="0" y="0" width="40" height="10"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <!-- МЯСОРУБКА -->
    <g id="meat-grinder">
        <path d="M 10 40 L 15 20 Q 25 15 35 20
        <rect x="5" y="40" width="40" height="20

```



```
        <a href="#question-5" class="er-fuchsia-500 pl-3 font-bold text-fuchsia-300">
```

```
        5. Группа корней из единицы
```

```
    </a>
```

```
    </summary>
```

```
    <div class="pl-6 space-y-2 text">
```

```
        <a href="#q5-def" class="block h">
```

```
        <a href="#q5-group" class="block h">
```

```
        <a href="#q5-prim" class="block h">
```

```
        <a href="#q5-crit" class="block h">
```

```
        <a href="#q5-cyclic" class="block h">
```

```
    </div>
```

```
</details>
```

```
<details class="group">
```

```
    <summary class="list-none cursor">
```

```
        <a href="#question-6" class="er-lime-500 pl-3 font-bold text-lime-300">
```

```
        6. Делимость, НОД и Евклид
```

```
    </a>
```

```
    </summary>
```

```
    <div class="pl-6 space-y-2 text">
```

```
        <a href="#q6-1" class="block h">
```

```
        <a href="#q6-2" class="block h">
```

```
        <a href="#q6-3" class="block h">
```

```
        <a href="#q6-4" class="block h">
```

```
        <a href="#q6-5" class="block h">
```

```
        <a href="#q6-6" class="block h">
```

```
    </div>
```

```
</details>
```

```
<details class="group">
```

```
    <summary class="list-none cursor">
```

```
        <a href="#question-7" class="er-pink-500 pl-3 font-bold text-pink-300">
```

```
        7. Взаимно простые числа
```

```
    </a>
```

```

        <a href="#q9-3" class="block">
        <a href="#q9-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
        10. Делимость и Схема Гильберта
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-rose-400">
        <a href="#q10-1" class="block">
        <a href="#q10-2" class="block">
        <a href="#q10-3" class="block">
        <a href="#q10-4" class="block">
        <a href="#q10-5" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
        11. НОД и НОК многочленов
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-cyan-400">
        <a href="#q11-1" class="block">
        <a href="#q11-2" class="block">
        <a href="#q11-3" class="block">
        <a href="#q11-4" class="block">
    </div>
</details>

```

```

        <div class="pl-6 mt-2 space-y-2">
            <a href="#q14-1" class="block">
            <a href="#q14-2" class="block">
            <a href="#q14-3" class="block">
                deg)</a>
                <a href="#q14-4" class="block">
            </div>
        </details>

        <details class="group">
            <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
                15. Производная. Критерий
            </a>
            </summary>
            <div class="pl-6 mt-2 space-y-2">
                <a href="#q15-1" class="block">
                <a href="#q15-2" class="block">
                <a href="#q15-3" class="block">
                <a href="#q15-4" class="block">
            </div>
        </details>

        <details class="group">
            <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
                16. Неприводимые многочлены
            </a>
            </summary>
            <div class="pl-6 mt-2 space-y-2">
                <a href="#q16-1" class="block">
                <a href="#q16-2" class="block">
                <a href="#q16-3" class="block">
            </div>
        </details>

```

```

        if (other !== detail) {
            other.removeAttr('aria-current');
        }
    });
}
});
});

// Scroll spy logic
document.addEventListener('DOMContentLoaded', () => {
    const sections = document.querySelectorAll(
        'h2, h3, h4, h5, h6'
    );
    const q5 = document.querySelector('#q5');
    const q6 = document.querySelector('#q6');
    const q7 = document.querySelector('#q7');
    const q13 = document.querySelector('#q13');
    const q14 = document.querySelector('#q14');
    const q15 = document.querySelector('#q15');

    let isScrolling = false;

    document.querySelectorAll('.group a').forEach(link => {
        link.addEventListener('click', () => {
            isScrolling = true;
            setTimeout(() => isScrolling = false, 200);
        });
    });

    const observer = new IntersectionObserver(entries => {
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersecting) {
                visibleId = entry.target.id;
            }
        });

        // Highlight active link
        document.querySelectorAll('a').forEach(a => {
            a.style.fontWeight = 'normal';
            if (a.href.endsWith(visibleId)) {
                a.style.fontWeight = 'bold';
            }
        });
    });
});

```

```
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a202c';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей браузеров  
некоторые веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
```

```

    }

    function downloadMD() {
        let clone = document.querySelector('#clone')

        // Smart markdown parsing based on marked.js
        // Smart markdown parsing based on marked.js
        clone.querySelectorAll('h2').forEach(el => {
        clone.querySelectorAll('h3').forEach(el => {
        clone.querySelectorAll('.analog').forEach(el => {
        clone.querySelectorAll('li').forEach(el => {
        clone.querySelectorAll('svg').forEach(el => {
            let caption = '';
            if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
                caption = el.nextElementSibling.alt || el.nextElementSibling.title;
            }
            el.textContent = '\n\n[ Изображение: ' + caption + ' ]\n\n';
        });
        clone.querySelectorAll('.img-caption').forEach(el => {
            el.remove();
        });

        let text = clone.innerHTML;
        text = text.replace(/\n{3,}/g, '\n\n');

        if (window !== window.top) {
            fallbackModal(text, 'Markdown');
            return;
        }

        const blob = new Blob([text], { type: 'text/markdown' });
        const url = URL.createObjectURL(blob);
        const link = document.createElement('a');
        link.href = url;
        link.download = 'vyisshaya_algebra.md';
        document.body.appendChild(link);
        link.click();
        document.body.removeChild(link);
    }
}

```



```
body.bg-white [class*=""]
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind
```

```
-sm font-bold flex items-center justify-center gap-4
      <span> </span> PDF
    </a>
    <button onclick="downloadMD()" type="button">
      -bold flex items-center justify-center gap-4
        <span> </span> Markdown
      </button>
      <button onclick="downloadHTML()" type="button">
t-sm font-bold flex items-center justify-center gap-4
      <span> </span> HTML
    </button>
  </div>

  <!-- Смена темы -->
  <div>
    <div class="text-xs text-slate-500">
      <div class="flex items-center gap-2">
        <button id="theme-dark" onclick="toggleTheme('dark')" type="button">
          justify-center text-amber-300 hover:text-white
          line-amber-500" title="Темная тема"> </button>
        <button id="theme-light" onclick="toggleTheme('light')" type="button">
          r justify-center text-amber-50 hover:text-white
          ">* </button>
        <button id="theme-terminal" onclick="toggleTheme('terminal')" type="button">
          -center justify-center text-green-400 hover:text-white
          терминала"> </button>
      </div>
    </div>
  </div>
</div>

</div>
</aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12" style="min-height: 600px;">
  -base min-w-0">
    <header class="text-center mb-12 border-b border-slate-700">
      <h1 class="text-4xl md:text-5xl font-bl
```

```

        <div class="overflow-x-auto w-full pb-4">
          <div class="grid grid-cols-3 gap-4">
            <!-- Column headers -->
            <div class="bg-yellow-900/40 border-1" data-bbox="273 195 495 240">
              оля и кольца, целые числа</div>
            <div class="bg-orange-900/40 border-1" data-bbox="495 195 717 240">
              олько полиномов</div>
            <div class="bg-cyan-900/40 border-1" data-bbox="717 195 940 240">
              сные числа</div>

            <!-- Row 1: База -->
            <a href="#question-1" onclick="setMainQuestion(1)" data-bbox="273 240 495 495" class="border-yellow-500 hover:bg-slate-700 transition duration-200">
              <div class="text-xs text-yellow" data-bbox="273 240 495 285">
                <div class="font-bold text-slate" data-bbox="273 285 495 330">
                  </div>
              </div>
              <a href="#question-9" onclick="setMainQuestion(9)" data-bbox="495 240 717 495" class="border-orange-500 hover:bg-slate-700 transition duration-200">
                <div class="text-xs text-orange" data-bbox="495 240 717 285">
                  <div class="font-bold text-slate" data-bbox="495 285 717 330">
                    </div>
                </div>
                <a href="#question-2" onclick="setMainQuestion(2)" data-bbox="717 240 940 495" class="border-cyan-500 hover:bg-slate-700 transition duration-200">
                  <div class="text-xs text-cyan-500" data-bbox="717 240 940 285">
                    <div class="font-bold text-slate" data-bbox="717 285 940 330">
                      </div>
                  </div>

                  <!-- Row 2: Базовые операции / Евклид -->
                  <a href="#question-6" onclick="setMainQuestion(6)" data-bbox="273 495 495 750" class="border-yellow-500 hover:bg-slate-700 transition duration-200">
                    <div class="text-xs text-yellow" data-bbox="273 495 495 540">
                      <div class="font-bold text-slate" data-bbox="273 540 495 585">
                        </div>
                    </div>
                    <a href="#question-10" onclick="setMainQuestion(10)" data-bbox="495 495 717 750" class="border-orange-500 hover:bg-slate-700 transition duration-200">
                      <div class="text-xs text-orange" data-bbox="495 495 717 540">
                        <div class="font-bold text-slate" data-bbox="495 540 717 585">
                          </div>
                      </div>

```

```

<!-- Row 5: Корни / Уравнения -->
<div class="hidden md:block"></div>
<a href="#question-13" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
<div class="text-xs text-orange
<div class="font-bold text-slate
</a>
<a href="#question-4" onclick="setM
-l-4 border-cyan-500 hover:bg-slate-700 trans
<div class="text-xs text-cyan-5
<div class="font-bold text-slate
</a>

<!-- Row 6: Кратность -->
<div class="hidden md:block"></div>
<a href="#question-14" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
<div class="text-xs text-orange
<div class="font-bold text-slate
</a>
<div class="hidden md:block"></div>

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>
<a href="#question-15" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
<div class="text-xs text-orange
<div class="font-bold text-slate
</a>
<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

<a href="#question-17" onclick="setM
-l-4 border-teal-500 hover:bg-slate-700 tra
<div class="text-xs text-teal-5

```

```
</div>
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
  <div class="flex items-center mb-6">
    <div class="bg-red-500/20 p-3 r
      <svg class="w-8 h-8 text-re
rg/2000/svg">
        <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
      </svg>
    </div>
    <h2 class="text-3xl pr-4 border
  </div>

  <div class="prose prose-invert max-w
    <p class="text-lg">
      Хватит тупить. Доказательств
есть для того, чтобы жать на кнопки. Вот жес
    </p>

    <h2 class="text-2xl font-bold b
(Меметика)</h2>

    <!-- Прямое -->
    <div class="bg-slate-800/80 p-6
      <h3 class="text-xl text-blur
      <p class="text-sm text-slate
      <ul class="list-disc pl-5 sp
        <li><span class="text-w
k$).</li>

        <li><span class="text-w
        <li><span class="text-w
ово.</li>

      </ul>
    </div>
```

```

        </ul>
    </div>

    <!-- Двойная делимость -->
    <div class="bg-slate-800 p-6" style="background-color: #2d3748; color: white;">
        <h3 class="text-xl text-orange-400" style="margin: 0;">
            <p class="text-sm text-slate-300" style="margin: 0;">
                <ul class="list-disc pl-5" style="margin: 0;">
                    <li><span class="text-white" style="font-size: 1.2em;">
                        вверх по уравнениям).</li>
                    <li><span class="text-white" style="font-size: 1.2em;">
                            рху вниз, показывая, что  $\Delta \vdots r_i$ 
                        <li><span class="text-white" style="font-size: 1.2em;">
                                х. Твой  $d$  больше.</li>
                </ul>
            </div>

            <h2 class="text-2xl font-bold text-white" style="margin: 10px 0;">
                ритмы экзекуции)</h2>

            <div class="grid grid-cols-1 md:grid-cols-2 gap-4" style="background-color: #2d3748; color: white; padding: 10px;">
                <div class="bg-slate-800 p-4" style="background-color: #2d3748; color: white; padding: 10px;">
                    <h4 class="text-red-400" style="margin: 0;">
                        <p class="text-sm text-white" style="margin: 0;">
                            <p class="text-xs font-white" style="margin: 0;">
                                со следующим. Если в конце 0 – пациент мертв
                            </div>

                            <div class="bg-slate-800 p-4" style="background-color: #2d3748; color: white; padding: 10px;">
                                <h4 class="text-red-400" style="margin: 0;">
                                    <p class="text-sm text-white" style="margin: 0;">
                                        <p class="text-xs font-white" style="margin: 0;">
                                            $c$. Пока получается 0 – корень жив. Как то
                                            корня минус один.</p>
                            </div>

                            <div class="bg-slate-800 p-4" style="background-color: #2d3748; color: white; padding: 10px;">

```

```

        <code class="text-x
    </div>
</li>

<li class="bg-slate-800/50
    <span class="text-2xl">
    <div>
        <h4 class="text-cyan
        <p class="text-sm t
есть сложение – есть ноль. Если есть умножен
        <code class="text-x
    </div>
</li>

<li class="bg-slate-800/50
    <span class="text-2xl">
    <div>
        <h4 class="text-cyan
        <p class="text-sm t
авь их драться. Умножение всегда врывается
        <code class="text-x
    </div>
</li>
</ul>
</div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====
```

```

        parts.push(Array.from(rules).map((r) => r.cssText));
    } catch {
        const href = (sheet as CSSStyleSheet).href;
        if (!href) continue;
        try {
            const res = await fetch(href);
            if (res.ok) parts.push(await res.text());
        } catch {
            /* чужой домен – пропускаем */
        }
    }
}
return parts.join("\n");
}

```

```

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
    border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem;
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
@media print {
    body { background: #fff; color: #0f172a; }
    .export-note, details { break-inside: avoid; }
    details { display: block; }
    details > div, details > p { display: block !important;

```



```

}

function glossarySection(termIds: string[]): string {
  if (!termIds.length) return "";
  const items = termIds
    .map((id) => {
      const e = GLOSSARY_BY_ID.get(id);
      if (!e) return "";
      return `<div class="export-term" id="term-${escapeHtml(id)}">
        <h3>${escapeHtml(e.title)}</h3>
        <p>${escapeHtml(e.short)}</p>
        ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
        ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
      </div>`;
    })
    .join("\n");

  return `<section class="export-gloss">
    <h2>Словарик терминов из этой темы</h2>
    <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
      ны сюда.</p>
    ${items}
  </section>`;
}

```

```

function wrap(opts: { title: string; subtitle: string; css: string; extraCss: string }): string {
  const stamp = new Date().toLocaleString("ru-RU");
  return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">

```

```

const num = title.match(/^s*(\d+)/)?[1];
const base = id.replace(/^[a-z0-9-]+/gi, "-").toLowerCase();
return `${num ? String(num).padStart(2, "0") + "-": ""}
}

```

/** Выгрузить одну главу. */

```

export async function downloadChapterHtml(chapter: Chapter) {
  const css = await collectCss();
  const { body, termIds } = prepareBody(chapter.content);
  const html = wrap({
    title: chapter.title,
    subtitle: chapter.category || "Универсальное пособие",
    css,
    body,
    gloss: glossarySection(termIds),
    origin: window.location.origin,
  });
  triggerDownload(html, safeName(chapter.title, chapter.id));
}

```

/** Выгрузить всё пособие одним файлом со сквозным оглавлением

```

export async function downloadBookHtml(chapters: Chapter[]) {
  const css = await collectCss();
  const allTerms: string[] = [];
  const bodies: string[] = [];

```

```

const toc = chapters

```

```

  .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a>
  .join("\n");

```

```

chapters.forEach((c) => {

```

```

  const { body, termIds } = prepareBody(c.content);

```

```

  termIds.forEach((t) => {

```

```

    if (!allTerms.includes(t)) allTerms.push(t);

```

```

  });

```

```

  bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="display:none">

```

```

<p style="text-align:right;font-size:.72rem;margin:0 0 10px 0">

```

```

  });

```

```
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"), "utf-8");
const registryBody = registry.slice(registry.indexOf("VIZUALIZATORY"), registry.length);
const vizIds = new Set([...registryBody.matchAll(/^\s{2}[a-zA-Z0-9_]+/g)].map(s => s.trim()));

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1])/10,
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
    image: /<img[\\s>]/i.test(html),
    interactive: vizIds.has(c.id),
    chars: html.length,
  };
});
```

ФАЙЛ 77/82: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossa",
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".m",
  external: ["react", "react-dom", "motion", "lucide-re",
});

const { chapters } = await import(path.join(ROOT, "tmp/"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
  `(?!<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
  "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
```

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, }

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
  if (!fs.existsSync(PAGES_DIR)) {
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
  }

  const pages = fs
    .readdirSync(PAGES_DIR)
    .filter((f) => f.endsWith(".png"))
    .sort();

  if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

  ensureDir(OUT_DIR);

  const doc = new PDFDocument({
    size: [A4.width, A4.height],
    margin: 0,
    autoFirstPage: false,
    info: {
      Title: "Универсальное пособие – полный исходный код",
      Author: "CI: rebuild-pdf",
    },
  });
```

Универсальное пособие • полный исходный код

```
-----
* Собирает ВСЕ исходный код репозитория в один текст
* public/export/full_code_all_pages.txt
*
* Список файлов берётся из git (чтобы ничего не забыть
* с фоллбэком на обход файловой системы, если git недо
*/
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человечитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в
  [/^src\/components\/$/, "Интерактивные компоненты и си
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+\$/ , "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re

/** Порядок разделов в документе. */
```

```

    .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
    .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
    .filter((rel) => CODE_EXT.has(path.extname(rel)))
    .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
    .sort((a, b) => {
      const ia = ORDER.indexOf(categoryOf(a));
      const ib = ORDER.indexOf(categoryOf(b));
      return ia !== ib ? ia - ib : a.localeCompare(b);
    });
  }

/** Достаём заголовки глав из данных пособия (без запуска)
function extractChapters(files) {
  const chapters = [];
  const seen = new Set();
  for (const rel of files.filter((f) => f.startsWith("src/"))) {
    const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
    const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
    let m;
    while ((m = re.exec(src)) !== null) {
      const [, id, title] = m;
      if (seen.has(id)) continue;
      seen.add(id);
      chapters.push({ id, title, file: rel });
    }
  }
  return chapters.sort((a, b) => {
    const na = parseInt(a.title.match(/(\d+)/)?.[0] ?? 0);
    const nb = parseInt(b.title.match(/(\d+)/)?.[0] ?? 0);
    return na - nb;
  });
}

function main() {
  const files = listFiles();
  const chapters = extractChapters(files);
  ensureDir(OUT_DIR);
}

```

```

push();
push();

current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push(HR);
    push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
    push(HR);
    push();
  }
  const content = fs.readFileSync(path.join(ROOT, rel));
  const lines = content.split("\n");
  push(SUBHR);
  push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
  push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | `);
  push(SUBHR);
  push();
  push(content.replace(/\n+$/, ""));
  push();
  push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`файлов: ${files.length}`);
console.log(`глав: ${chapters.length}`);
console.log(`строк: ${txt.split("\n").length}`);
console.log(`выход: ${path.relative(ROOT, TXT_PATH)}`);
}

```



```
    font: "DejaVu-Sans-Mono",
    fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
    pointsize: 8,
    /** Максимум символов в строке (далее – жёсткий пере
    cols: 138,
    /** Строк содержимого на странице (+2 служебные: шапк
    rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toF
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
  for (const cmd of ["magick", "convert"]) {
    try {
      execFileSync(cmd, ["-version"], { stdio: "ignore"
      return cmd;
    } catch {
      /* пробуем следующий */
    }
  }
  throw new Error(
    "ImageMagick не найден. Установите его: sudo apt-ge
  );
}
```

```

    return parts;
}

function paginate(txt) {
    const source = txt.replace(/\r\n/g, "\n").split("\n");
    const pages = [];
    for (let i = 0; i < source.length; i += PAGE.rows) {
        pages.push(source.slice(i, i + PAGE.rows));
    }
    return pages;
}

async function main() {
    if (!fs.existsSync(TXT_PATH)) {
        throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
    }

    const im = imageMagickCmd();
    const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

    fs.rmSync(PAGES_DIR, { recursive: true, force: true });
    ensureDir(PAGES_DIR);

    const total = pages.length;
    const jobs = pages.map((lines, idx) => {
        const num = String(idx + 1).padStart(4, "0");
        const header = `Универсальное пособие • полный исходный код
        Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
        const body = [header, "-".repeat(PAGE.cols), ...lines];
        const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
        const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
        fs.writeFileSync(txtFile, body + "\n", "utf8");
        return { txtFile, pngFile, num };
    });

    const args = (job) => [
        "-density", String(PAGE.density),

```

```
-----
  console.log(`      выход:   ${path.relative(ROOT, PAGE)}
`);

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message);
  process.exit(1);
});
```

=====

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

ФАЙЛ 82/82: [.github/workflows/rebuild-pdf.yml](#)
КАТЕГОРИЯ: CI / автоматизация | СТРӨК: 128 | РАЗМЕР: 4.0 КБ

name: Rebuild code PDF

```
# Пайплайн пересборки PDF при каждом коммите:
#
#   исходный код  → full_code_all_pages.txt   (script)
#               |
#               → page-0001.png ... page-NNNN.png (artifacts)
#                               |
#                               → full_code_all_page.pdf
#
# Готовые TXT и PDF коммитятся обратно в ветку (их отдаст GitHub)
# промежуточные PNG сохраняются как artifacts запуска.
```

```
on:
  push:
    branches:
      - "**"
    paths-ignore:
      # чтобы коммит самого workflow не запускал бесконечно
      - "public/export/**"
```

```
sudo apt-get install -y --no-install-recommen
# На Ubuntu политика ImageMagick иногда запре
# разрешаем то, что нужно пайплайну (text -> p
for policy in /etc/ImageMagick-6/policy.xml /
    if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
    fi
done
convert -version
```

- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
EXPORT_DENSITY: \${github.event.inputs.densi
run: npm run export:png
- name: Step 3 – png → pdf
run: npm run export:pdf
- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
- name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`\${public}/export/full_code_all_pages
 echo "| \`\${public}/export/full_code_all_pages
 echo "| PNG-страниц | \$(ls tmp/export-pages